

CS222

Operating Systems

Lecture 11

Virtual Memory

(Section 100001)

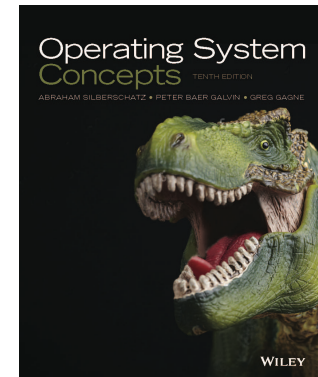
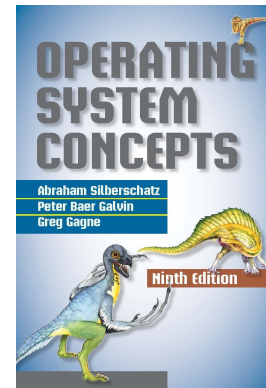
ผศ. ดร. กษิธิศ ชาญเขียว

ckasidit@tu.ac.th

Textbook

- Avi Silberschatz, Peter B. Galvin and Greg Gagne; Operating System Concepts, 9th Edition; John Wiley & Sons, Inc; 2012; ISBN 978-1118063330

- Chapter 10 (10th edition)
Virtual Memory



- Original Slides
- <https://www.os-book.com/OS9/slide-dir/index.html>
- <https://www.os-book.com/OS10/slide-dir/index.html>

Chapter 10: Virtual Memory

- Background
- Demand Paging
- Copy-on-Write
- Page Replacement
- Allocation of Frames
- Thrashing
- Memory-Mapped Files
- Other Considerations
- Operating-System Examples

Objectives

- Define virtual memory and describe its benefits.
- Illustrate how pages are loaded into memory using demand paging.
- Apply the FIFO, optimal, and LRU page-replacement algorithms.
- Describe the working set of a process, and explain how it is related to program locality.
- Design a virtual memory manager simulation in the C programming language.

Background

- Code needs to be in memory to execute, but entire program rarely used
 - Error code, unusual routines, large data structures
- Entire program code not needed at same time
- Consider ability to execute partially-loaded program
 - Program no longer constrained by limits of physical memory
 - Each program takes less memory while running -> more programs run at the same time
 - ▶ Increased CPU utilization and throughput with no increase in response time or turnaround time
 - Less I/O needed to load or swap programs into memory -> each user program runs faster

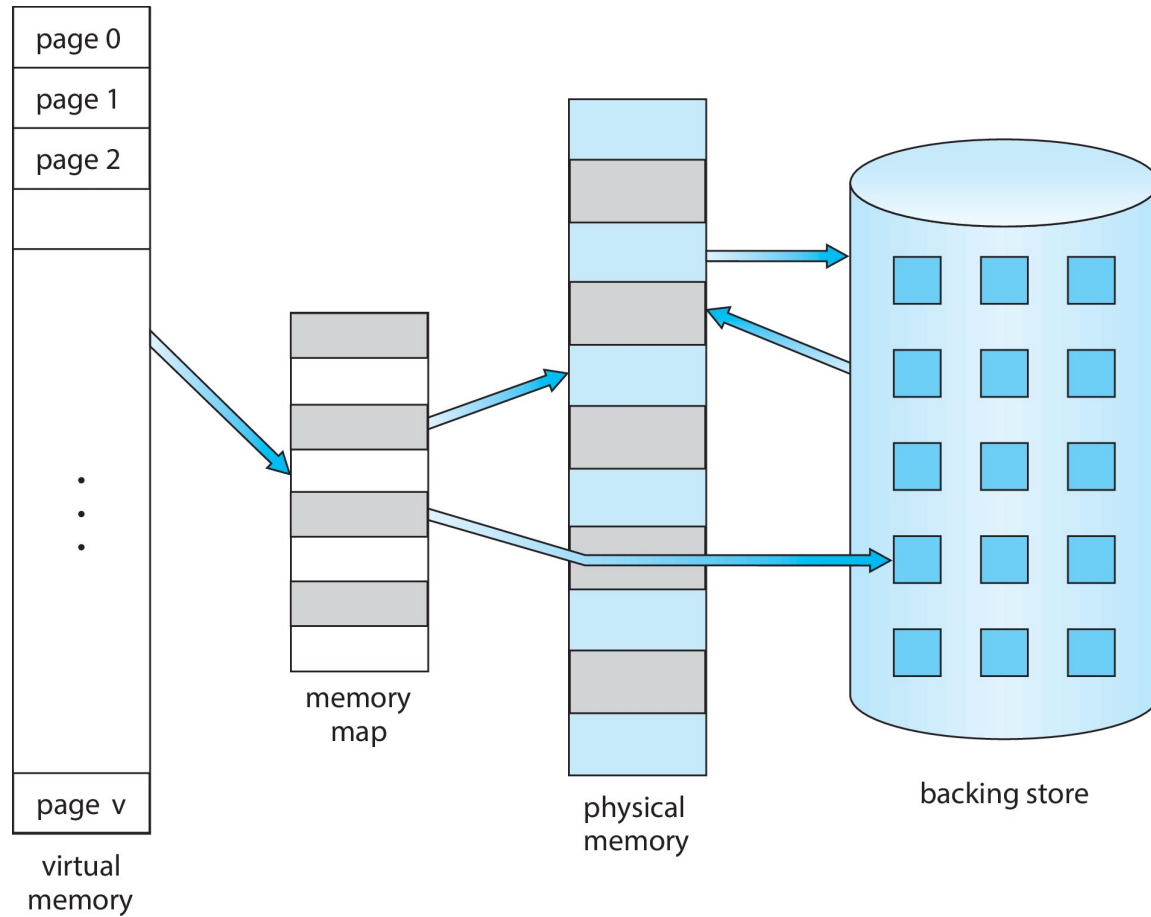
Virtual memory

- **Virtual memory** – separation of user logical memory from physical memory
 - Only part of the program needs to be in memory for execution
 - Logical address space can therefore be much larger than physical address space
 - Allows address spaces to be shared by several processes
 - Allows for more efficient process creation
 - More programs running concurrently
 - Less I/O needed to load or swap processes
 - ▶ เพราะ load หรือ swap เป็น pages ไม่ต้อง load หรือ swap ทั้งโปรแกรม

Virtual memory (Cont.)

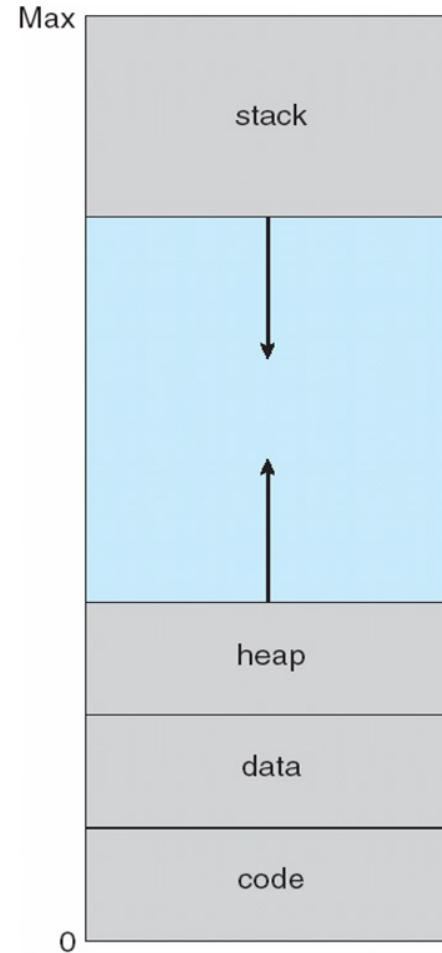
- **Virtual address space** – logical view of how process is stored in memory
 - Usually start at address 0, contiguous addresses until end of space
 - Meanwhile, physical memory organized in page frames
 - MMU must map logical to physical
- Virtual memory can be implemented via:
 - Demand paging
 - Demand segmentation

Virtual Memory That is Larger Than Physical Memory

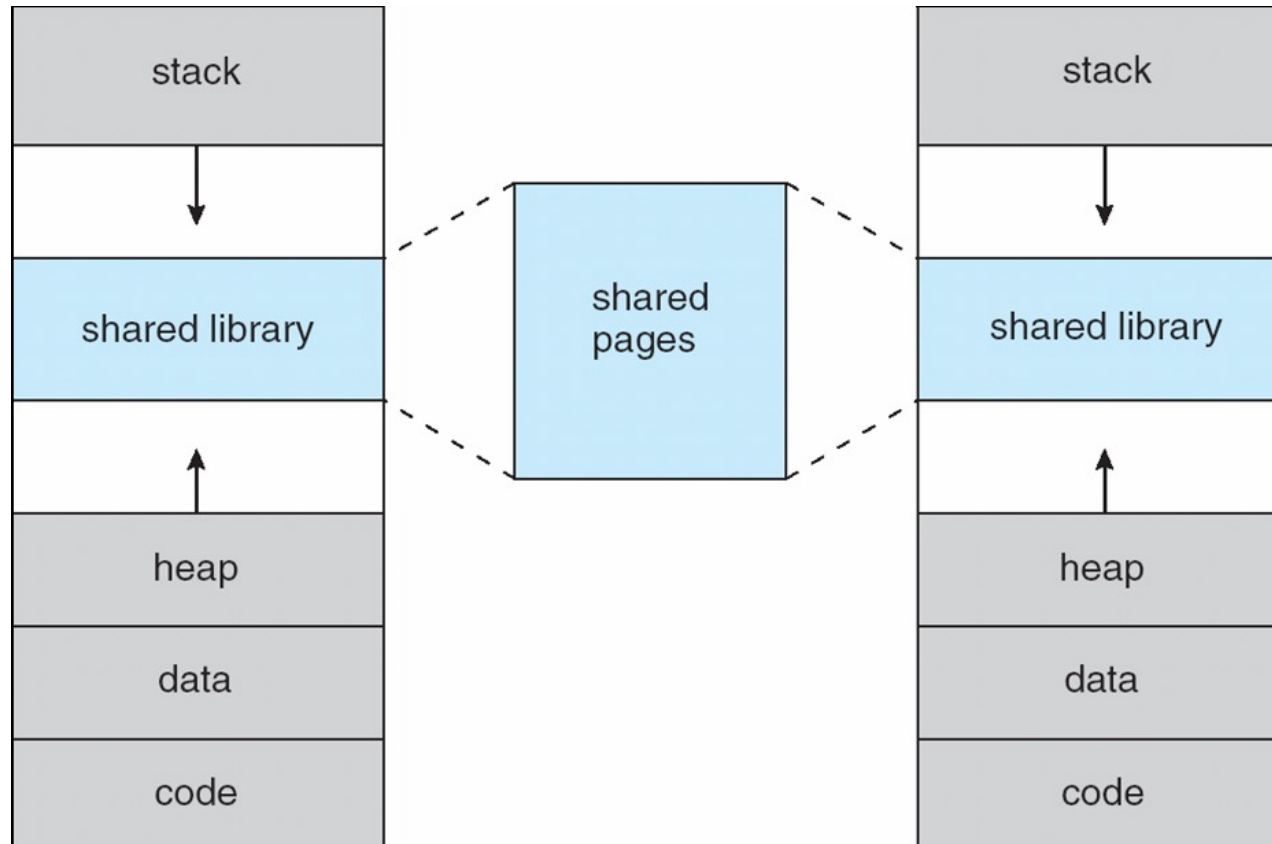


Virtual-address Space

- Usually design logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
 - Maximizes address space use
 - Unused address space between the two is hole
 - ▶ No physical memory needed until heap or stack grows to a given new page
- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc.
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during `fork()`, speeding process creation



Shared Library Using Virtual Memory



Demand Paging

- แต่ก่อนเรา bring entire process into memory at load time
- **Demand Paging** คือการ bring a page into memory only when it is needed
 - Less I/O needed, no unnecessary I/O
 - Less memory needed
 - Faster response
 - More users
- Page is needed $\Rightarrow$ reference to it
 - invalid reference $\Rightarrow$ abort
 - not-in-memory $\Rightarrow$ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
 - Swapper that deals with pages is a **pager**

Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (**v** $\Rightarrow$ in-memory – **memory resident**, **i** $\Rightarrow$ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:

Frame #	valid-invalid bit
	v
	v
	v
	i
...	
	i
	i

page table

- During MMU address translation, if valid–invalid bit in page table entry is **i** $\Rightarrow$ page fault

Page Table When Some Pages Are Not in Main Memory

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

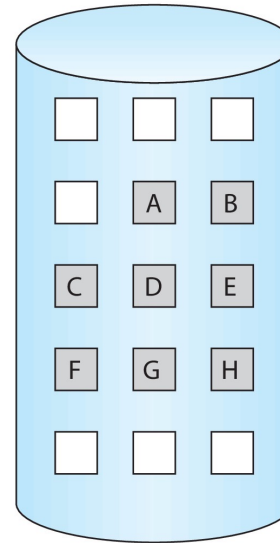
logical
memory

valid-invalid bit		
frame		
0	4	v
1		i
2	6	v
3		i
4		i
5	9	v
6		i
7		i

page table

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

physical memory

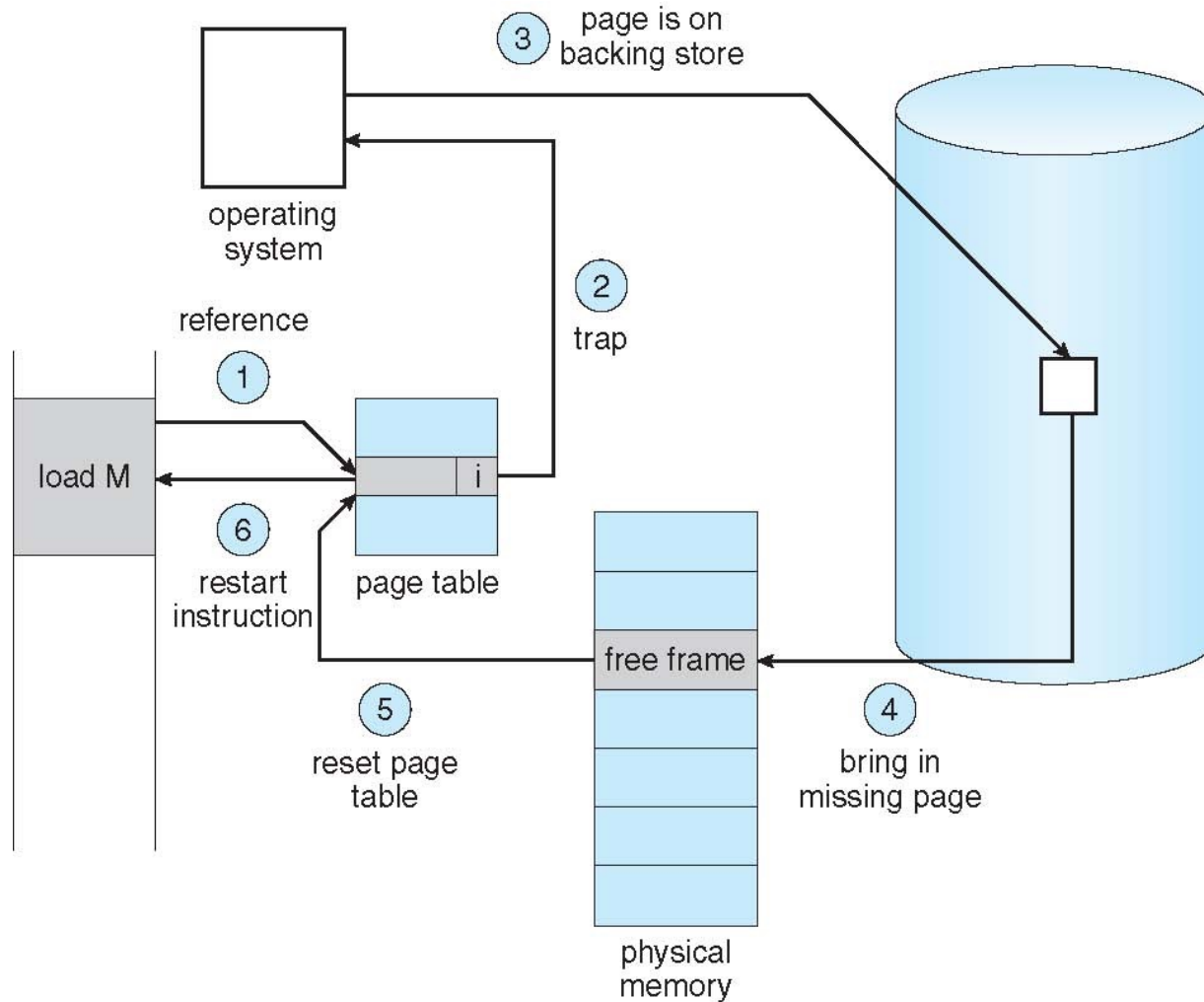


backing store

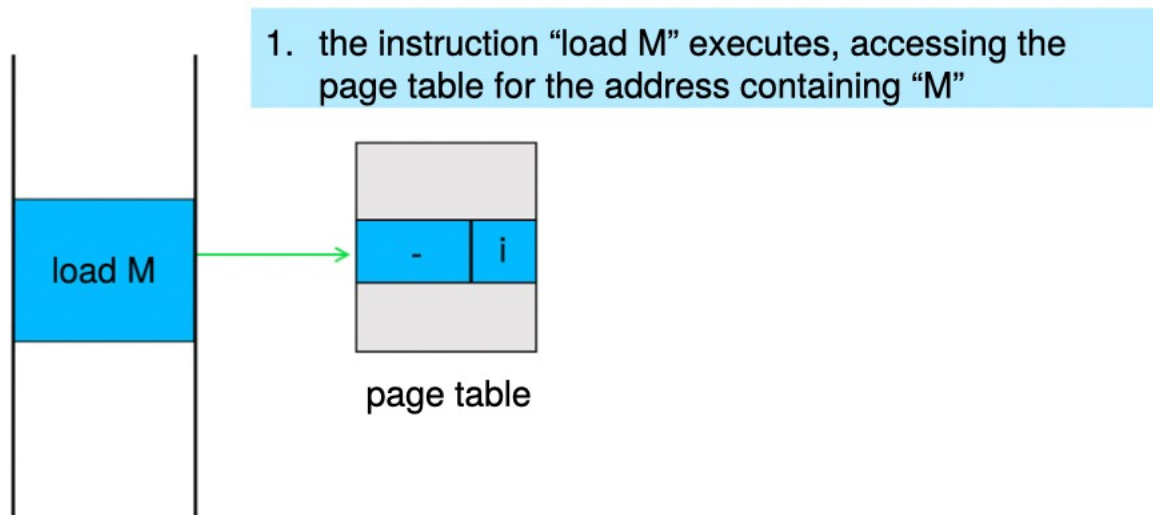
Steps in Handling Page Fault

1. If there is a reference to a page, first reference to that page will trap to operating system
 - Page fault
2. Operating system looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort
 - Just not in memory
3. Find free frame
4. Swap page into frame via scheduled disk operation
5. Reset tables to indicate page now in memory
Set validation bit = **v**
6. **Restart** the instruction that caused the page fault

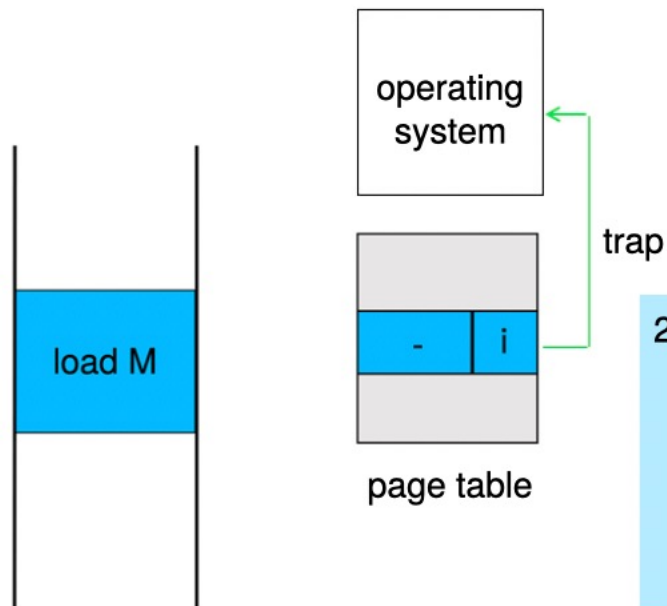
Steps in Handling a Page Fault (Cont.)



Steps in Handling a Page Fault

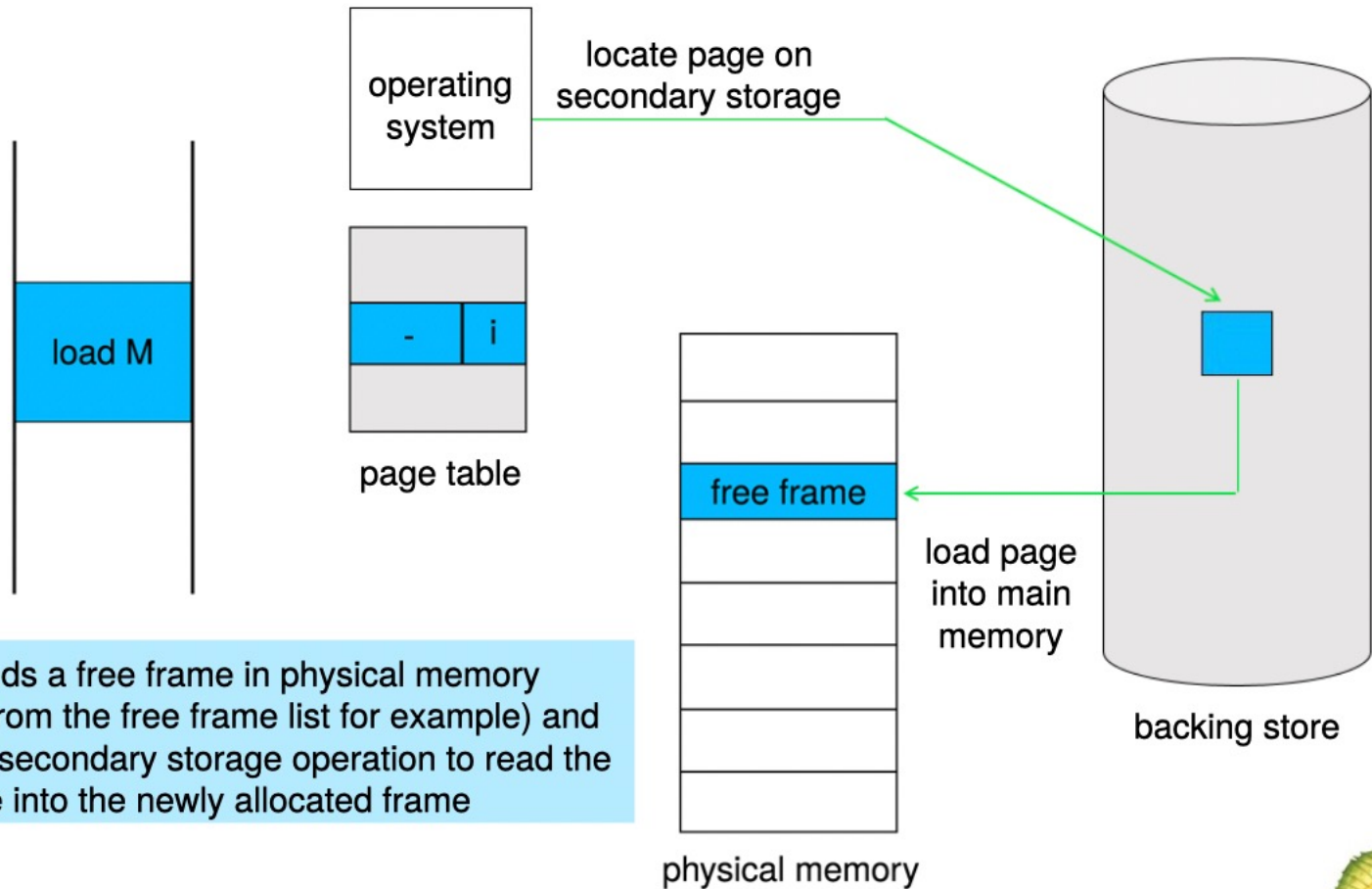


Steps in Handling a Page Fault



2. the page is marked as invalid, causing a page fault trap to the kernel
 - the operating system checks if the reference is legal or not
 - if it is illegal, the process is terminated
 - in this case it is valid but the page is not currently in a frame in physical memory

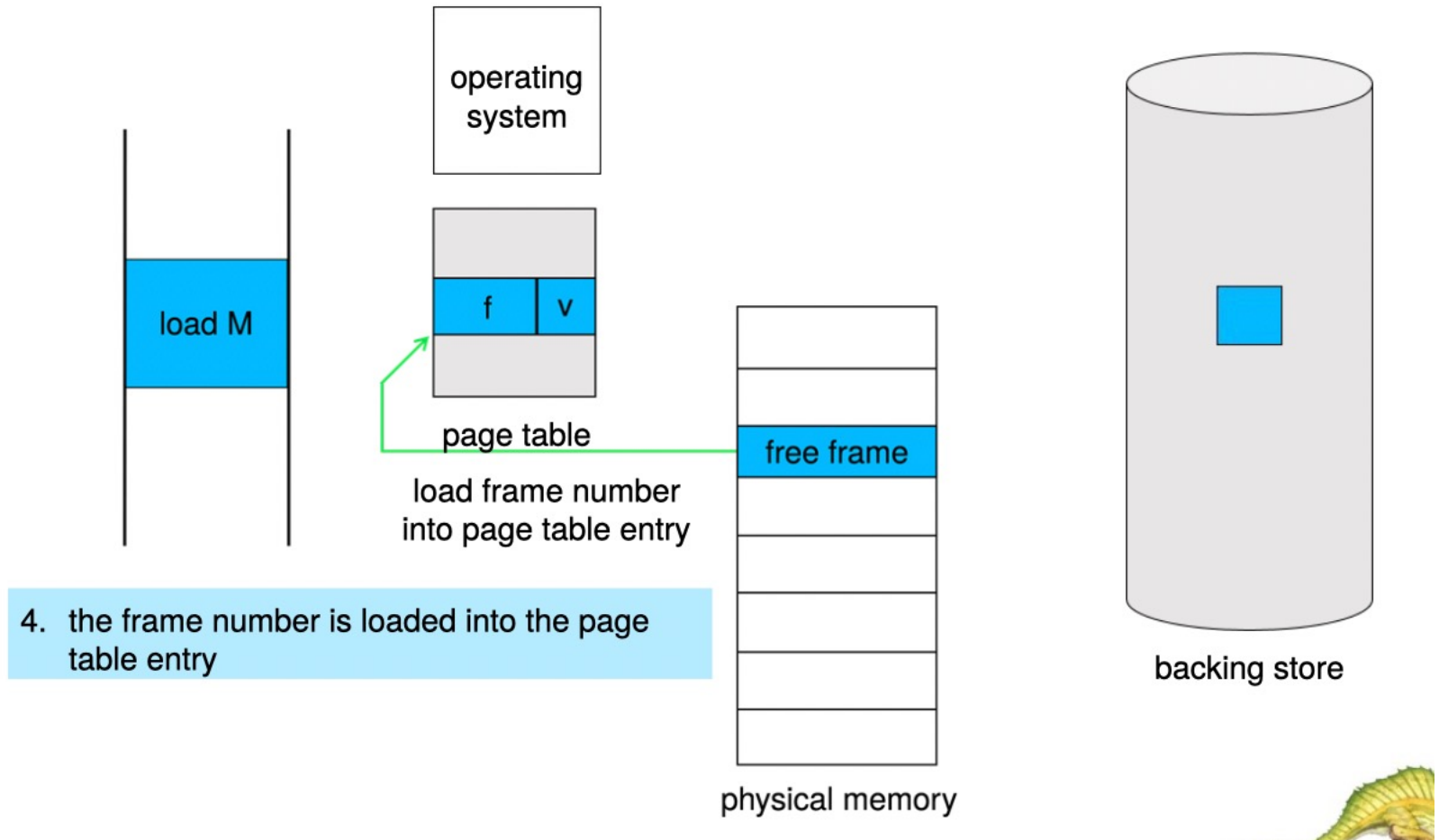
Steps in Handling a Page Fault



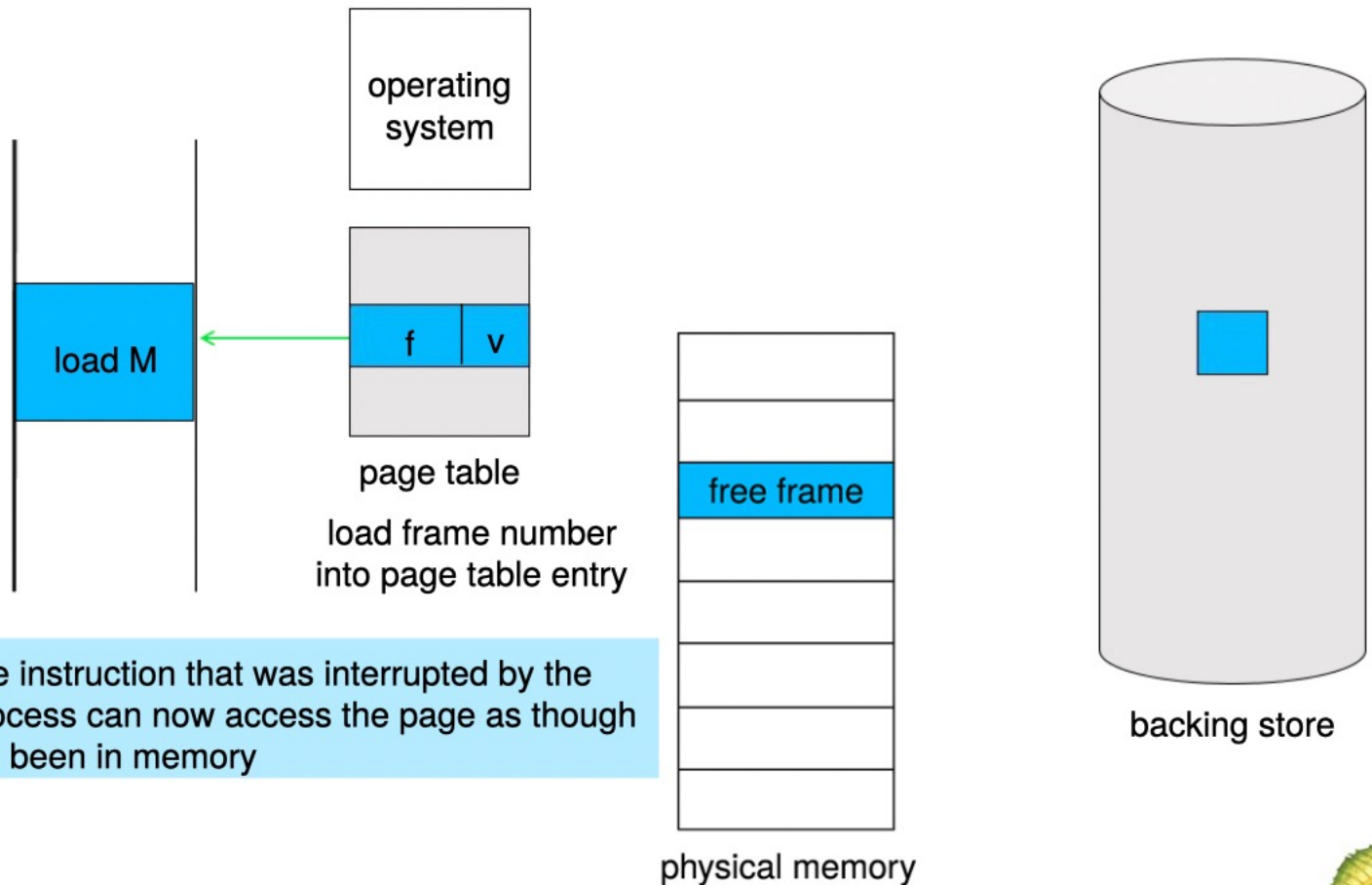
3. the kernel finds a free frame in physical memory (taking one from the free frame list for example) and schedules a secondary storage operation to read the desired page into the newly allocated frame



Steps in Handling a Page Fault



Steps in Handling a Page Fault



5. we restart the instruction that was interrupted by the trap – the process can now access the page as though it had always been in memory





จำลองการเกิด **Page Fault** บน **x86_64** **4-level Heirarhical Page Table**

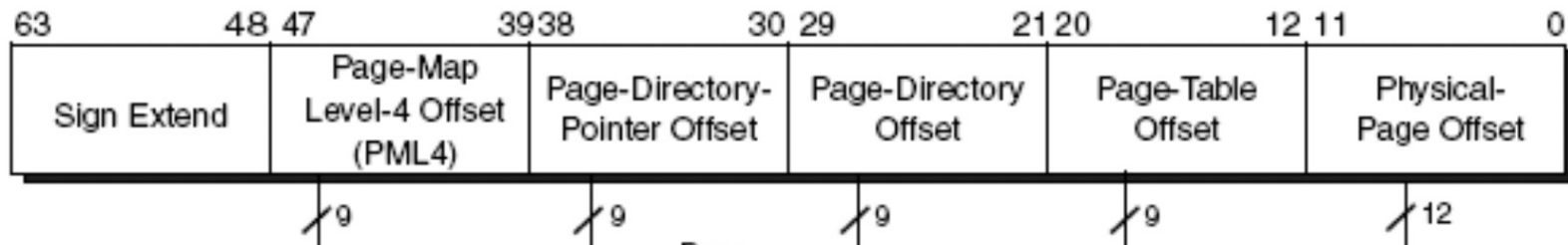




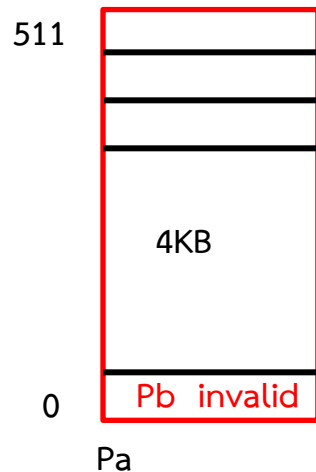
$$VA = 0x\ 10 = 0b\ 0001\ 0000$$

000₈ 000₈ 000₈ 000₈ 0020₈

Linear/Virtual Address

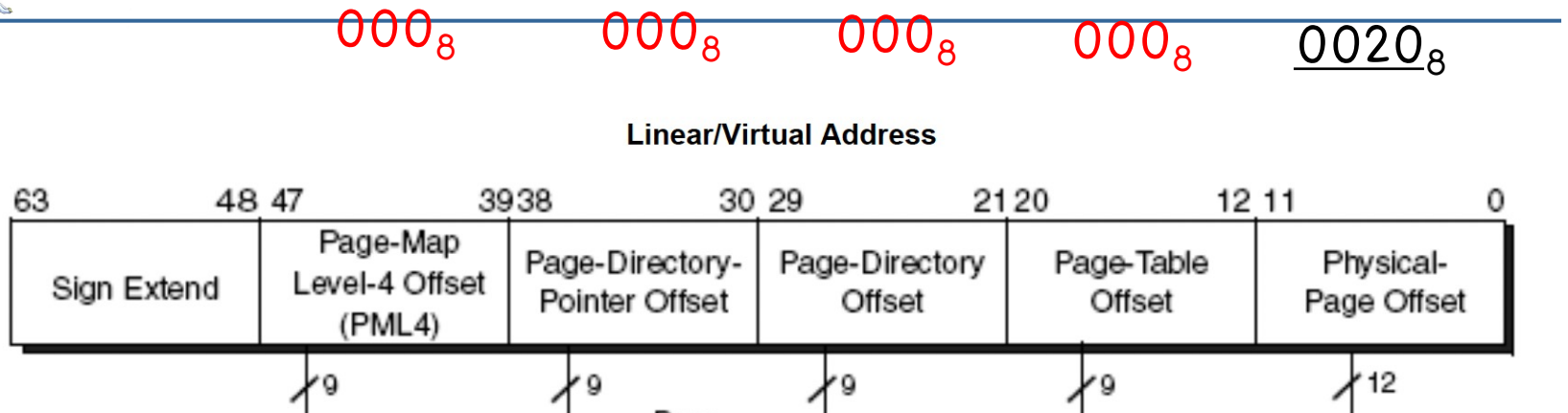


8 Bytes

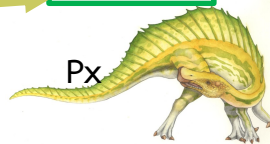
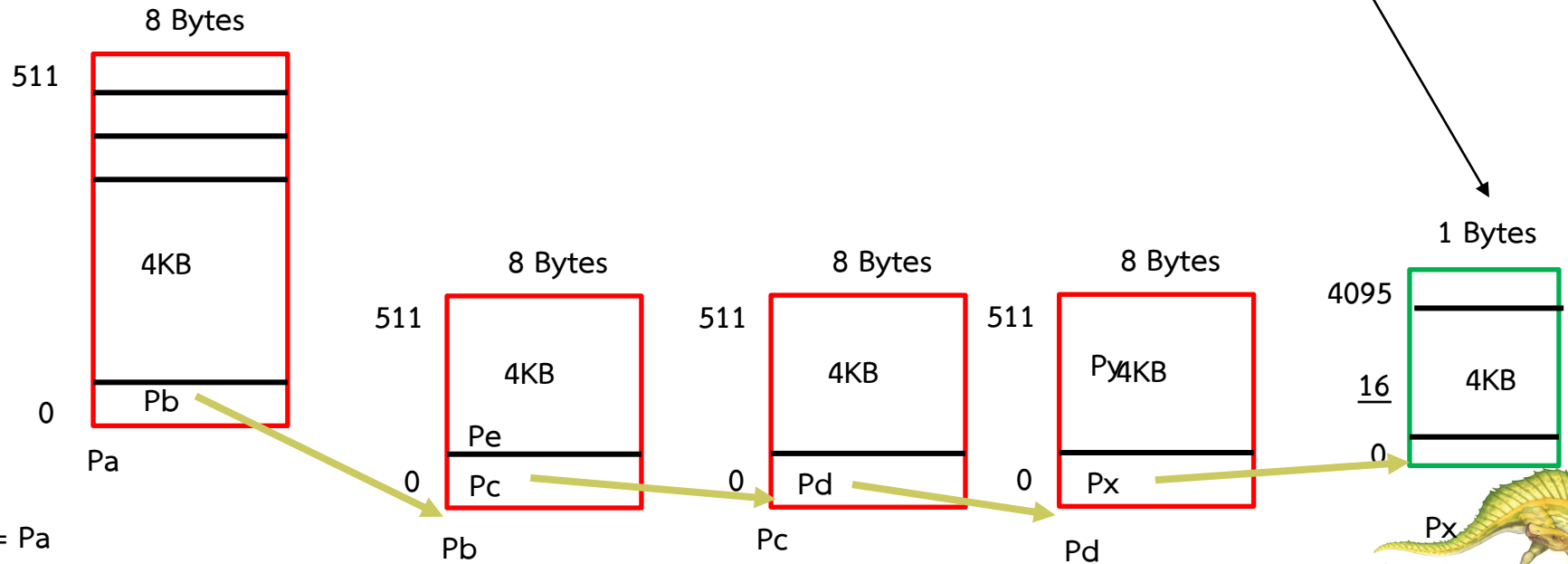




$$VA = 0x\ 10 = 0b\ 0001\ 0000 = 0o\ 20$$



User Process Memory



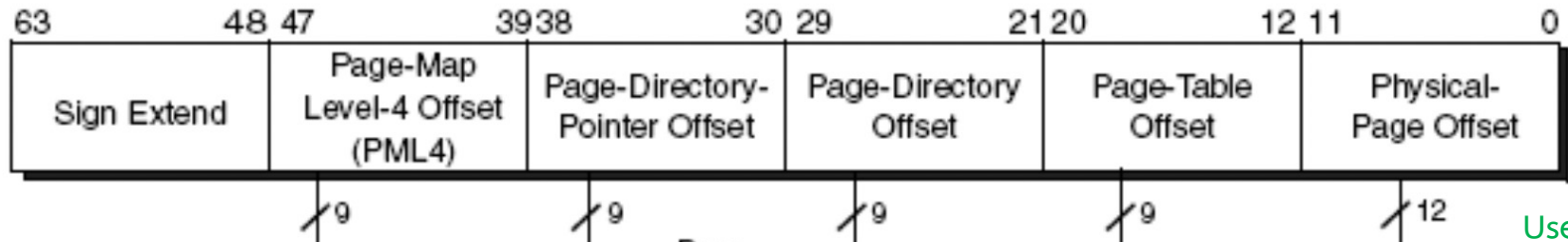
CR3 = Pa



$$VA = 0x\ 3000 = 0b\ 0011\ 0000\ 0000\ 0000 = 0o\ 30000$$

000₈ 000₈ 000₈ 003₈ 0000₈

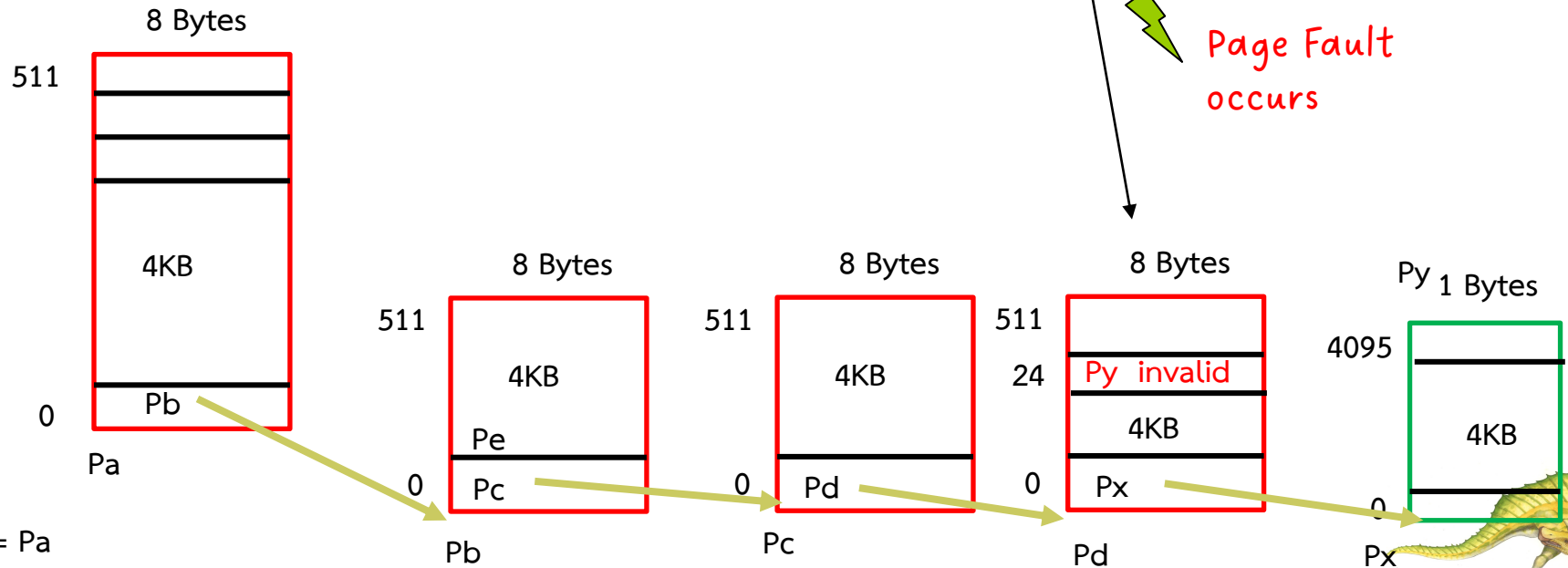
Linear/Virtual Address



User Process
Memory



Page Fault
occurs



CR3 = Pa

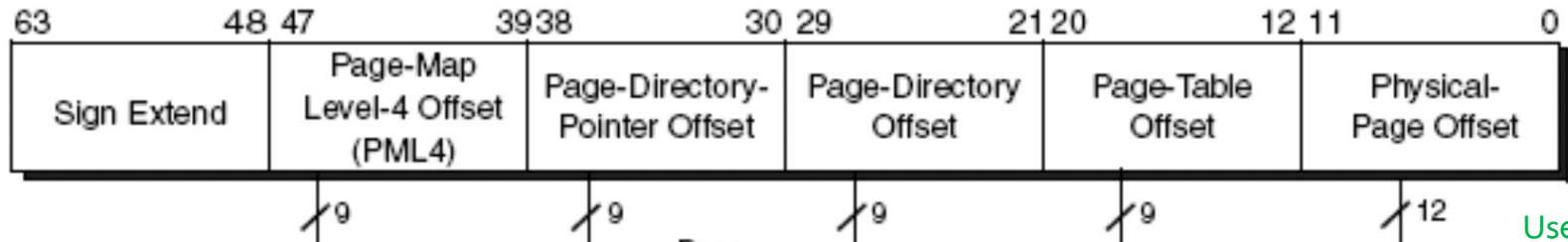




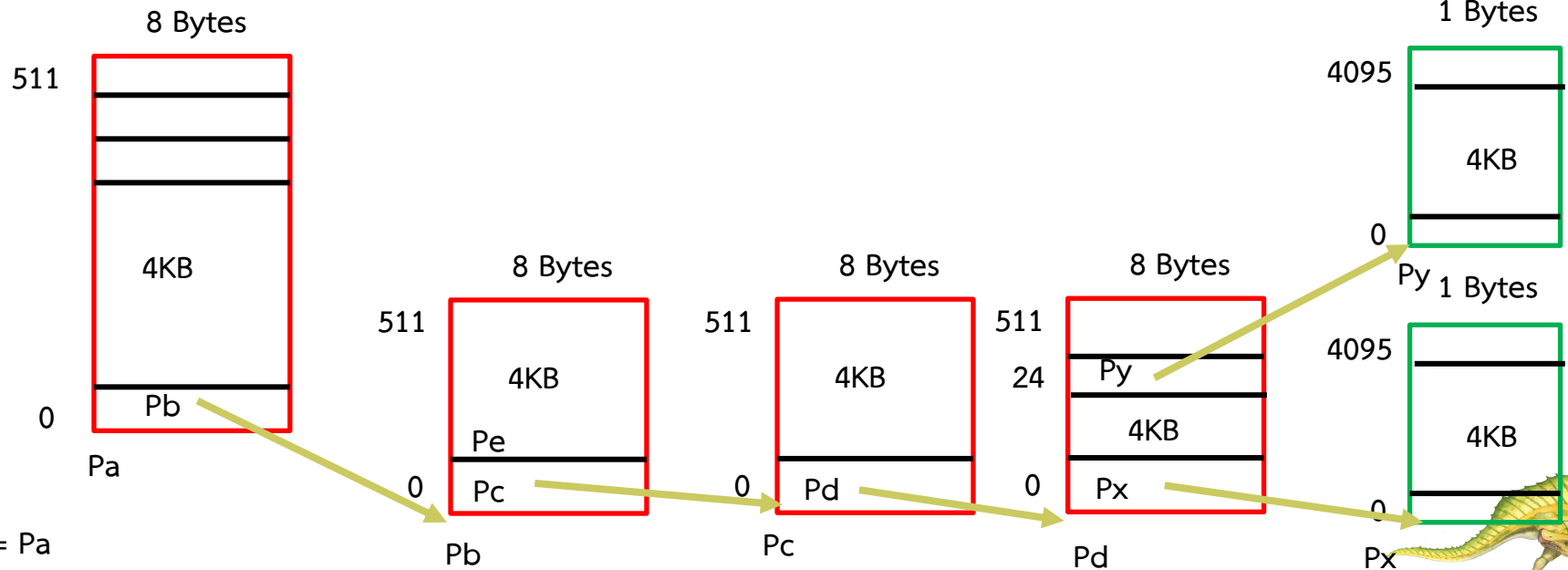
$$VA = 0x\ 3000 = 0b\ 0011\ 0000\ 0000\ 0000 = 0o\ 30000$$

000₈ 000₈ 000₈ 003₈ 0000₈

Linear/Virtual Address



User Process Memory



CR3 = Pa



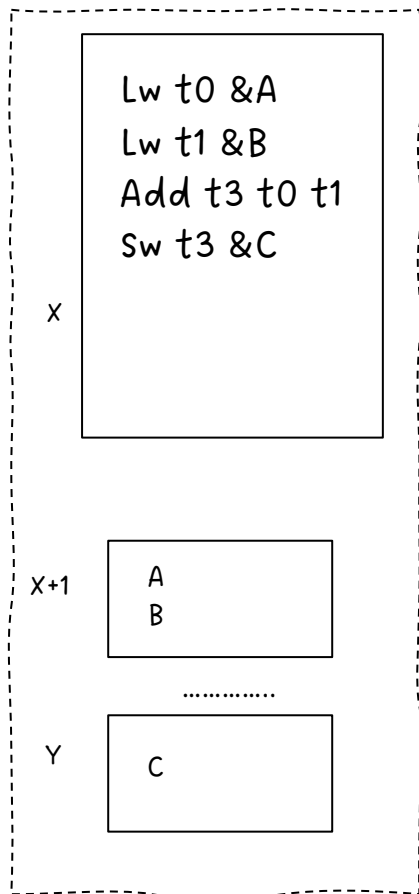
Aspects of Demand Paging

- Extreme case – start process with *no* pages in memory
 - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
 - And for every other process pages on first access
 - **Pure demand paging**
- Actually, a given instruction could access multiple pages -> multiple page faults
 - Consider fetch and decode of instruction which adds 2 numbers from memory and stores result back to memory
 - Pain decreased because of **locality of reference**
 - **locality of reference** คือการที่โปรเซสมีแนวโน้มที่จะใช้งานหน่วยความจำที่อยู่ใกล้กัน และมีแนวโน้มที่จะใช้คำสั่งที่ใช้บ่อยซ้ำๆกันในโปรแกรม

Aspects of Demand Paging

- **Hardware support** needed for demand paging
 - Page table with valid / invalid bit
 - Secondary memory (swap device with **swap space**)
 - Instruction restart
 - ▶ **Instruction restart** หมายถึงการรันคำสั่งที่ทำให้เกิด **Page Fault Trap** หลังจากการ **Page Fault handler** ทำงานเสร็จแล้ว
 - ▶ เมื่อ **Page Fault Handler** ทำงานเสร็จ และ **OS** เลือก **Process** ที่เกิด **Page Fault** เข้าใช้งานซีพียูอีกครั้งหนึ่ง
 - ▶ เมื่อซีพียูประมวลผล **Process** มันจะต้อง **Restart Instruction** ที่ทำให้เกิด **Page Fault** ไม่ใช่รันคำสั่งถัดไป เหมือนการเกิด **Trap/Interrupt** แบบอื่น
 - ▶ จำเป็นต้องมี **Hardware Support** มาช่วยกรณีนี้

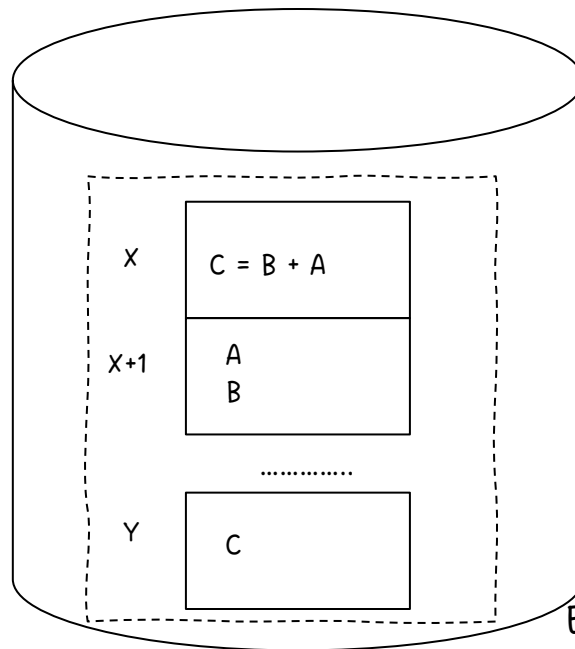
Pure Demand Paging



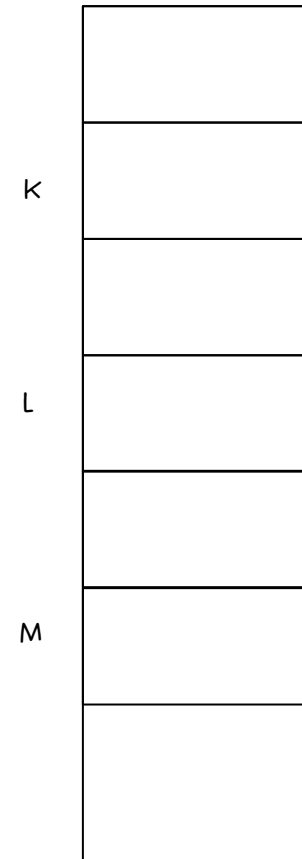
Virtual address space

Page Table

p	f.	v/i
x		1
x+1		1
y		i

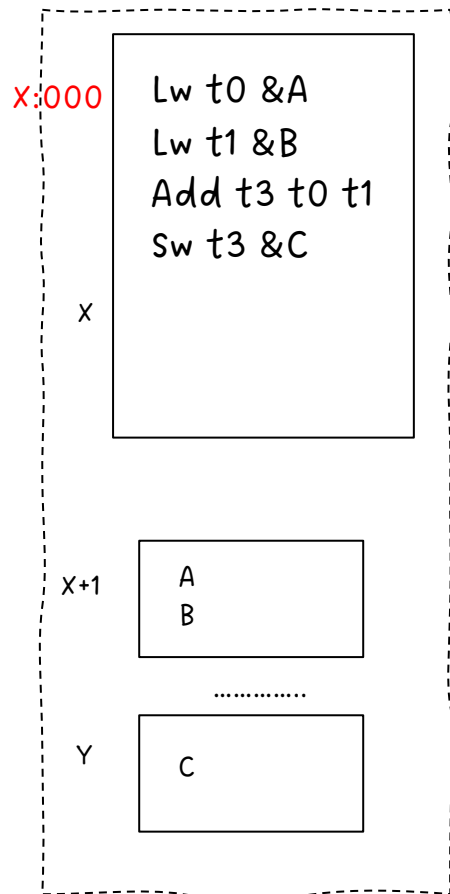


Executable file



Physical address space

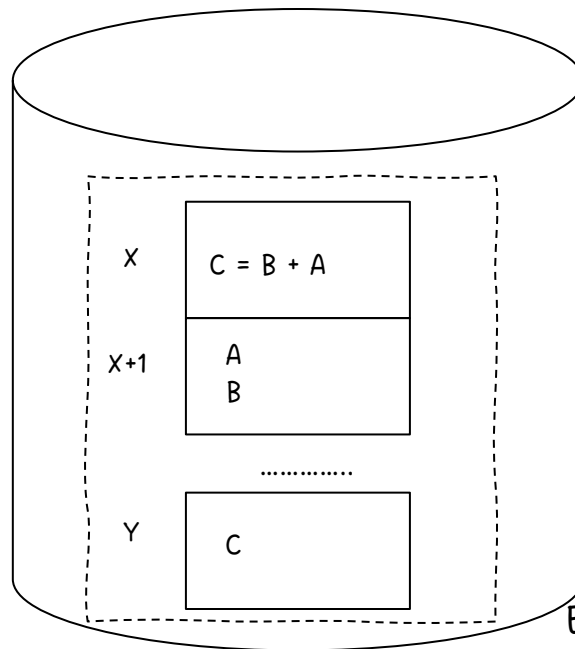
(1) เริ่มประมวลผล x:000 แต่พบ 1 ใน PT เกิด page fault



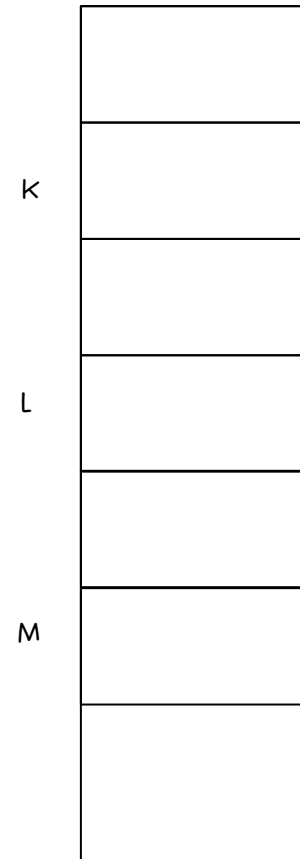
Virtual address space

Page Table

p	f.	v/i
x		1
x+1		1
y		i

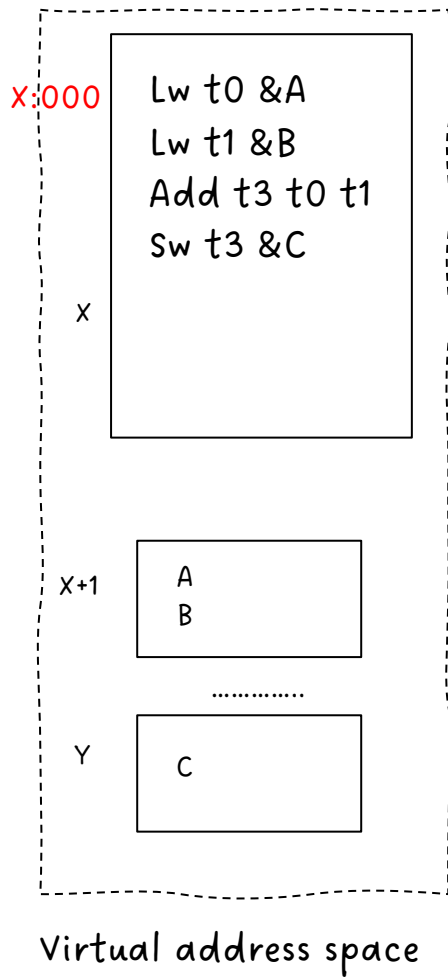


Executable file



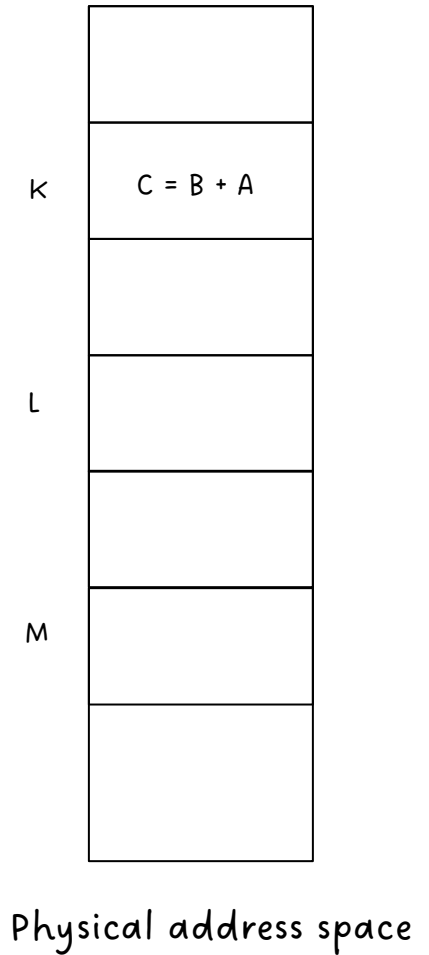
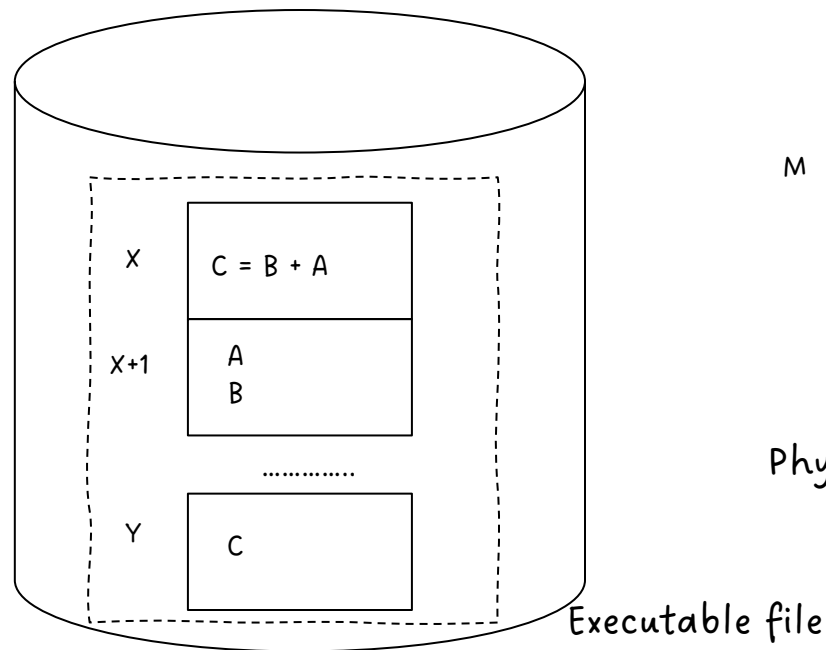
Physical address space

- (1) เริ่มประมวลผล x:000 แต่พบ I ใน PT เกิด page fault
- (2) อ่าน X จาก storage มาไว้ที่ frame K update PT

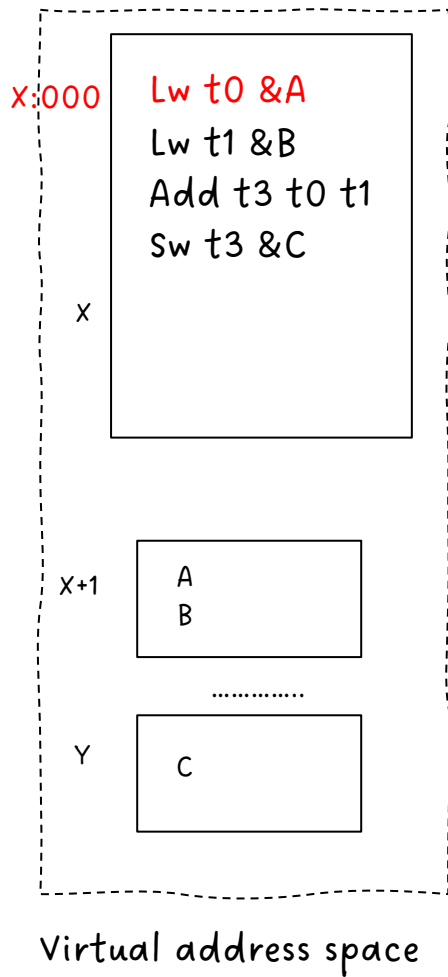


Page Table

p	f.	v/i
x	k	v
x+1		I
y		i

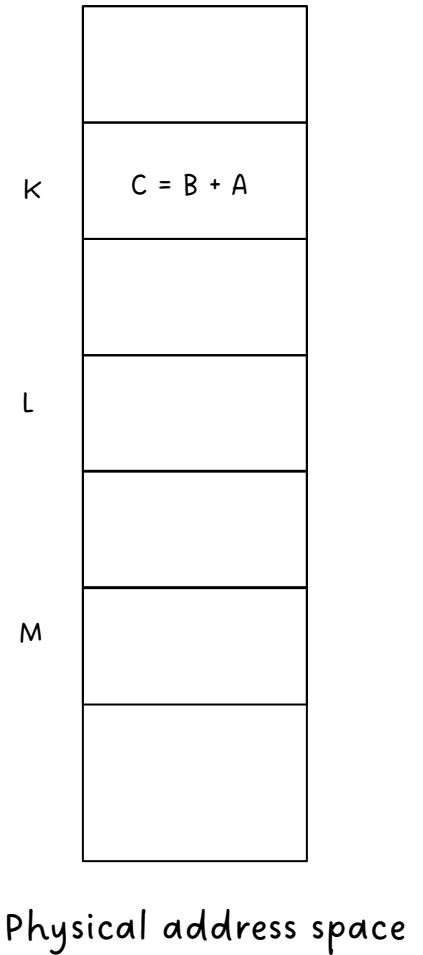
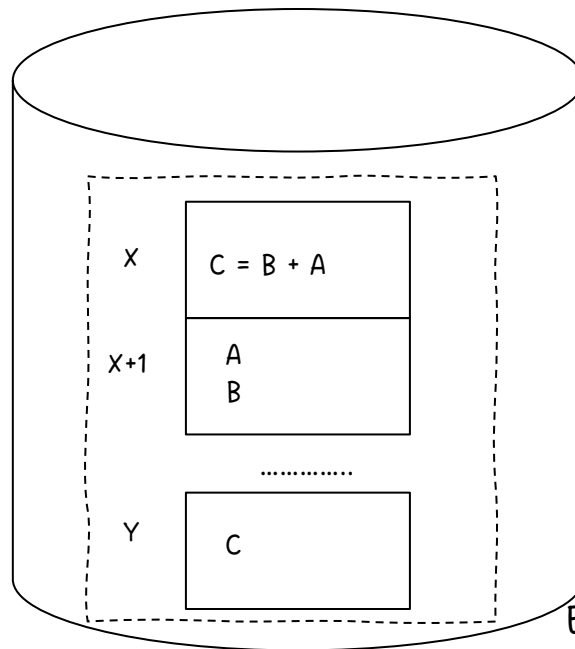


- (1) Restat คำสั่งที่ x:000
- (2) เกิด page fault เพราะ x+1 ไม่อยู่ใน phy mem



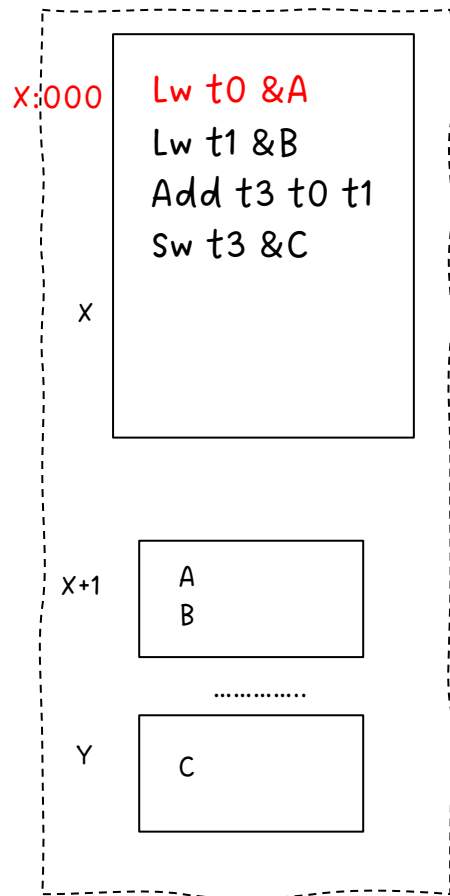
Page Table

p	f.	v/i
x	k	v
x+1		l
y		i



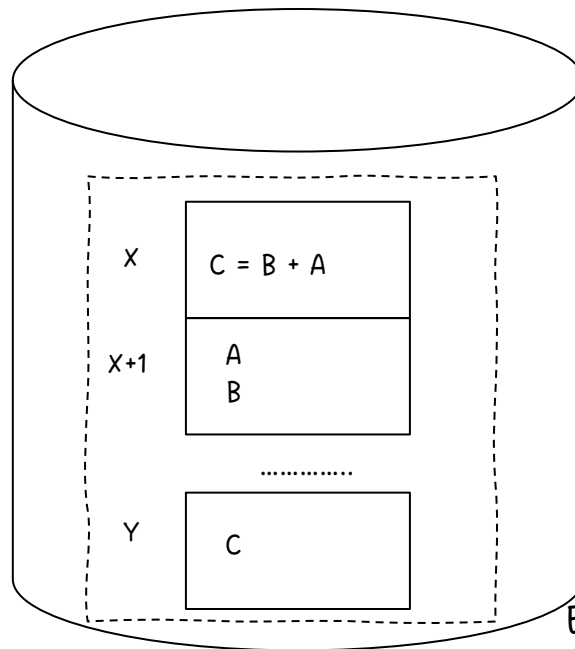
(1) Restat คำสั่งที่ x:000

(2) เกิด page fault เพราะ x+1 ไม่อยู่ใน phy mem ทำให้ OS เอา x+1 เข้ามาที่ L

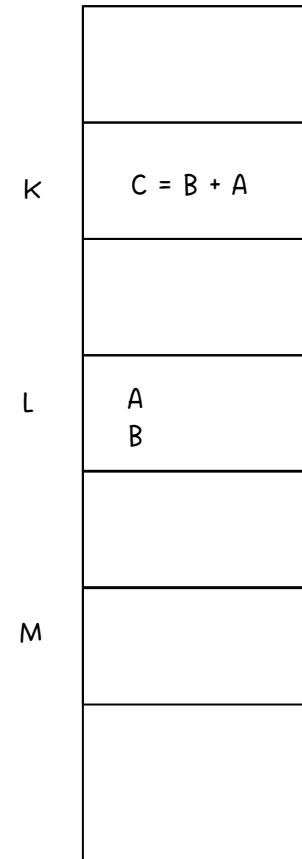


Virtual address space

Page Table		
p	f.	v/i
x	k	v
x+1	L	v
y		i

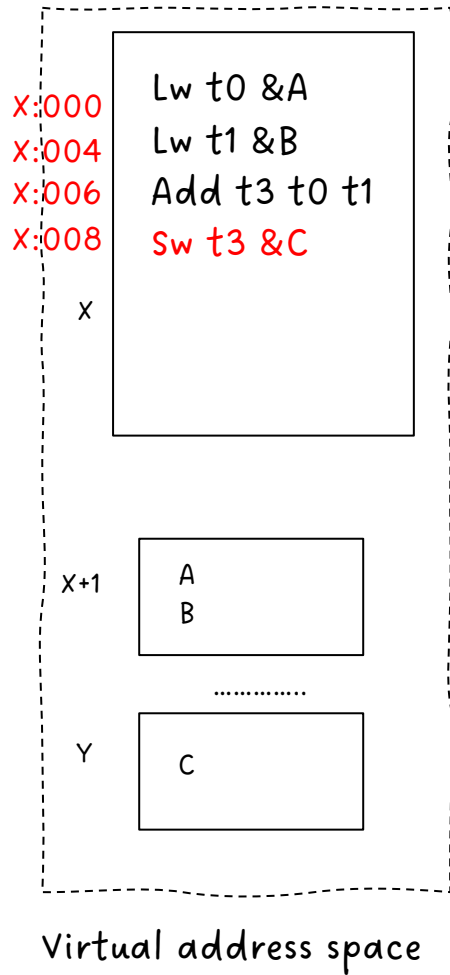


Executable file



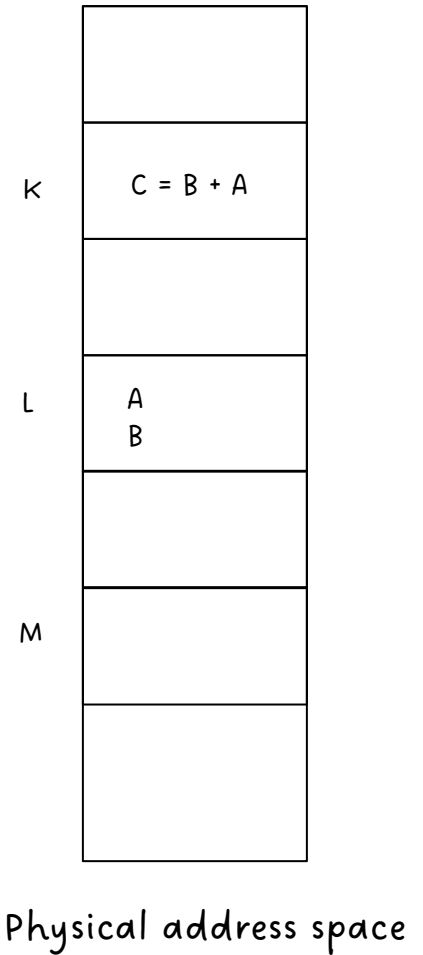
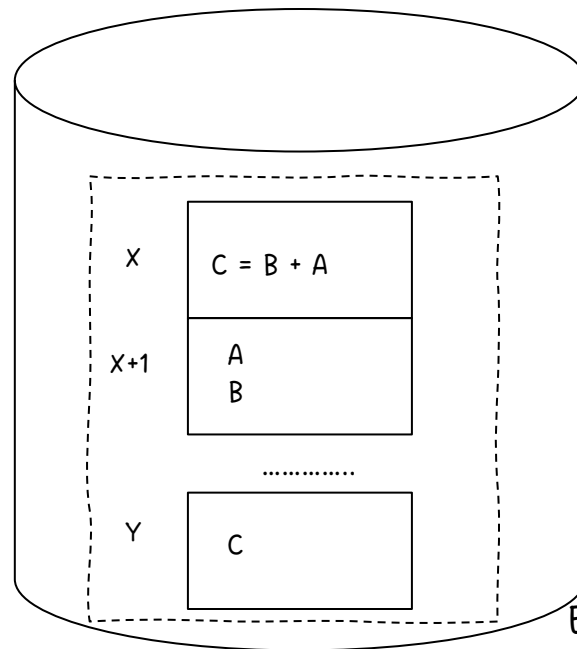
Physical address space

- (1) Restat คำสั่งที่ x:000
- (2) Run คำสั่ง x:004
- (3) Run คำสั่ง x:006
- (4) Run คำสั่ง x:008 (เกิด page fault เพราะ ?)



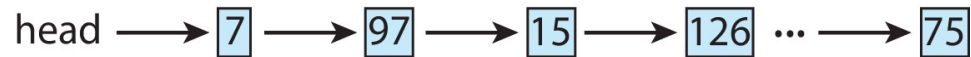
Page Table

p	f.	v/i
x	k	v
x+1	L	v
y		i



Free-Frame List

- When a page fault occurs, the operating system must bring the desired page from secondary storage into main memory.
- Most operating systems maintain a **free-frame list** -- a pool of free frames for satisfying such requests.



- Operating system typically **allocate free frames using a technique known as zero-fill-on-demand** -- **the content of the frames zeroed-out before being allocated.**
- When a system starts up, all available memory is placed on the free-frame list.

Stages in Demand Paging – Worse Case

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
 - a) Wait in a queue for this device until the read request is serviced
 - b) Wait for the device seek and/or latency time
 - c) Begin the transfer of the page to a free frame

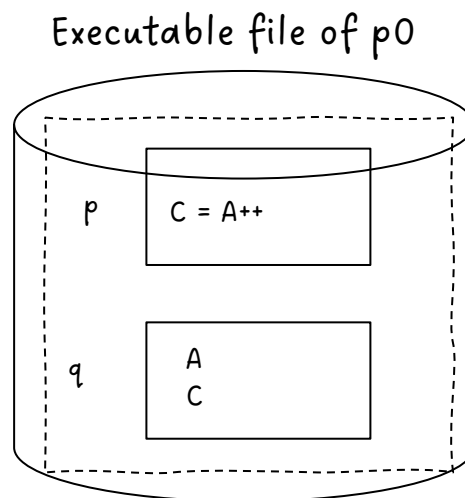
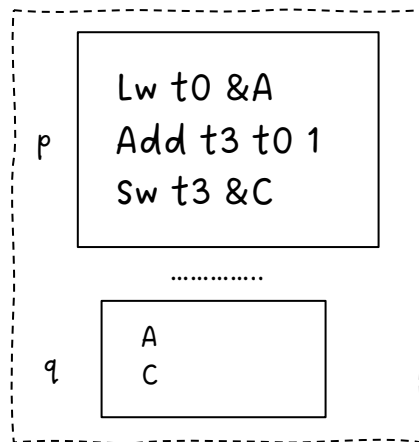
Stages in Demand Paging (Cont.)

6. While waiting, allocate the CPU to some other user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

เงื่อนไขตัวอย่างการประมวลผล (1)

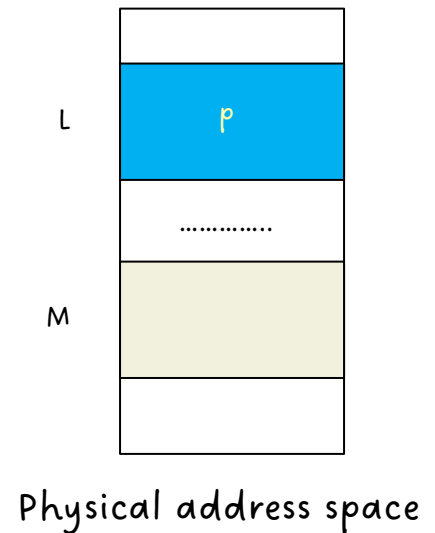
สมมติว่า:

1. ในระบบมีโปรเซส $p0$ และ $p1$
2. CPU ประมวลผล $p0$ อยู่ดังนั้น $p0$ มี state คือ **running** และ $p1$ อยู่ใน **Ready State**
3. $p0$ มี page "p" อยู่ใน physical mem

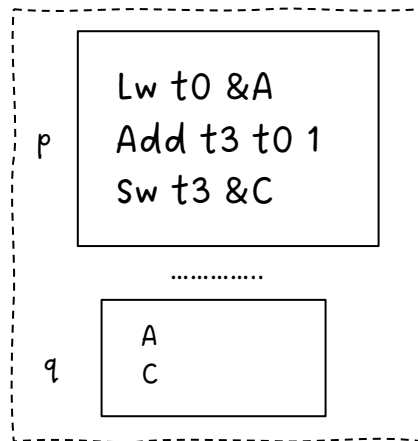
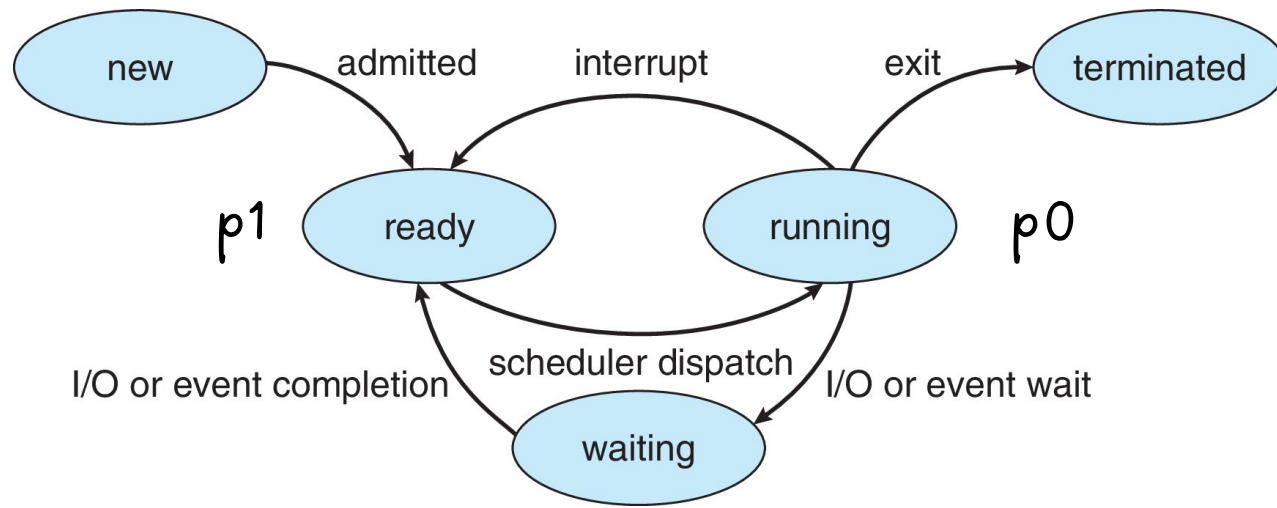


Page Table of $p0$

p	f.	v/i
p	L	v
q		i

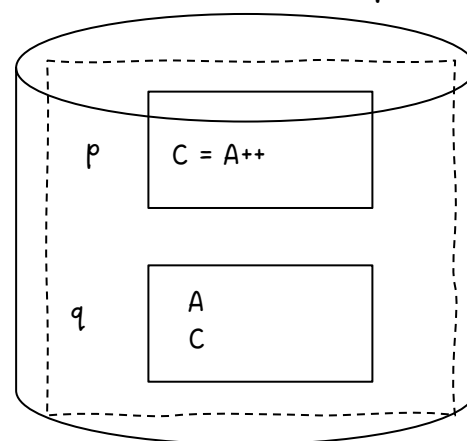


เงื่อนไขตัวอย่างการประมวลผล (2)



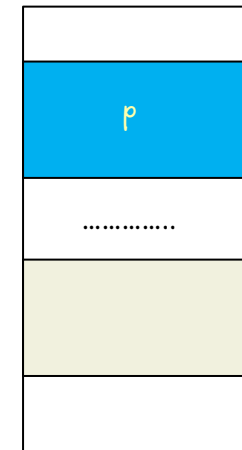
Virtual address space

Executable file of p0



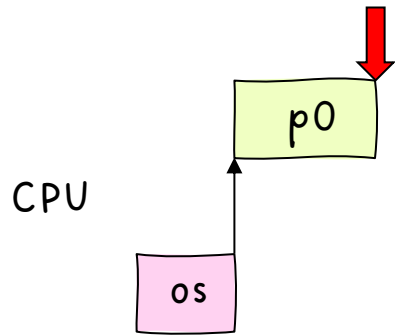
Page Table of p0

p	f.	v/i
p	L	v
q		i

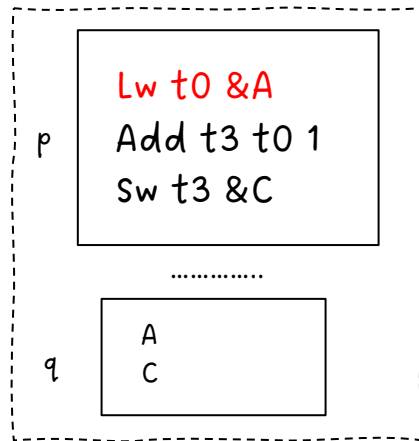


Physical address space

เงื่อนไขตัวอย่างการประมวลผล (3)

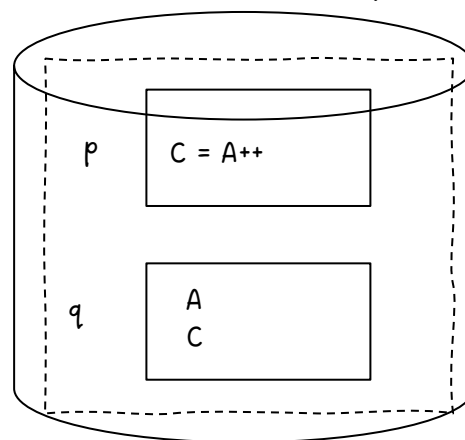


I/O



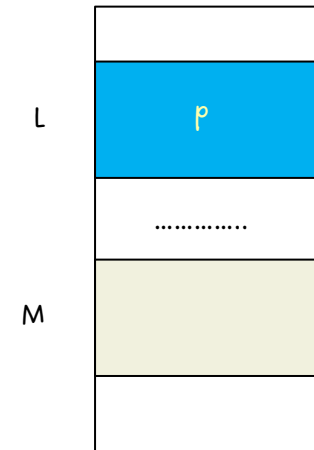
Virtual address space

Executable file of p0



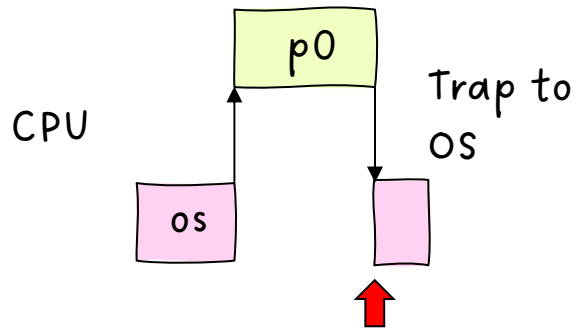
Page Table of p0

p	f.	v/i
p	L	v
q		i

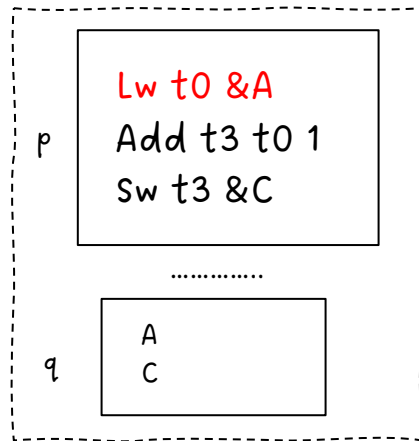


Physical address space

1. Trap to the operating system

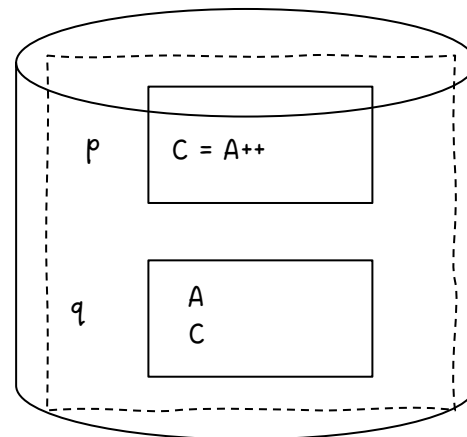


I/O



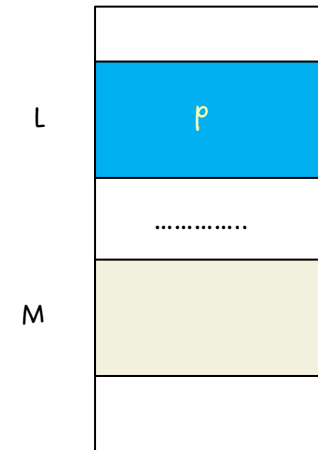
Virtual address space

Executable file of p0



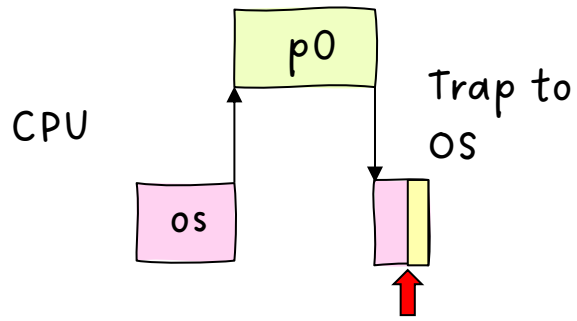
Page Table of p0

p	f.	v/i
p	L	v
q		i

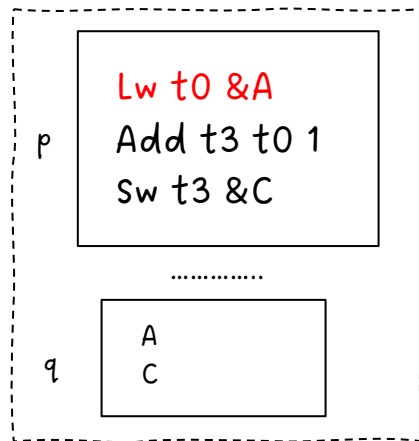


Physical address space

2. Save the user registers and process state

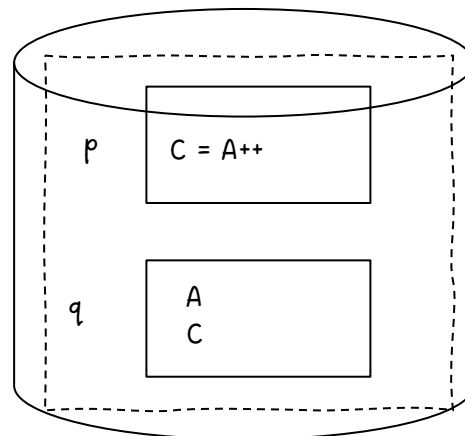


I/O



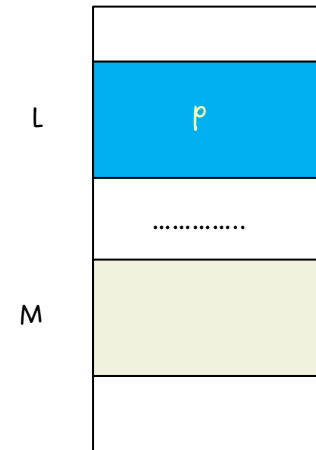
Virtual address space

Executable file of p0



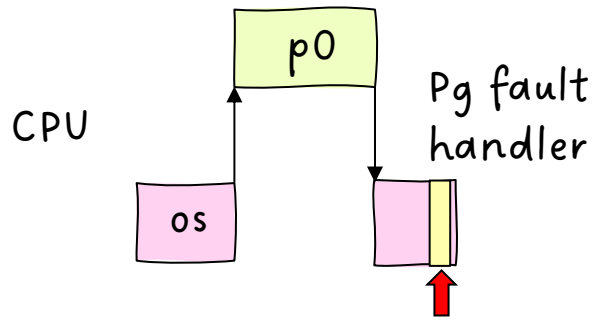
Page Table of p0

p	f.	v/i
p	L	v
q		i

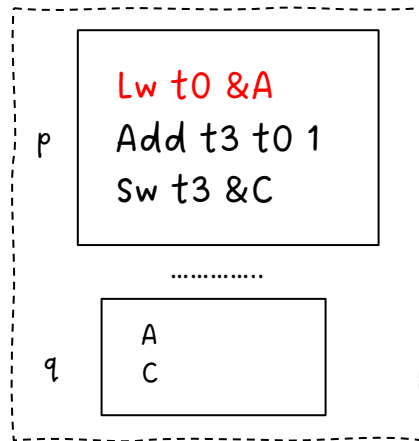


Physical address space

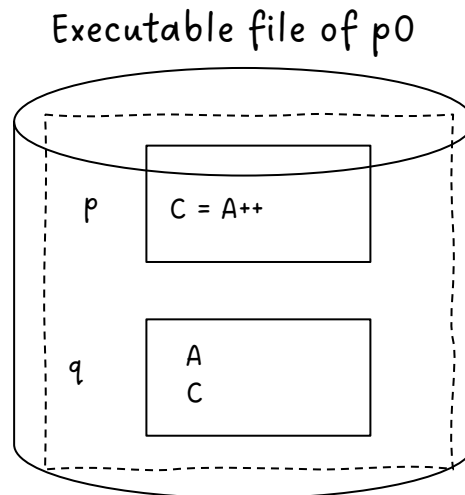
3. Determine that the interrupt was a page fault



I/O

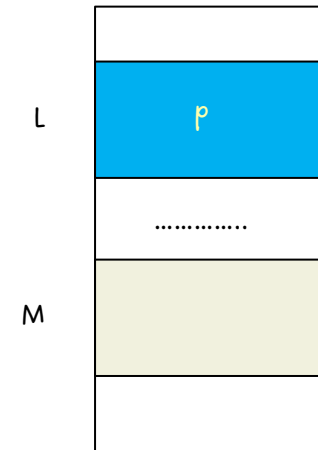


Virtual address space



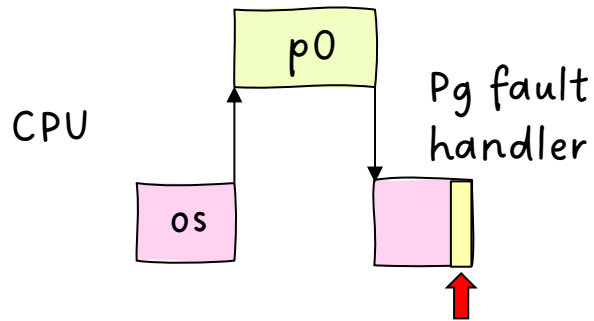
Page Table of p0

p	f.	v/i
p	L	v
q		i

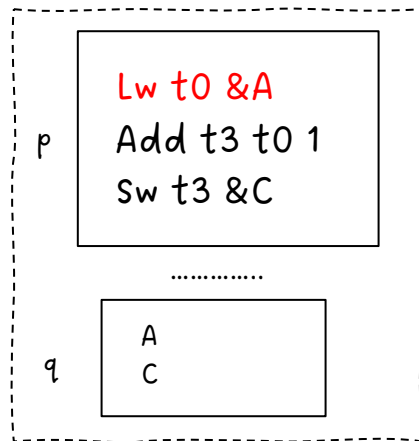


Physical address space

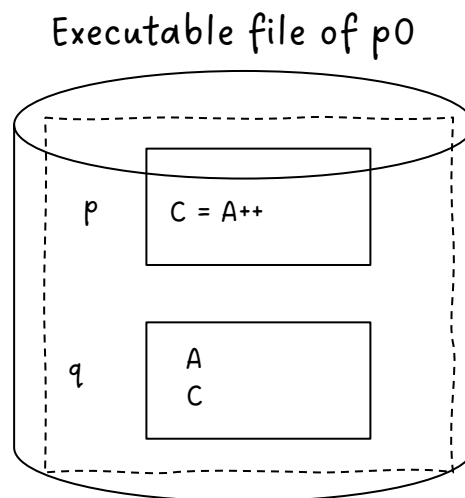
4. Check that the page reference was legal and determine the location of the page on the disk



I/O

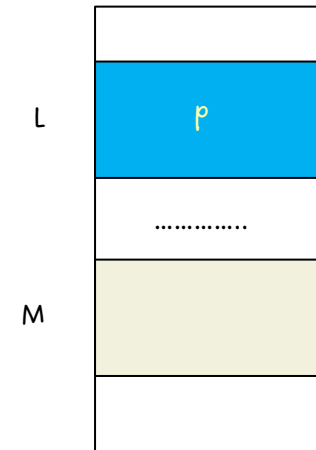


Virtual address space



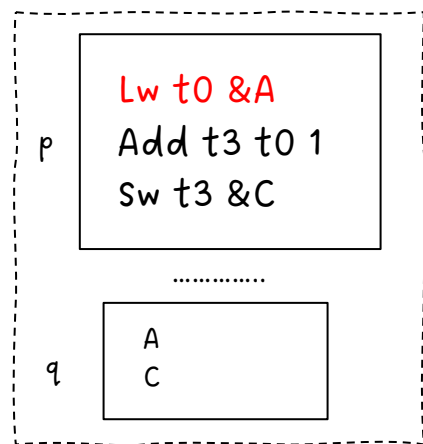
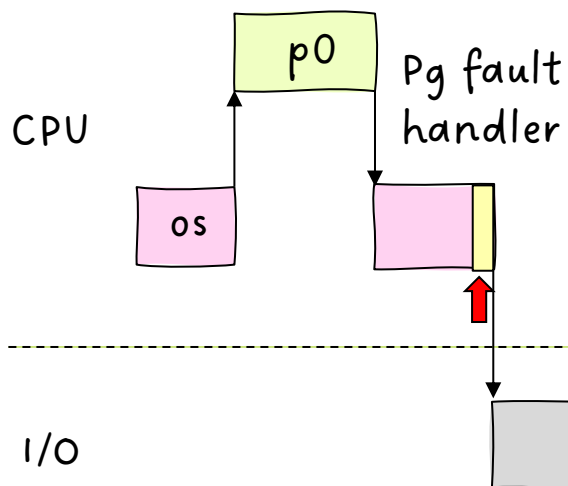
Page Table of p0

p	f.	v/i
p	L	v
q		i

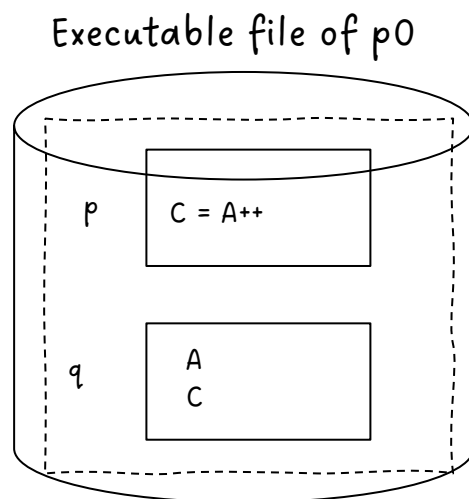


Physical address space

5. Issue a read from the disk to a free frame

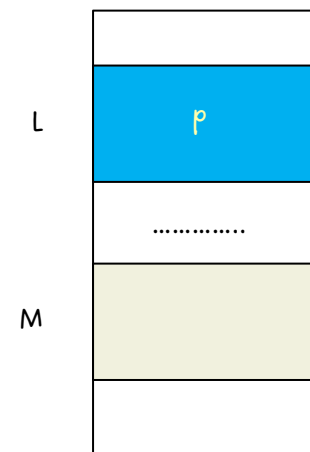


Virtual address space



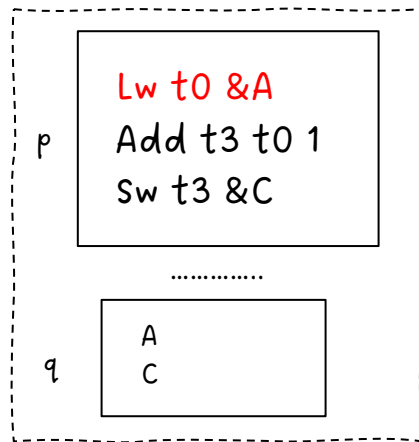
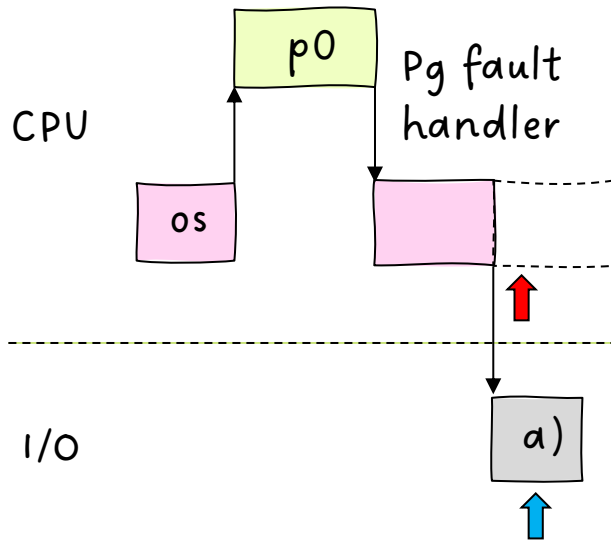
Page Table of p0

p	f.	v/i
p	L	v
q		i



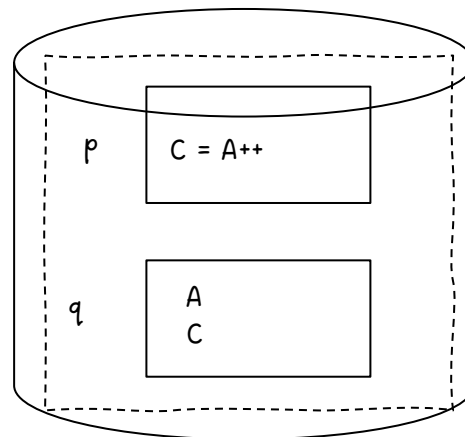
Physical address space

a) Wait in a queue for this device until the read request is serviced



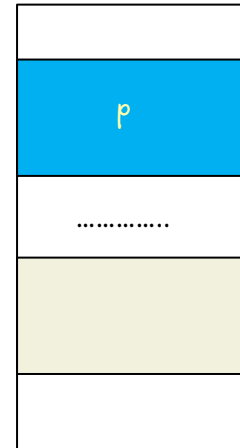
Virtual address space

Executable file of p0



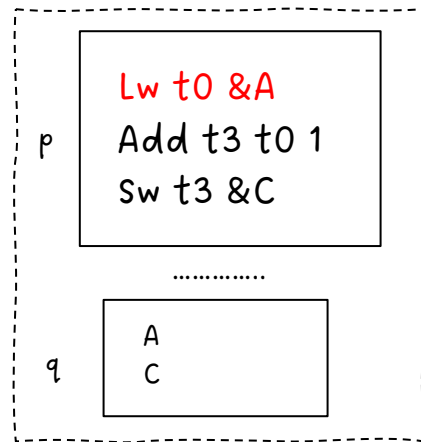
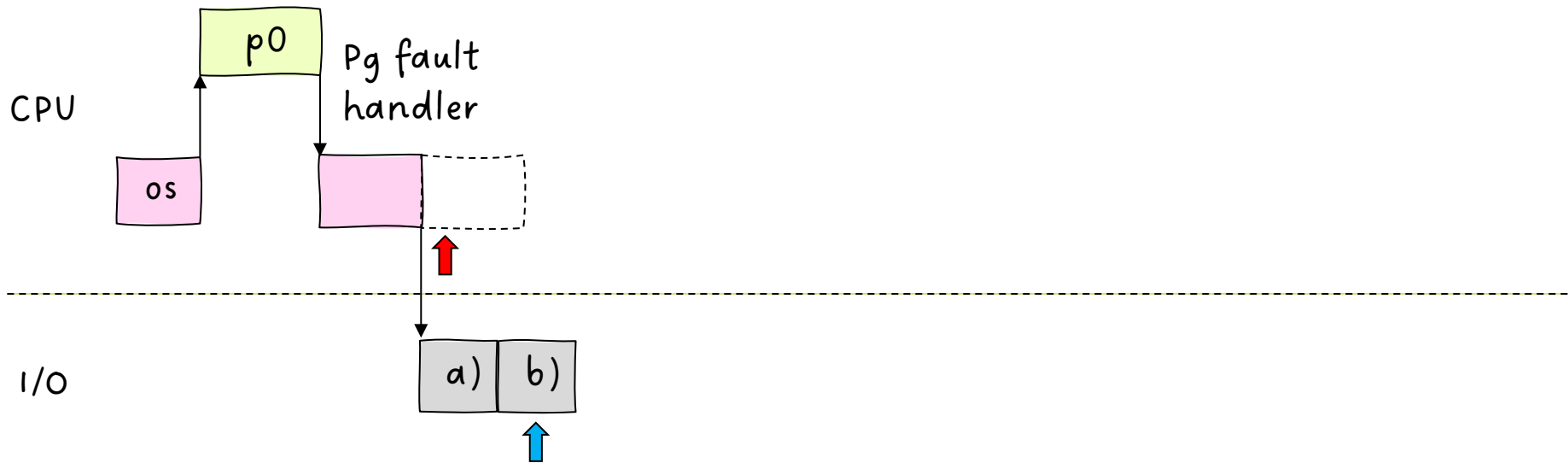
Page Table of p0

p	f.	v/i
p	L	v
q		i



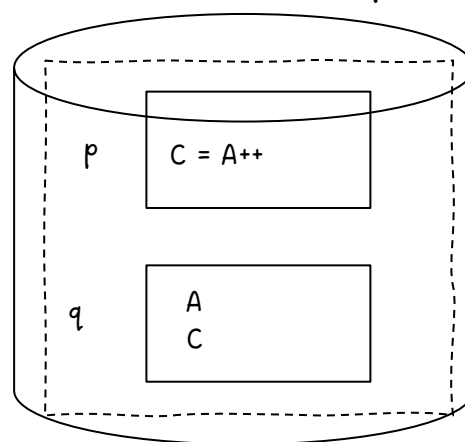
Physical address space

b) Wait for the device seek and/or latency time



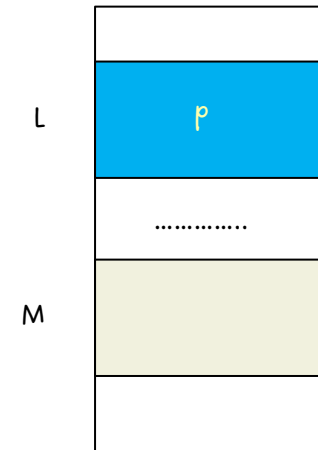
Virtual address space

Executable file of p0



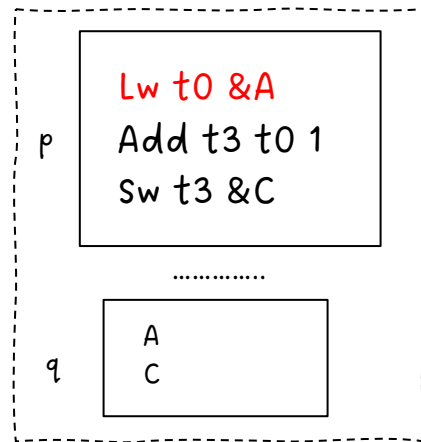
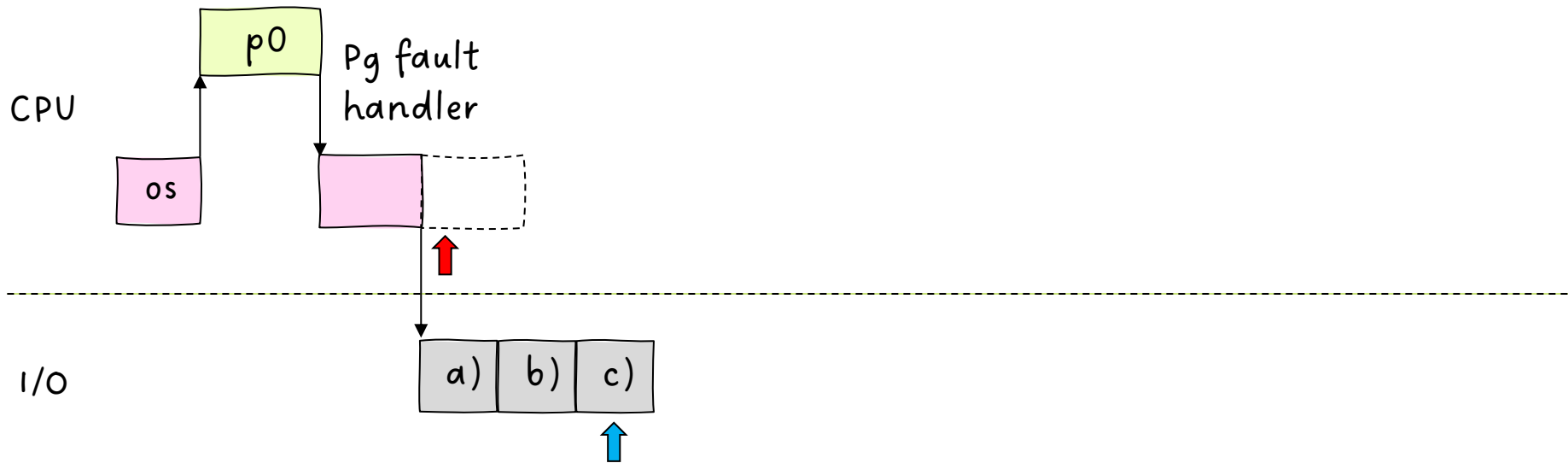
Page Table of p0

p	f.	v/i
p	L	v
q		i



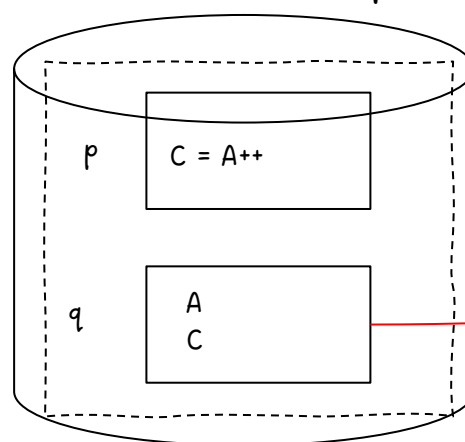
Physical address space

c) Begin the transfer of the page to a free frame (DMA)



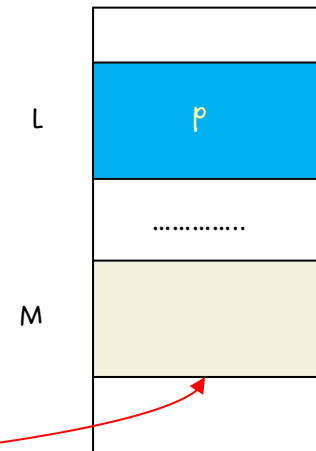
Virtual address space

Executable file of p0



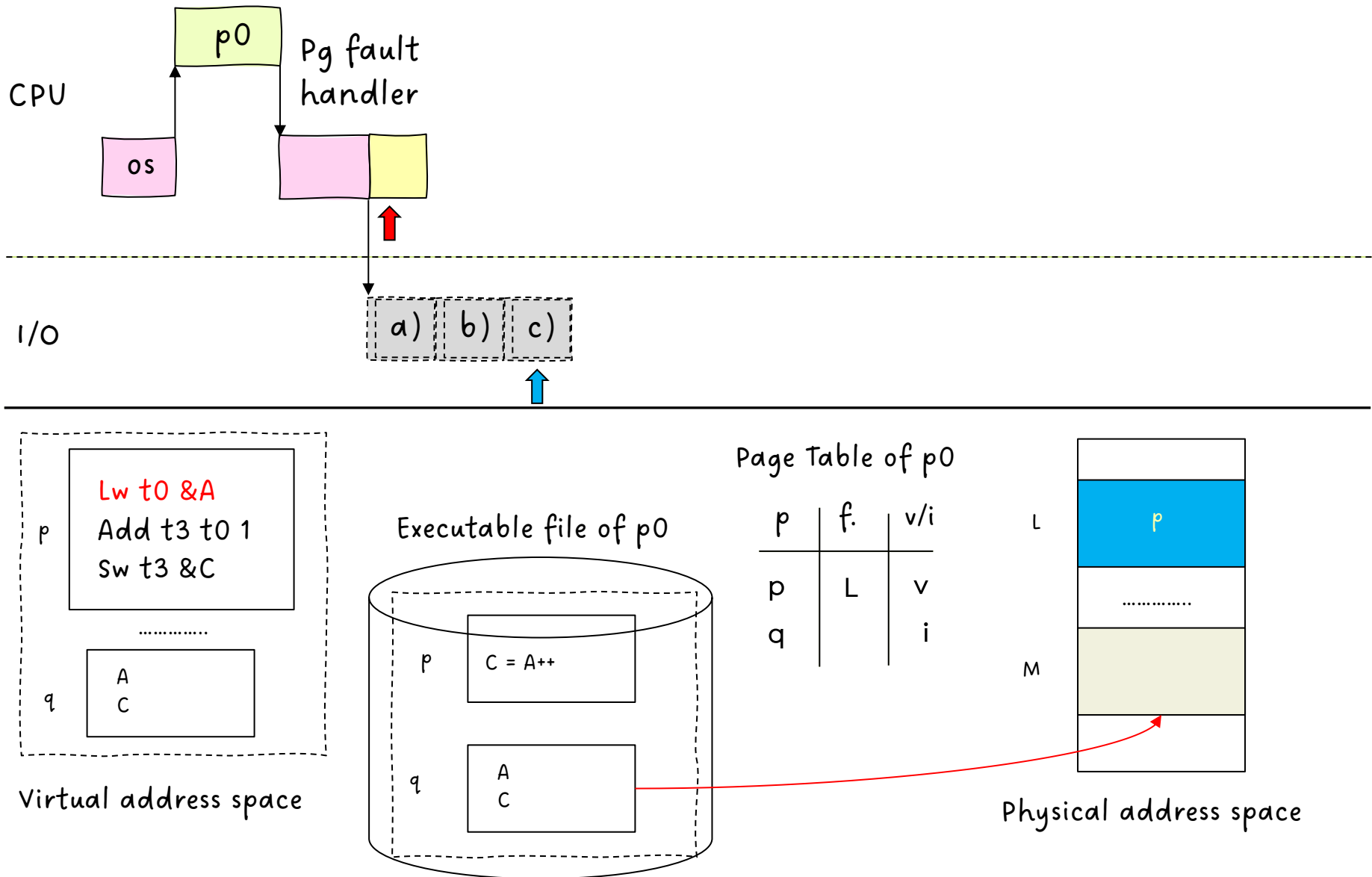
Page Table of p0

p	f.	v/i
p	L	v
q		i

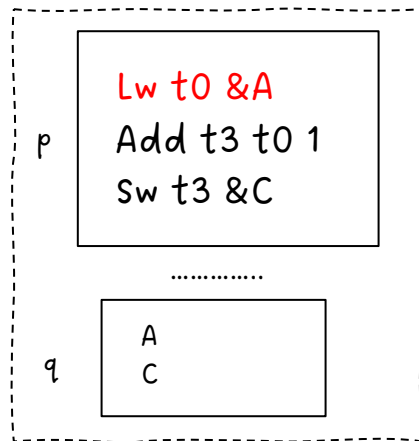
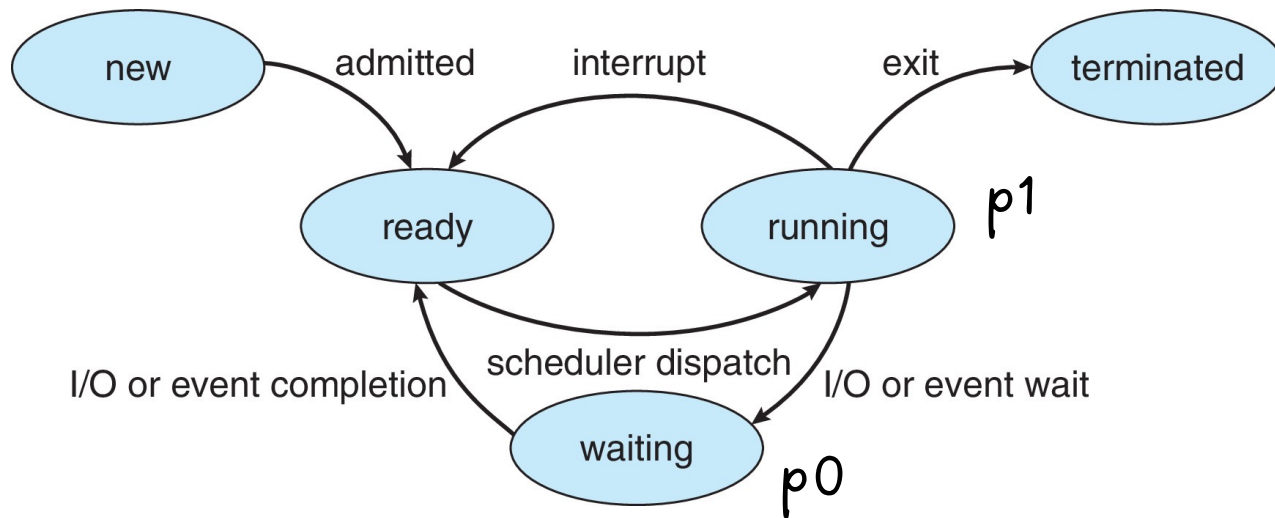


Physical address space

6. While waiting, allocate the CPU to some other user

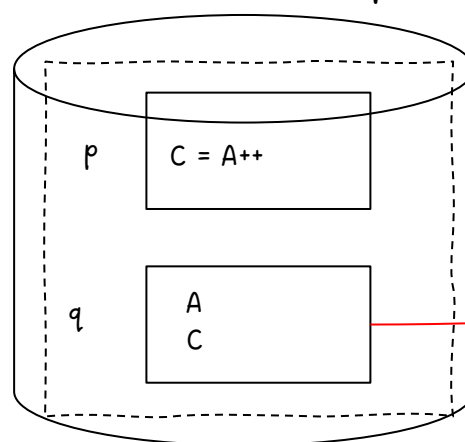


6. While waiting, allocate the CPU to some other user



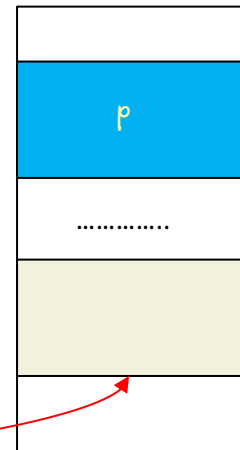
Virtual address space

Executable file of p0



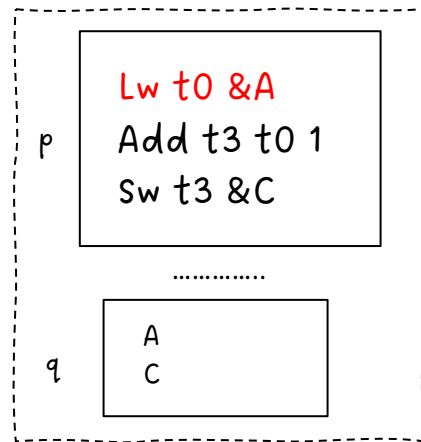
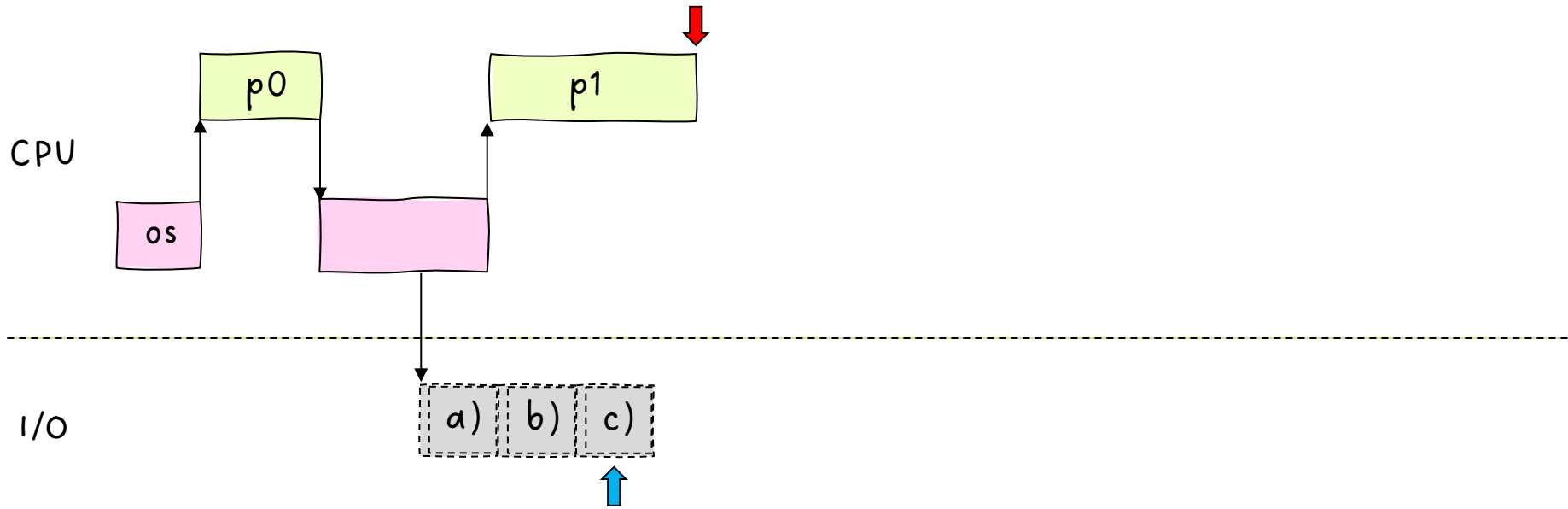
Page Table of p0

p	f.	v/i
p	L	v
q		i



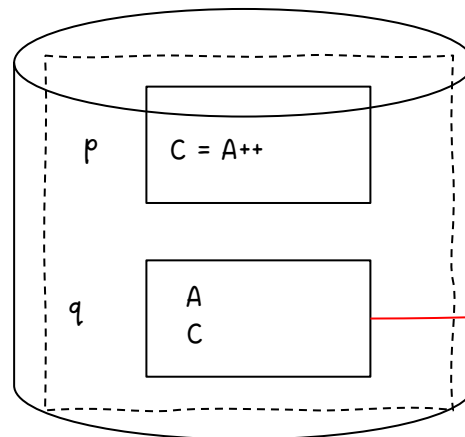
Physical address space

6) While waiting, allocate the CPU to some other user



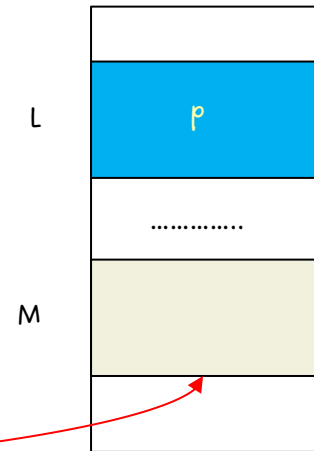
Virtual address space

Executable file of p0



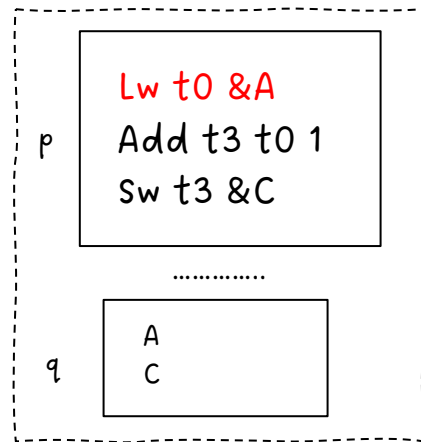
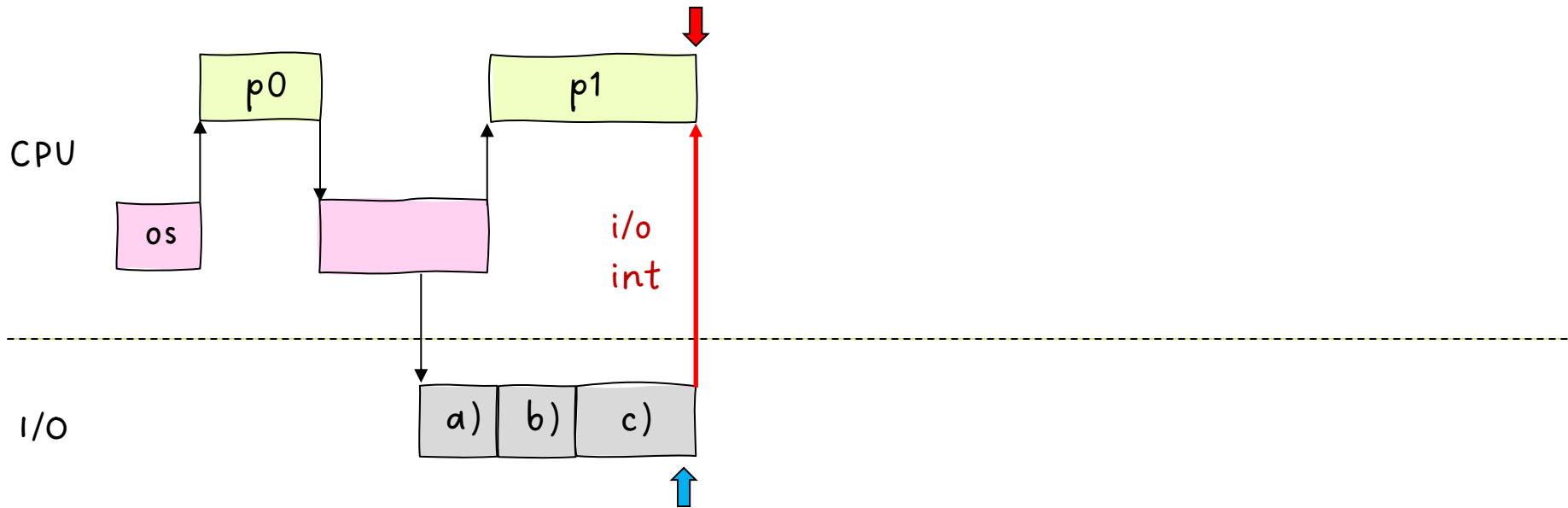
Page Table of p0

p	f.	v/i
p	L	v
q		i



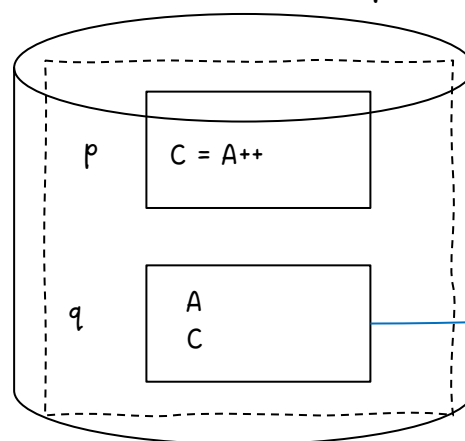
Physical address space

7. Receive an interrupt from the disk I/O subsystem (I/O completed)



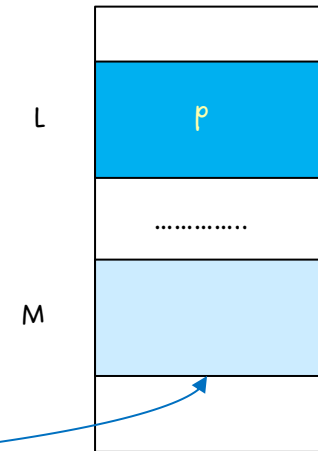
Virtual address space

Executable file of p0



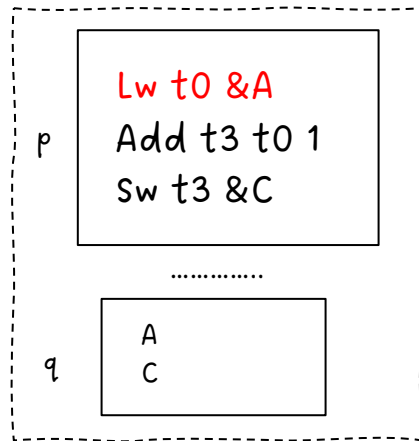
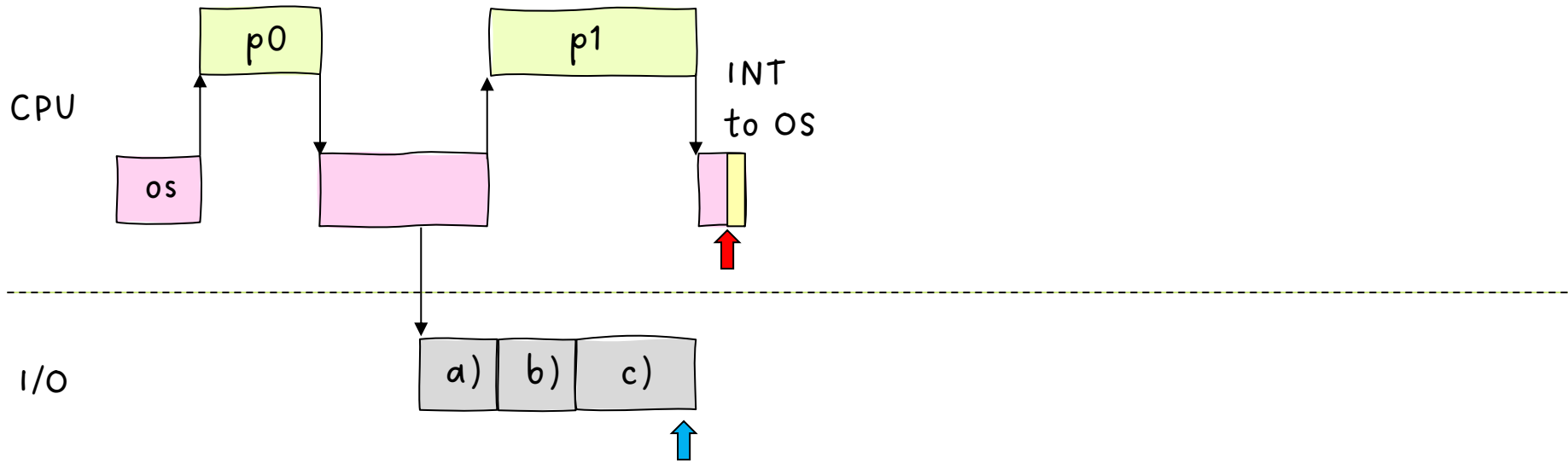
Page Table of p0

p	f.	v/i
p	L	v
q		i



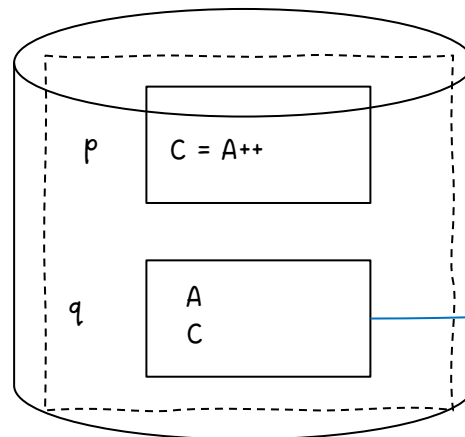
Physical address space

8. Save the registers and process state for the other user



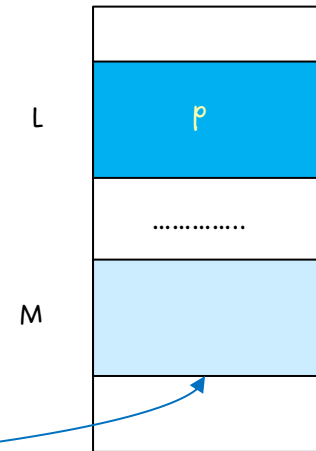
Virtual address space

Executable file of **p0**



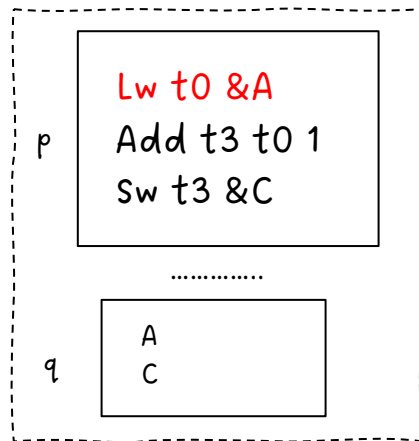
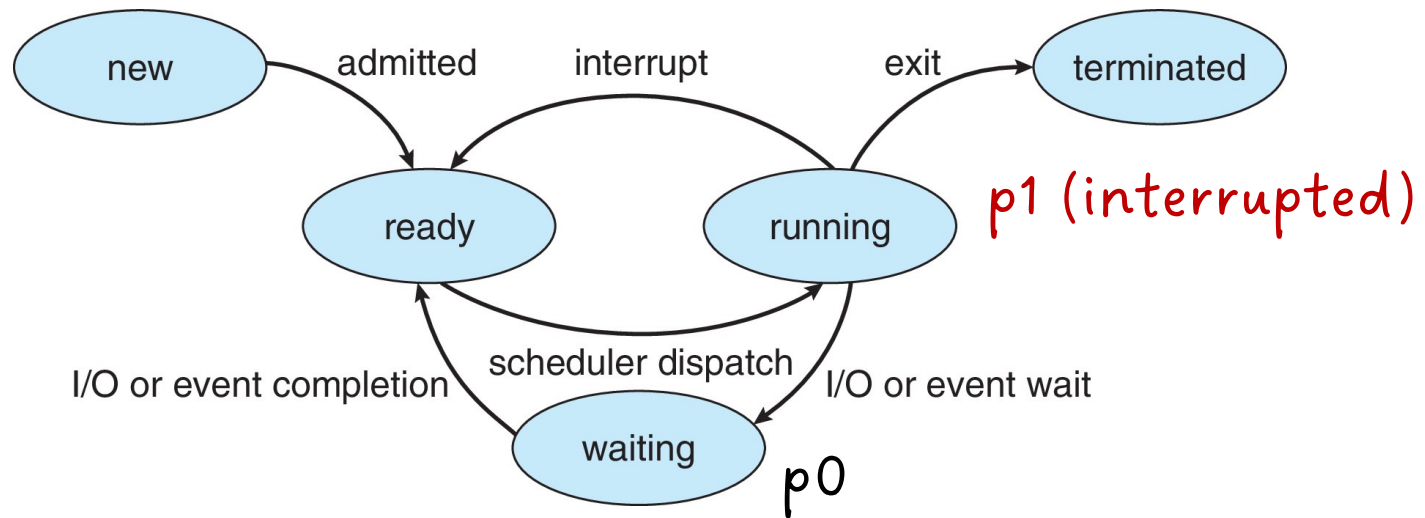
Page Table of **p0**

p	f.	v/i
p	L	v
q		i



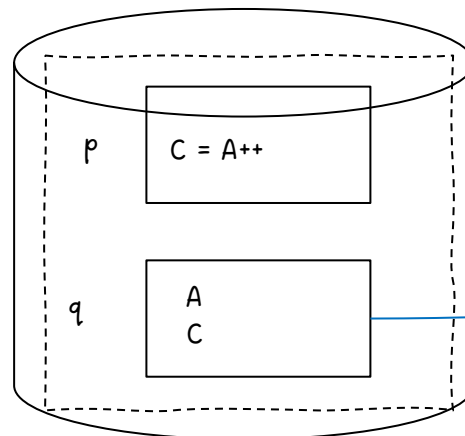
Physical address space

8. Save the registers and process state for the other user



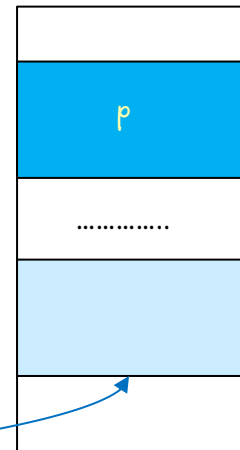
Virtual address space

Executable file of p0



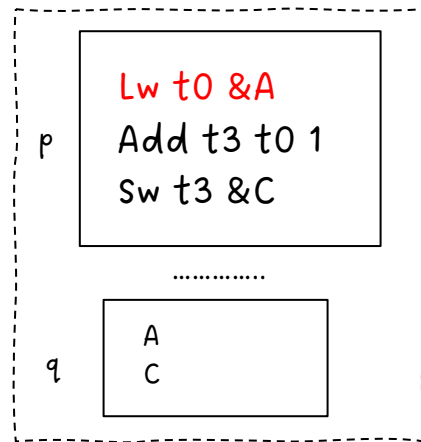
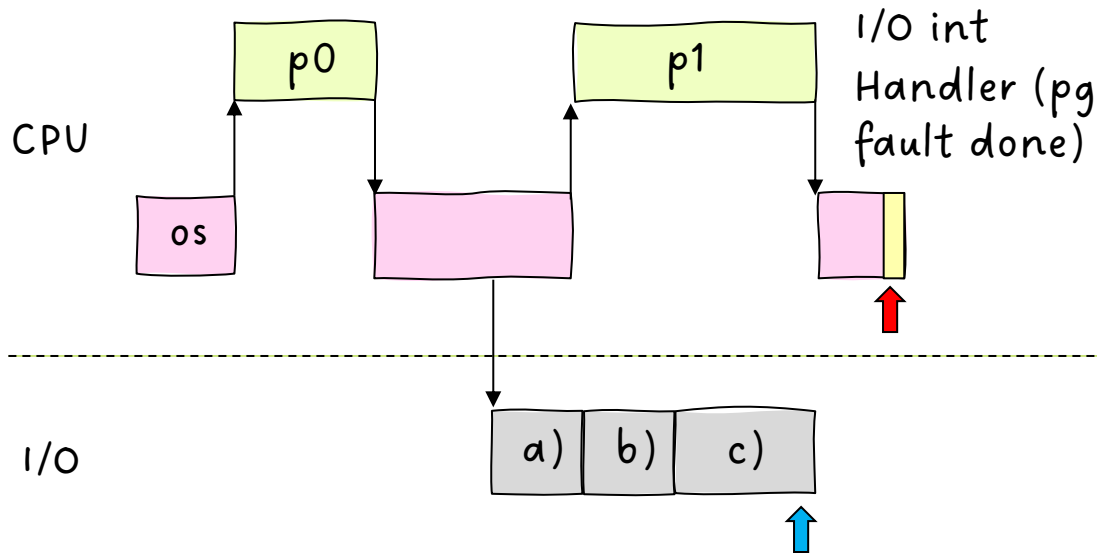
Page Table of p0

p	f.	v/i
p	L	v
q		i



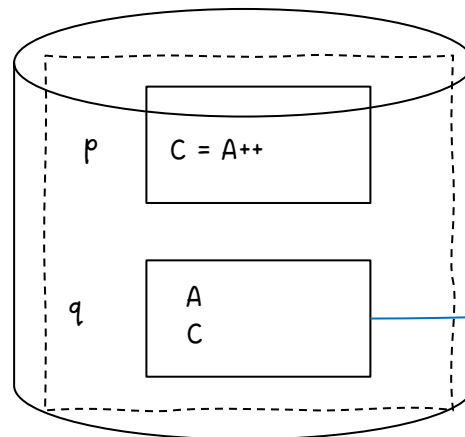
Physical address space

9. Determine that the interrupt was from the disk



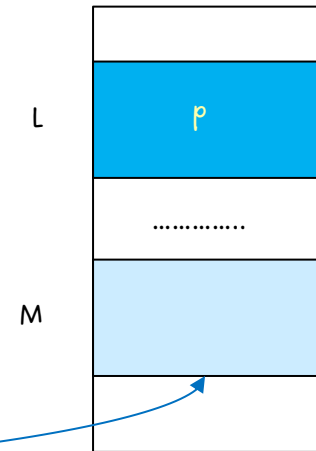
Virtual address space

Executable file of p0



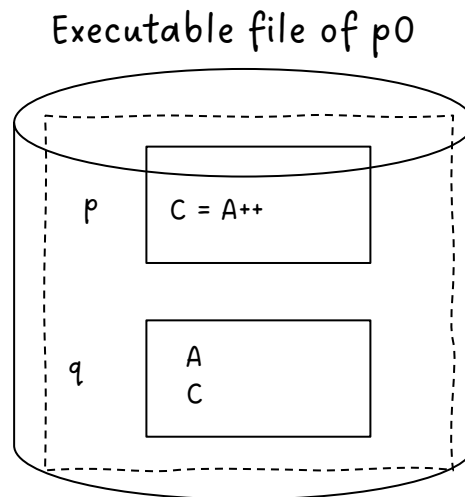
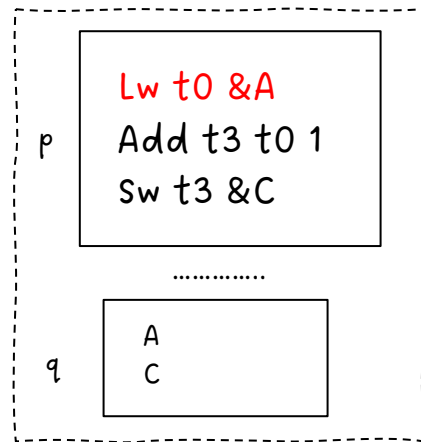
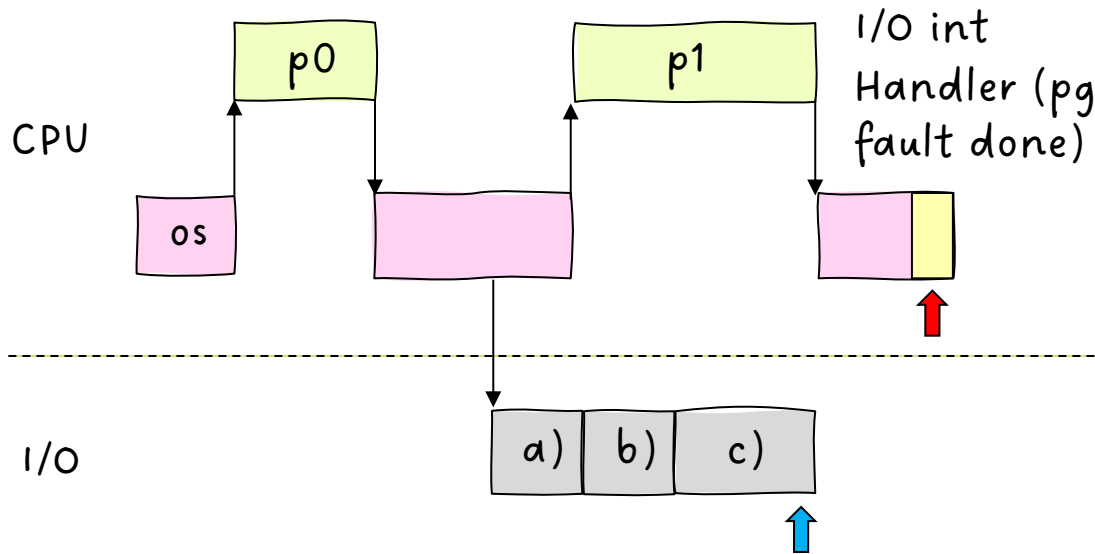
Page Table of p0

p	f.	v/i
p	L	v
q		i



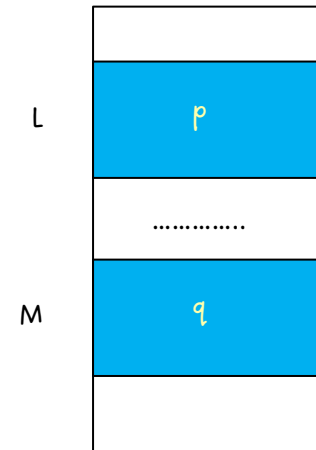
Physical address space

10. Correct the page table and other tables to show page is now in memory



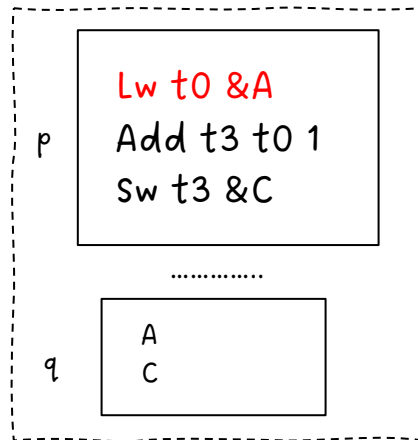
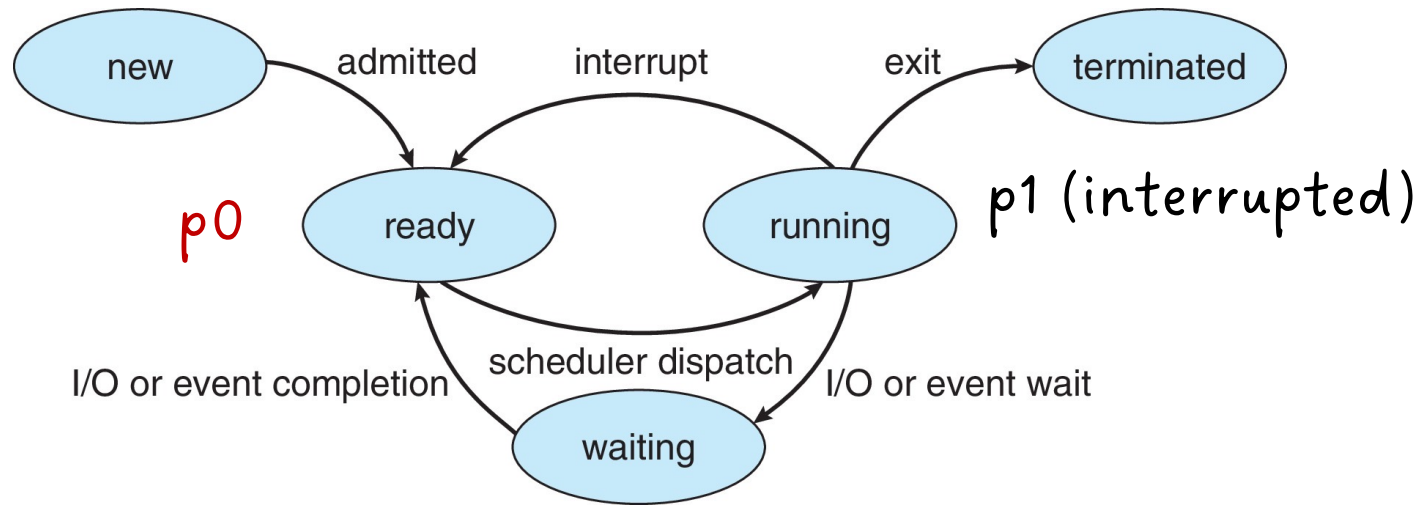
Page Table of p0

p	f.	v/i
p	L	v
q	M	v



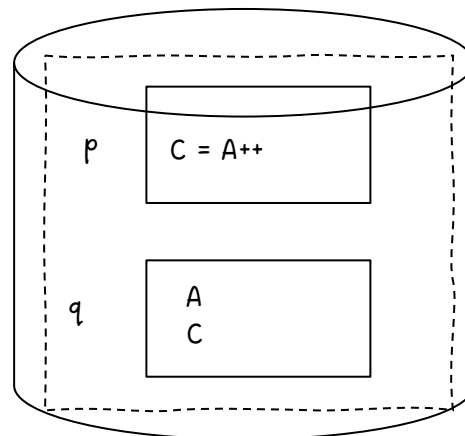
Physical address space

10. Correct the page table and other tables to show page is now in memory



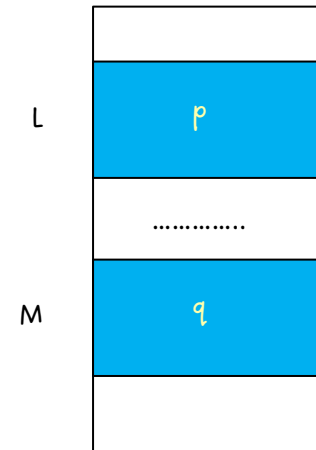
Virtual address space

Executable file of p0



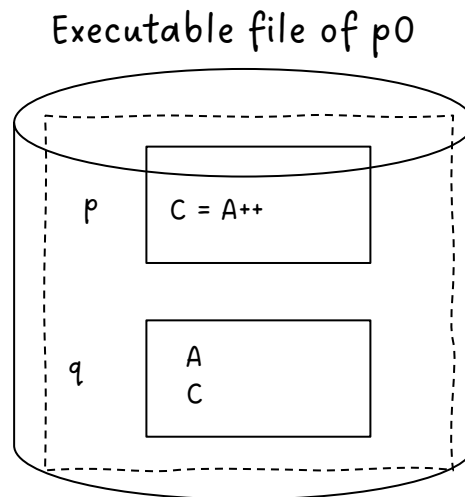
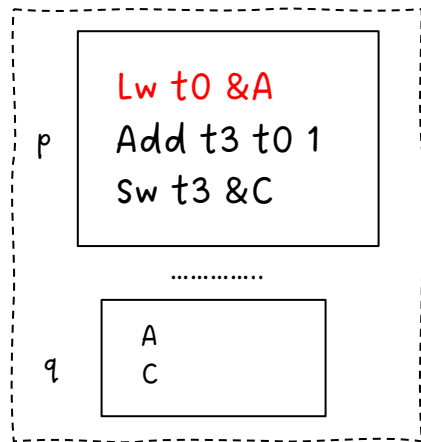
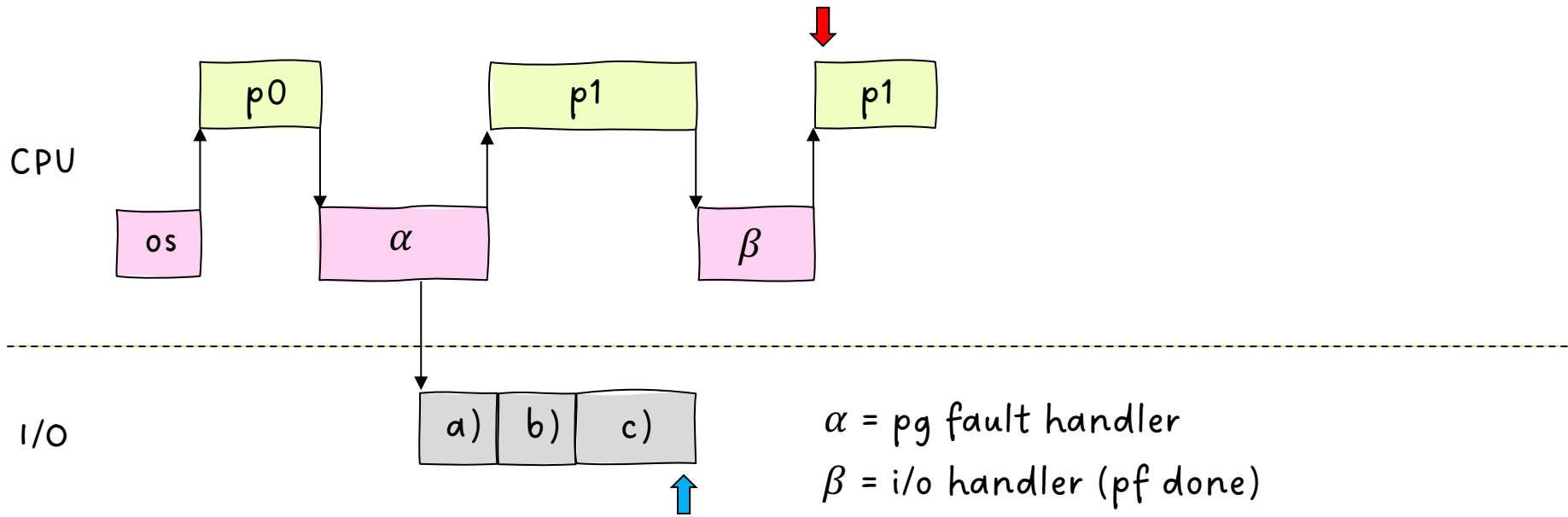
Page Table of p0

p	f.	v/i
p	L	v
q	M	v



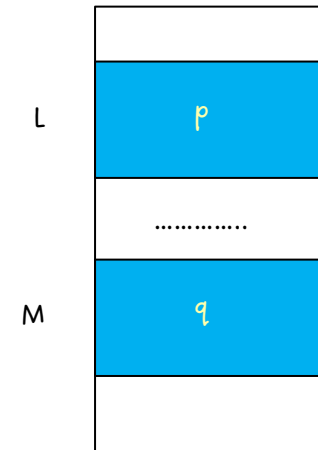
Physical address space

11. Wait for the CPU to be allocated to this process again



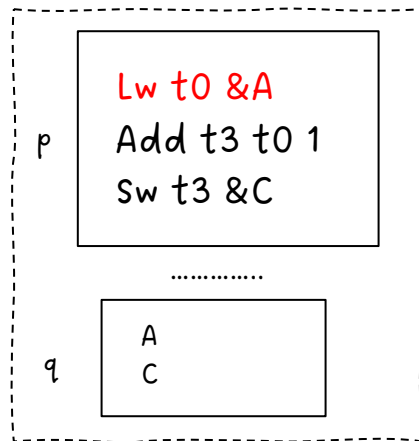
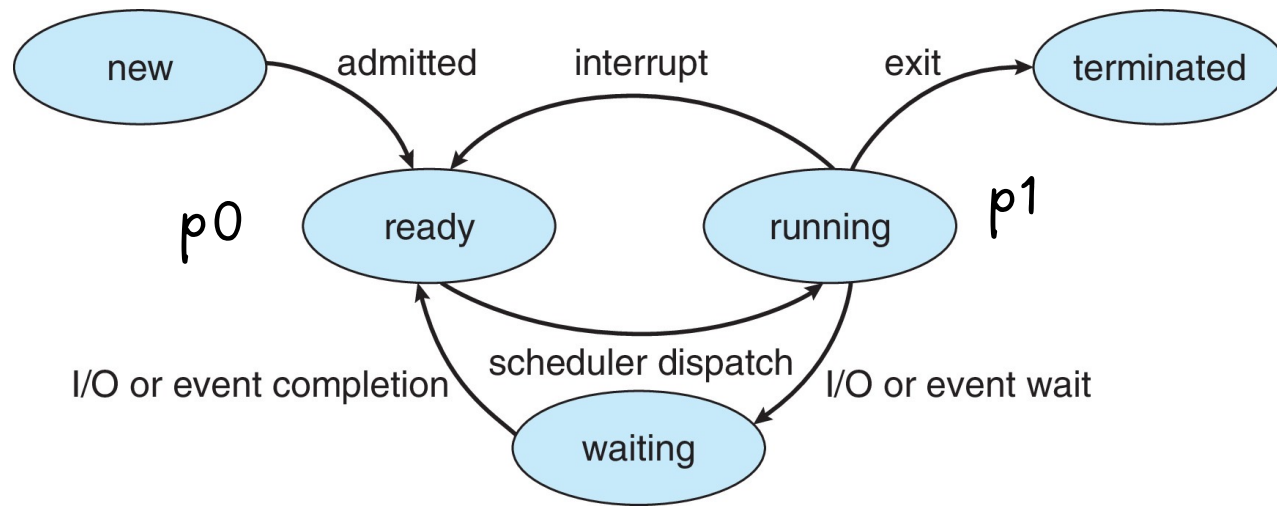
Page Table of p0

p	f.	v/i
p	L	v
q	M	v



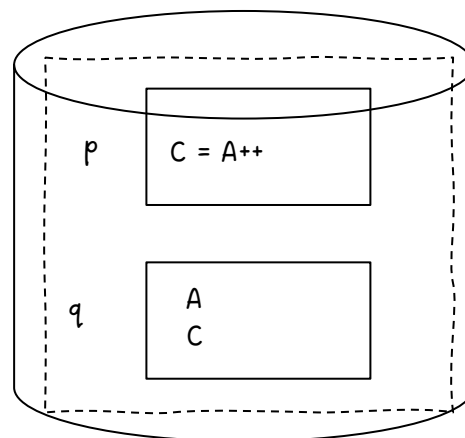
Physical address space

11. Wait for the CPU to be allocated to this process again



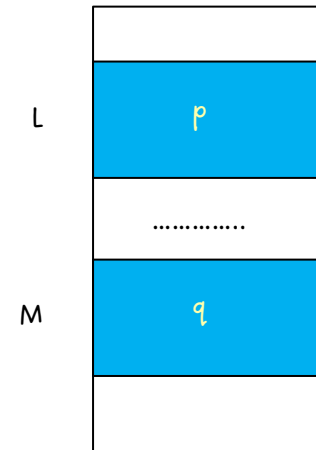
Virtual address space

Executable file of p0



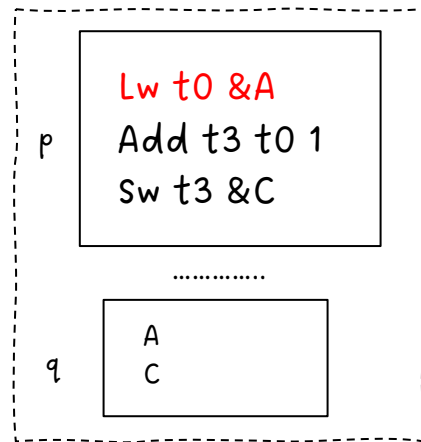
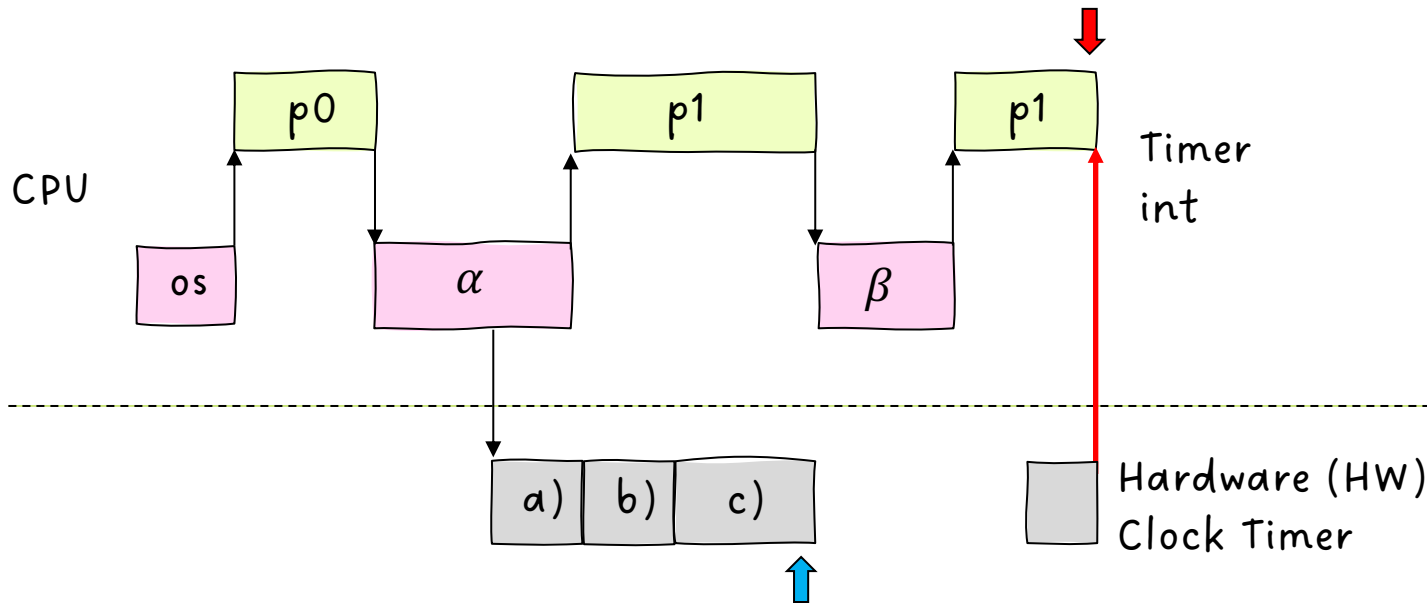
Page Table of p0

p	f.	v/i
p	L	v
q	M	v



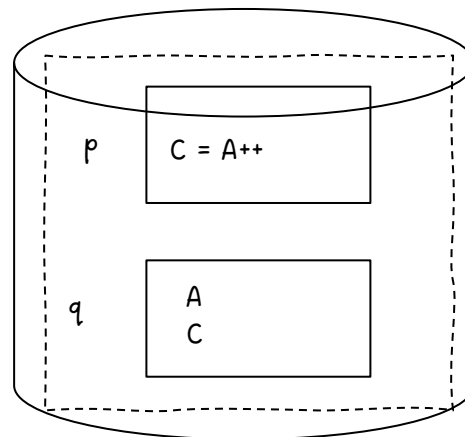
Physical address space

11. Wait for the CPU to be allocated to this process again



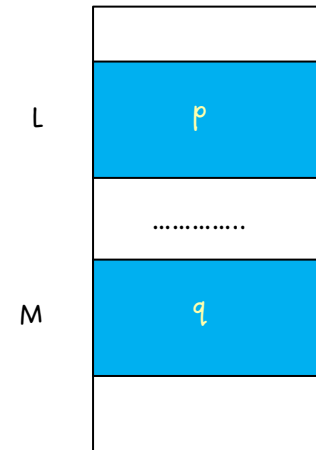
Virtual address space

Executable file of p0



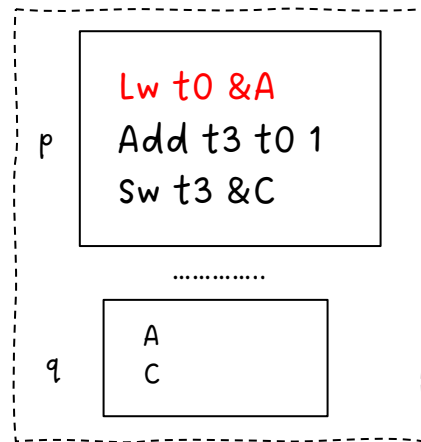
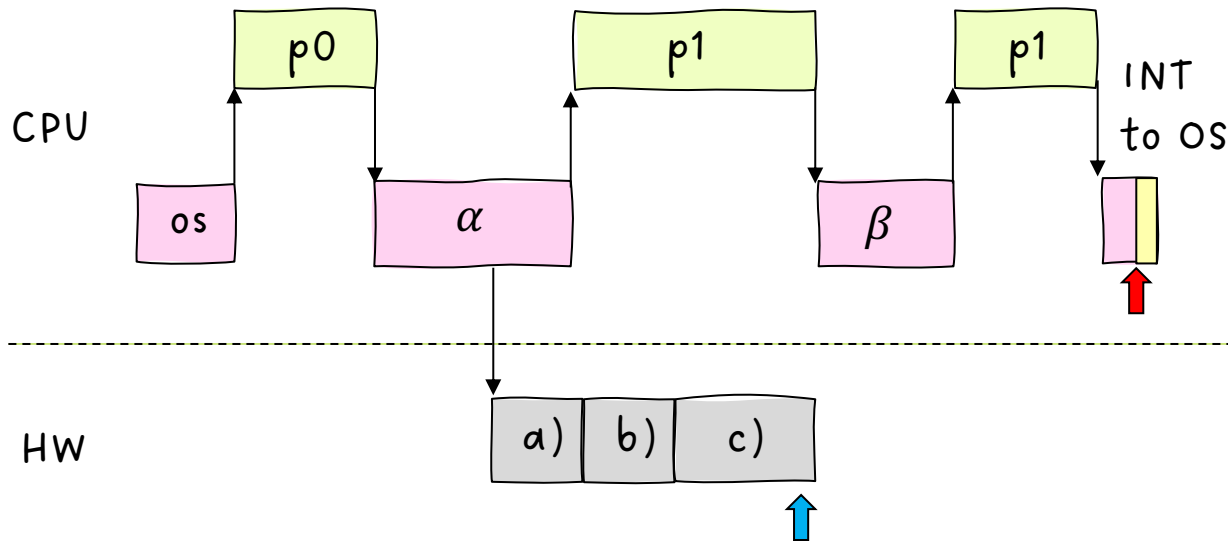
Page Table of p0

p	f.	v/i
p	L	v
q	M	v



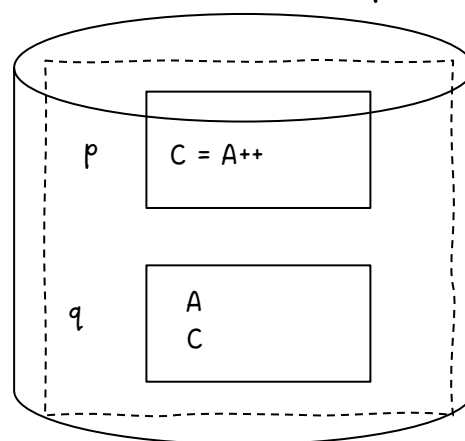
Physical address space

11. Wait for the CPU to be allocated to this process again



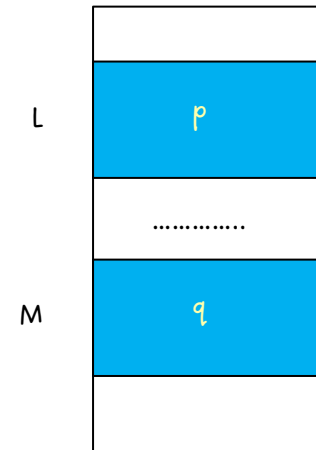
Virtual address space

Executable file of p0



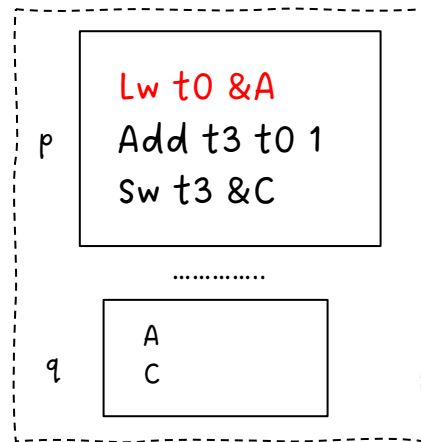
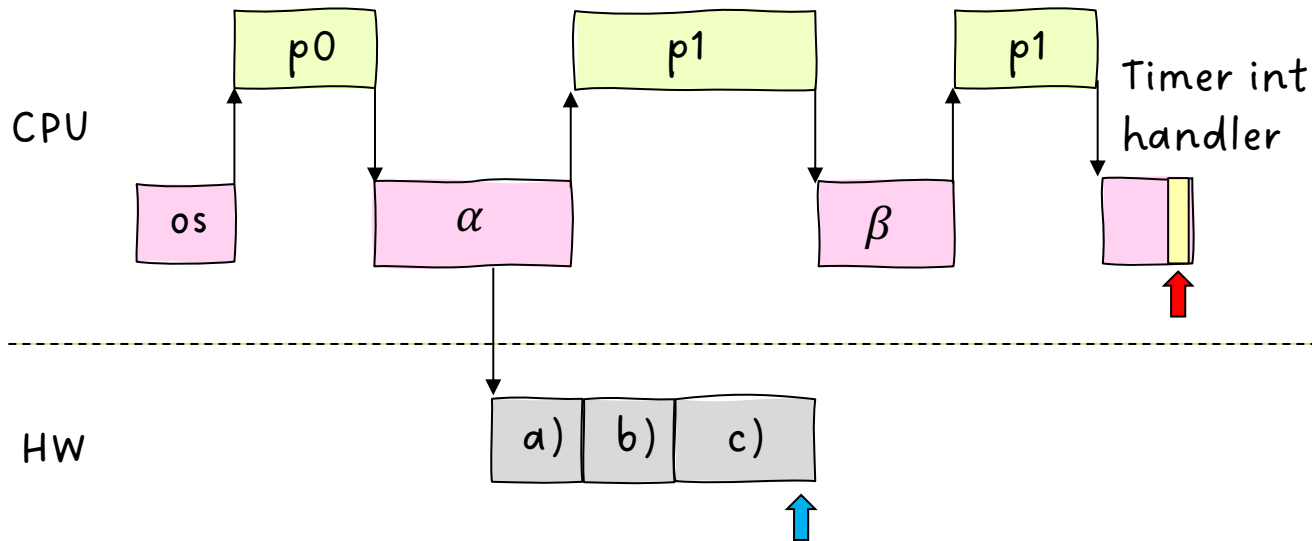
Page Table of p0

p	f.	v/i
p	L	v
q	M	v



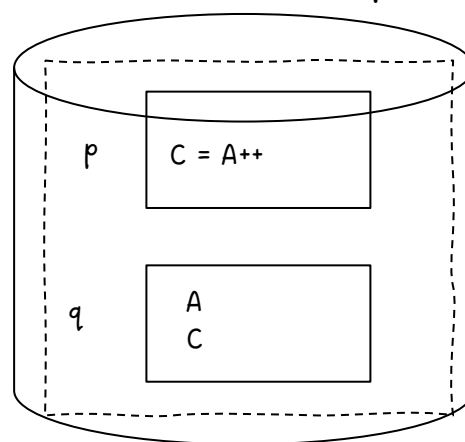
Physical address space

11. Wait for the CPU to be allocated to this process again



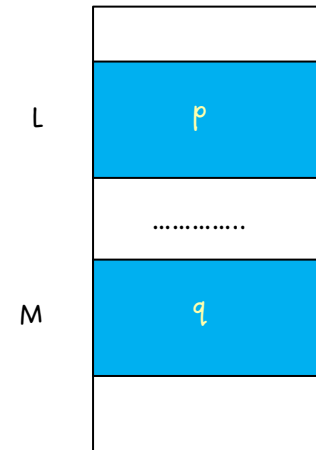
Virtual address space

Executable file of p_0



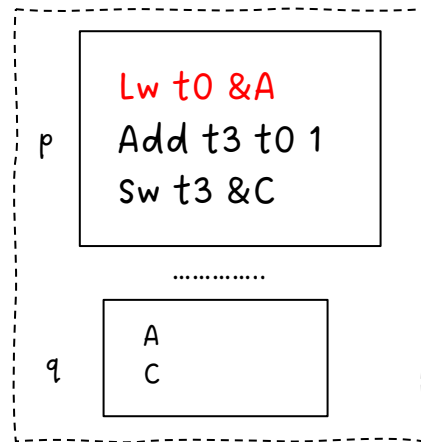
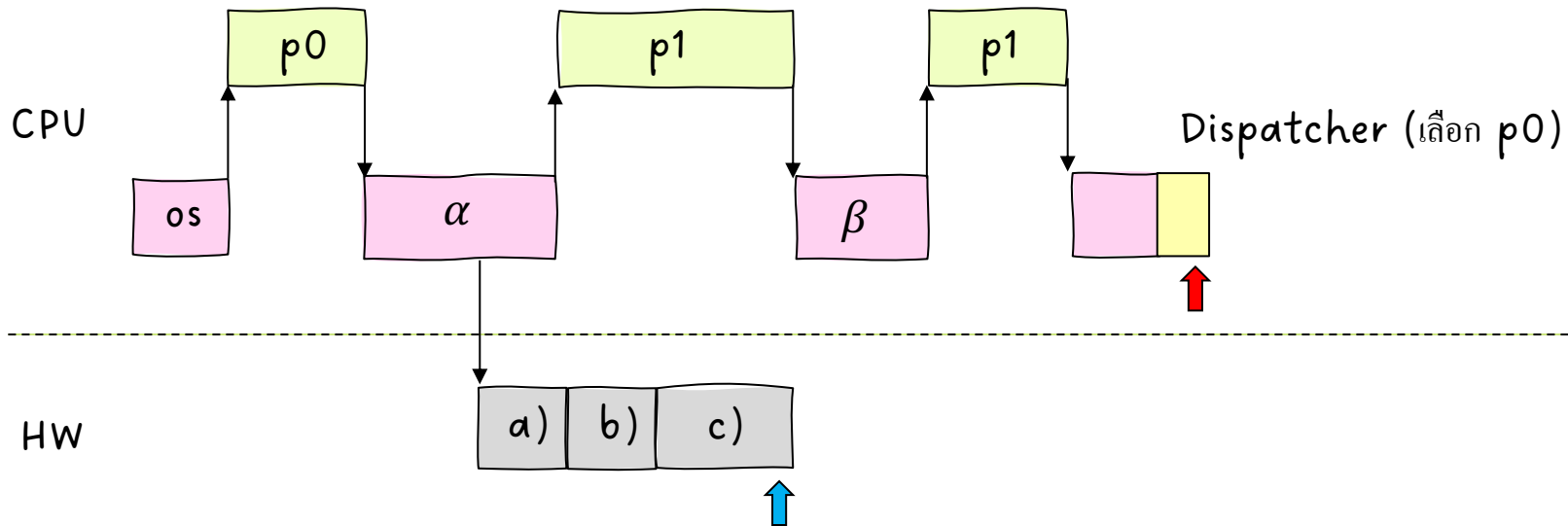
Page Table of p_0

p	f.	v/i
p	L	v
q	M	v



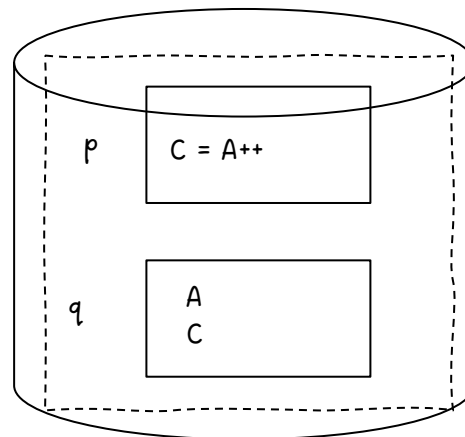
Physical address space

12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction



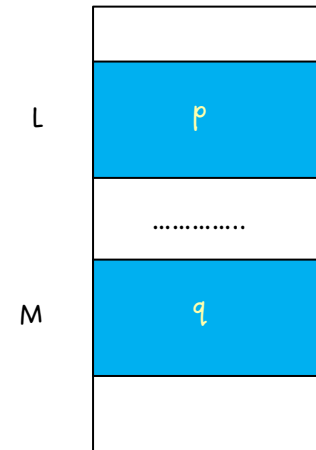
Virtual address space

Executable file of p0



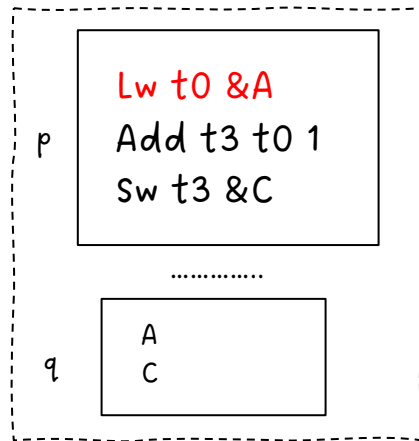
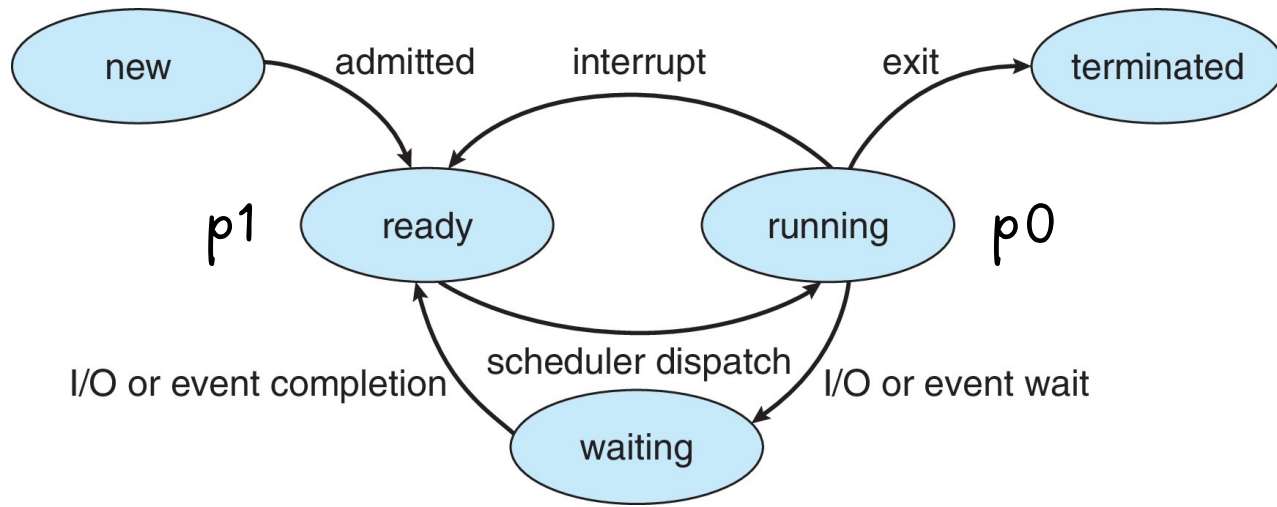
Page Table of p0

p	f.	v/i
p	L	v
q	M	v



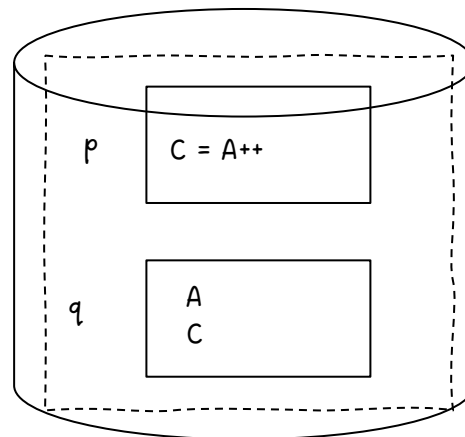
Physical address space

12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction



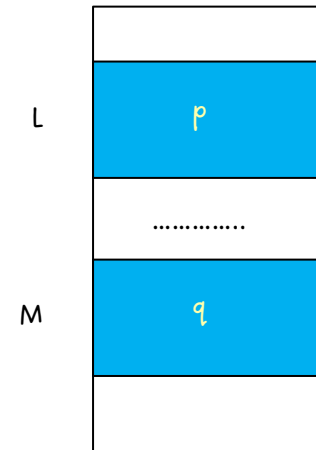
Virtual address space

Executable file of p0



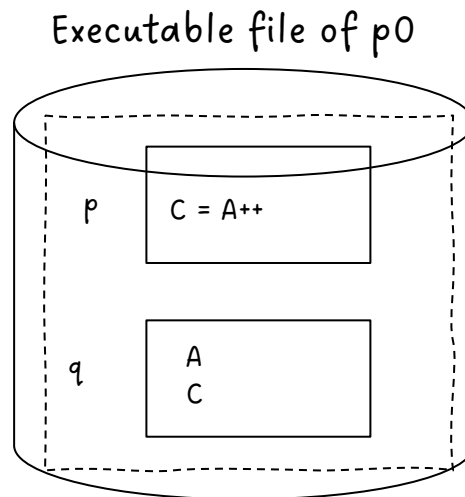
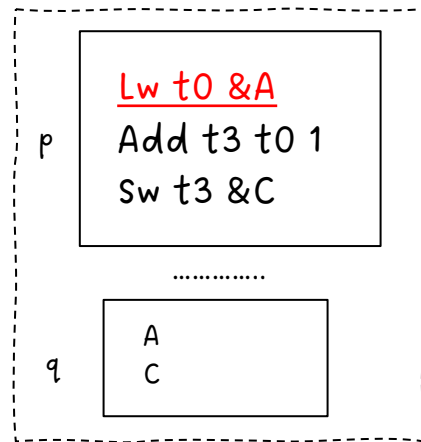
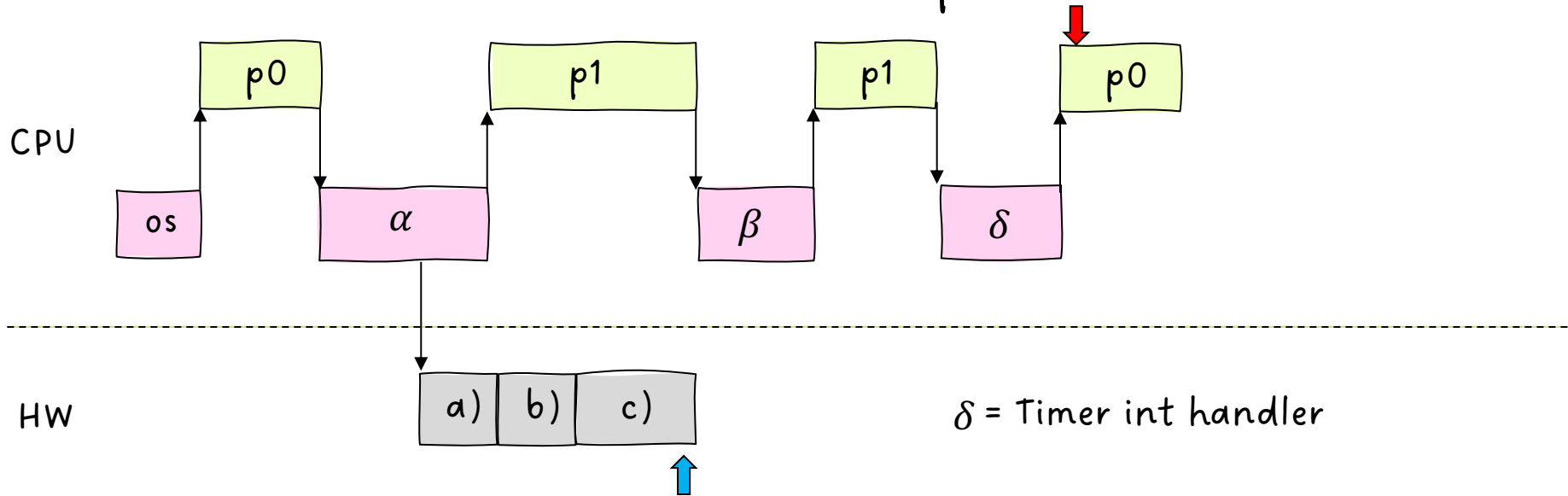
Page Table of p0

p	f.	v/i
p	L	v
q	M	v



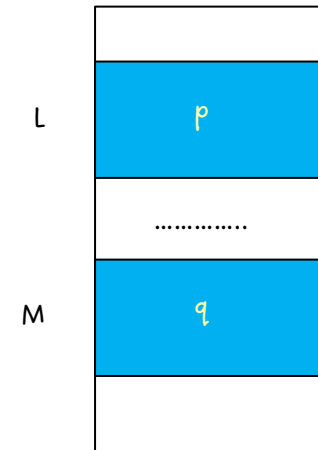
Physical address space

12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

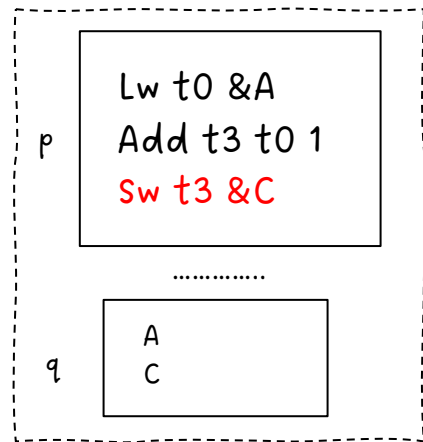
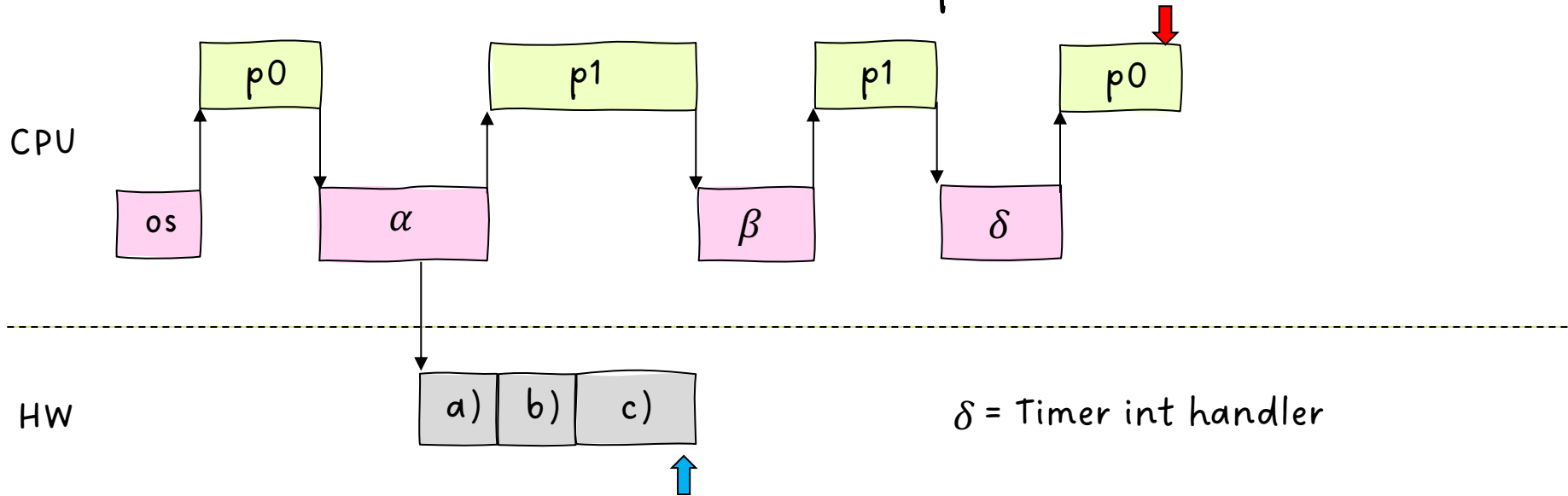


Page Table of p0

p	f.	v/i
p	L	v
q	M	v

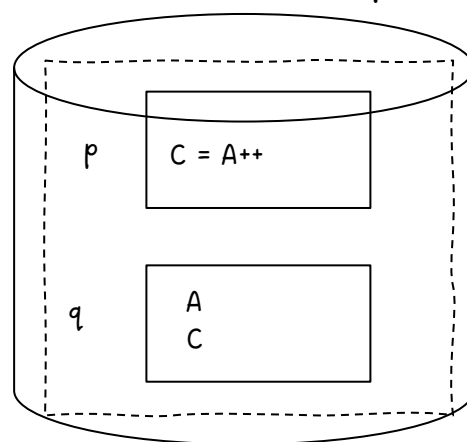


12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction



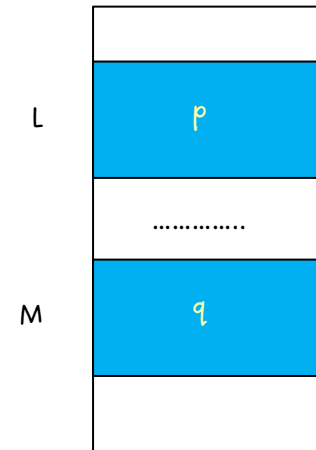
Virtual address space

Executable file of p0



Page Table of p0

p	f.	v/i
p	L	v
q	M	v



Physical address space

Performance of Demand Paging

- Three major activities
 - Service the interrupt – careful coding means just several hundred instructions needed
 - Read the page – lots of time
 - Restart the process – again just a small amount of time
- Page Fault Rate $0 \leq p \leq 1$
 - if $p = 0$ no page faults
 - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)
 - EAT = $(1 - p)$ x memory access
 - + p (page fault overhead
 - + swap page out
 - + swap page in)

Demand Paging Example

- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- $EAT = (1 - p) \times 200 + p (8 \text{ milliseconds})$
 $= (1 - p) \times 200 + p \times 8,000,000$
 $= 200 + p \times 7,999,800$
- If one access out of 1,000 causes a page fault, then
EAT = 8.2 microseconds.
This is a slowdown by a factor of 40!!
- If want performance degradation < 10 percent
 - $220 > 200 + 7,999,800 \times p$
 $20 > 7,999,800 \times p$
 - $p < .0000025$
 - < one page fault in every 400,000 memory accesses

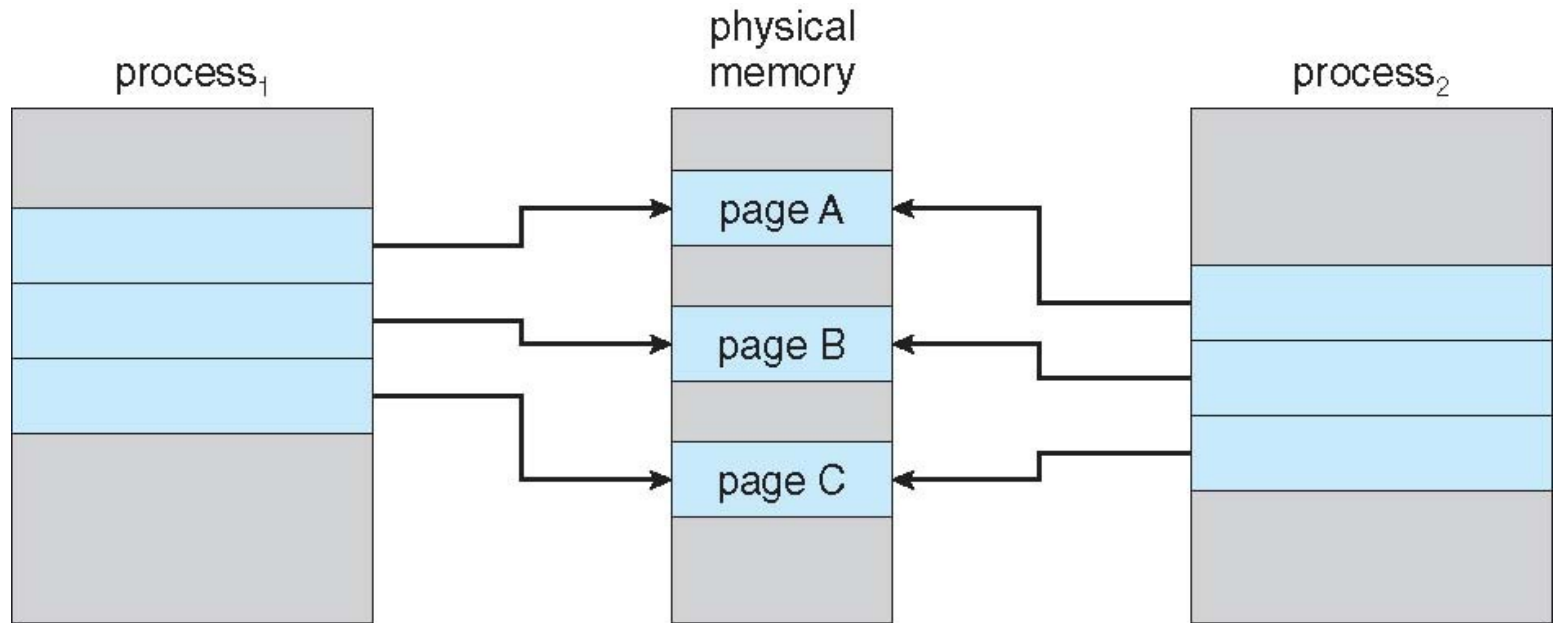
Demand Paging Optimizations

- Swap space I/O faster than file system I/O even if on the same device
 - Swap allocated in larger chunks, less management needed than file system
- Copy entire process image to swap space at process load time
 - Then page in and out of swap space
 - Used in older BSD Unix
- Demand page in from program binary on disk, but discard rather than paging out when freeing frame
 - Used in Solaris and current BSD
 - Still need to write to swap space
 - ▶ Pages not associated with a file (like stack and heap) – **anonymous memory**
 - ▶ Pages modified in memory but not yet written back to the file system
- Mobile systems
 - Typically don't support swapping
 - Instead, demand page from file system and reclaim read-only pages (such as code)

Copy-on-Write

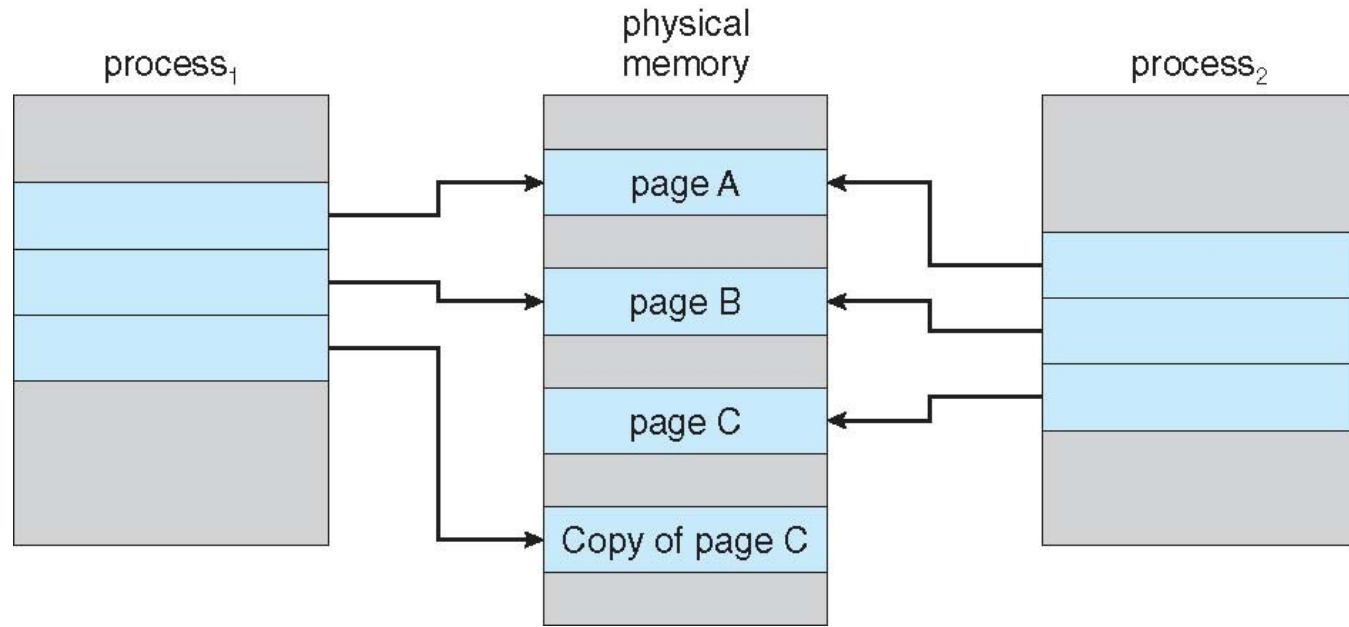
- **Copy-on-Write** (COW) allows both parent and child processes to initially *share* the same pages in memory
 - If either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation as only modified pages are copied
- In general, free pages are allocated from a **pool** of **zero-fill-on-demand** pages
 - Pool should always have free frames for fast demand page execution
 - ▶ Don't want to have to free a frame as well as other processing on page fault
 - Why zero-out a page before allocating it?
- `vfork()` variation on `fork()` system call has parent suspend and child using copy-on-write address space of parent
 - Designed to have child call `exec()`
 - Very efficient

Before Process 1 Modifies Page C



กำหนดให้ shared pages มีคุณสมบัติเป็น Write-protected คือห้ามเขียน

After Process 1 Modifies Page C



1. เมื่อ process ใด process หนึ่งเขียน Write-protected ก็จะทำให้เกิด Trap
2. Trap handler จะทำการ Copy Page C แล้วเปลี่ยน Page Table Entry ให้ชี้ไปยัง Page “Copy of page C” ซึ่งไม่ติด write protect
3. หลังจากนั้น P1 สามารถ modify page 3 ของมันได้

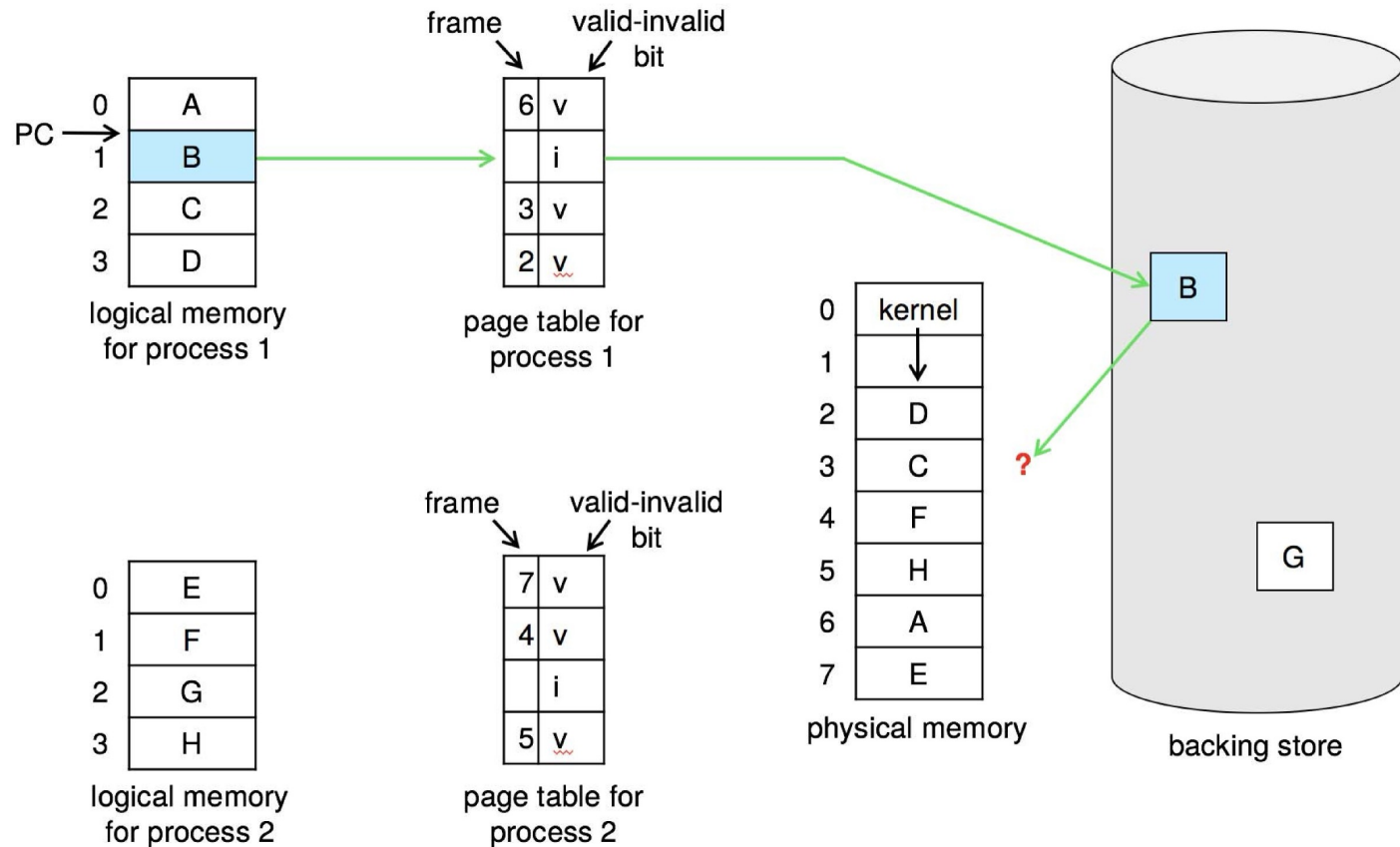
What Happens if There is no Free Frame?

- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- Page replacement – find some page in memory, but not really in use, page it out
 - Algorithm – terminate? swap out? replace the page?
 - Performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

Page Replacement

- Prevent **over-allocation** of memory by modifying page-fault service routine to *include page replacement*
- Use **modify (dirty) bit** to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory
 - ทำให้เกิดการแยกกันอย่างแท้จริง ของ *logical* และ *physical memory*
 - ทำให้เราสามารถ *load program* ขนาดใหญ่บนพื้นที่ *physical memory* จำกัดได้

Need For Page Replacement



Basic Page Replacement

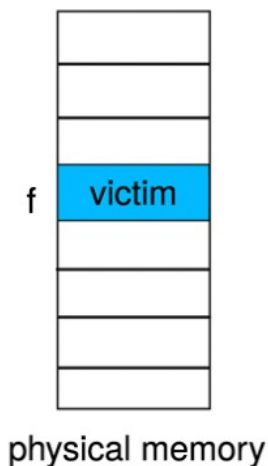
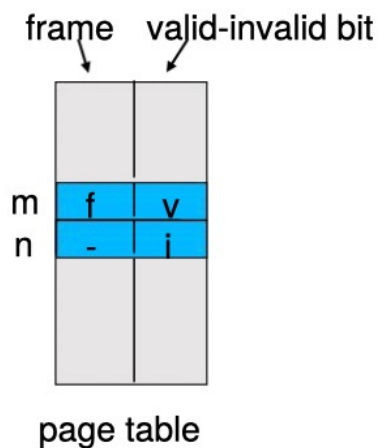
1. Find the location of the desired page on disk
2. Find a free frame:
 - If there is a free frame, use it
 - If there is no free frame, use a page replacement algorithm to select a **victim frame**
 - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Note now **potentially 2 page transfers** for page fault – increasing EAT

Page Replacement

in this example

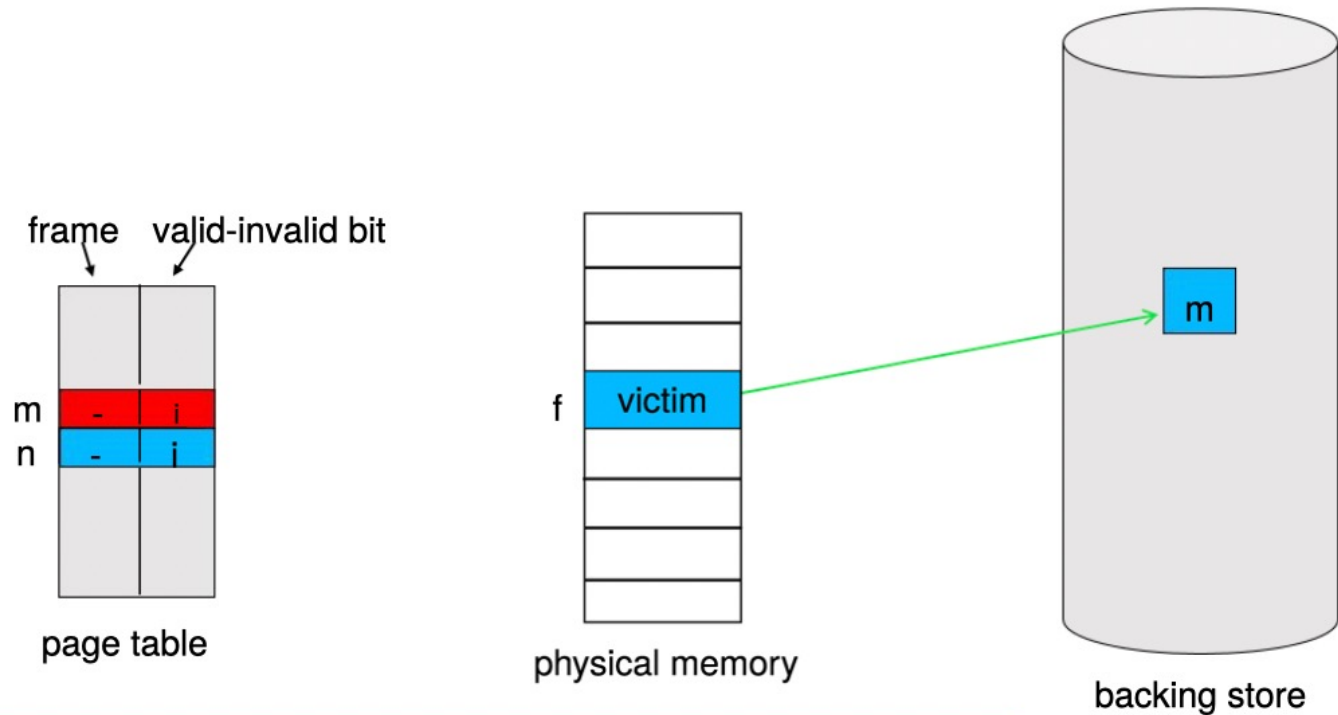
- page (m) is in memory at frame (f) and its page table entry is set to (v)alid
- page (n) is not currently in memory, so it is set to (i)nvalid
- page (n) has been accessed by the current instruction and a page fault has occurred



1. find a frame
 - (a) if a frame is already free, select it
 - (b) if there is no free frame, use a page-replacement algorithm to select a victim frame here there is no free frame so we select (f) as the victim

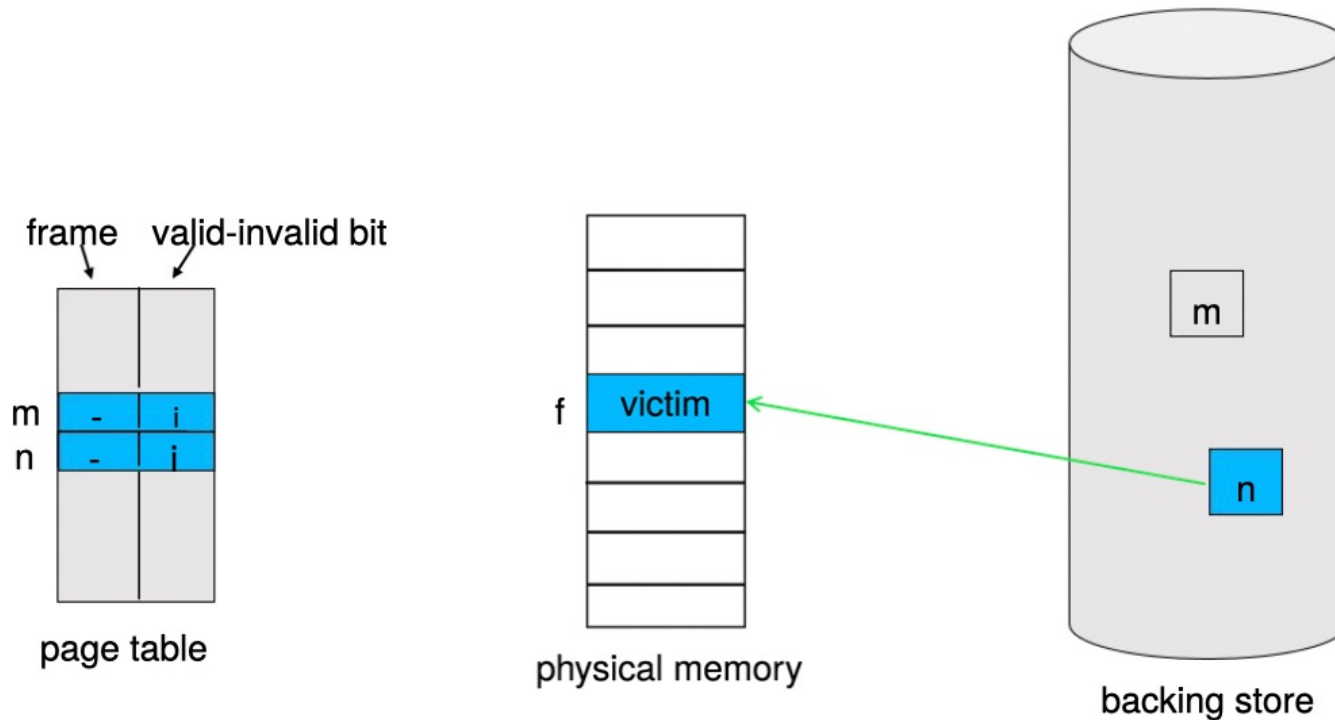


Page Replacement



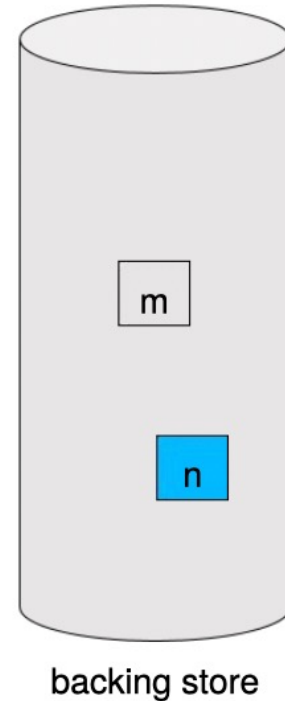
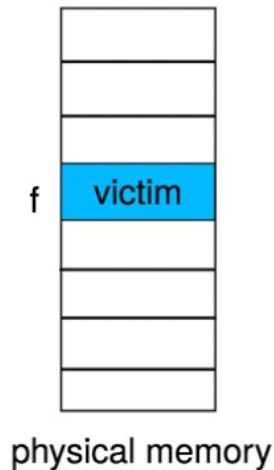
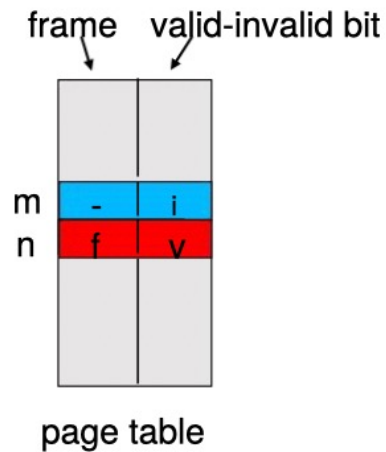
2. (f) is currently holding page (m)
we write the page to swap space on secondary storage
(if necessary), free the frame by marking (m) as invalid

Page Replacement



3. we locate page (n) on backing store and schedule an I/O transfer to DMA the contents into frame (f)

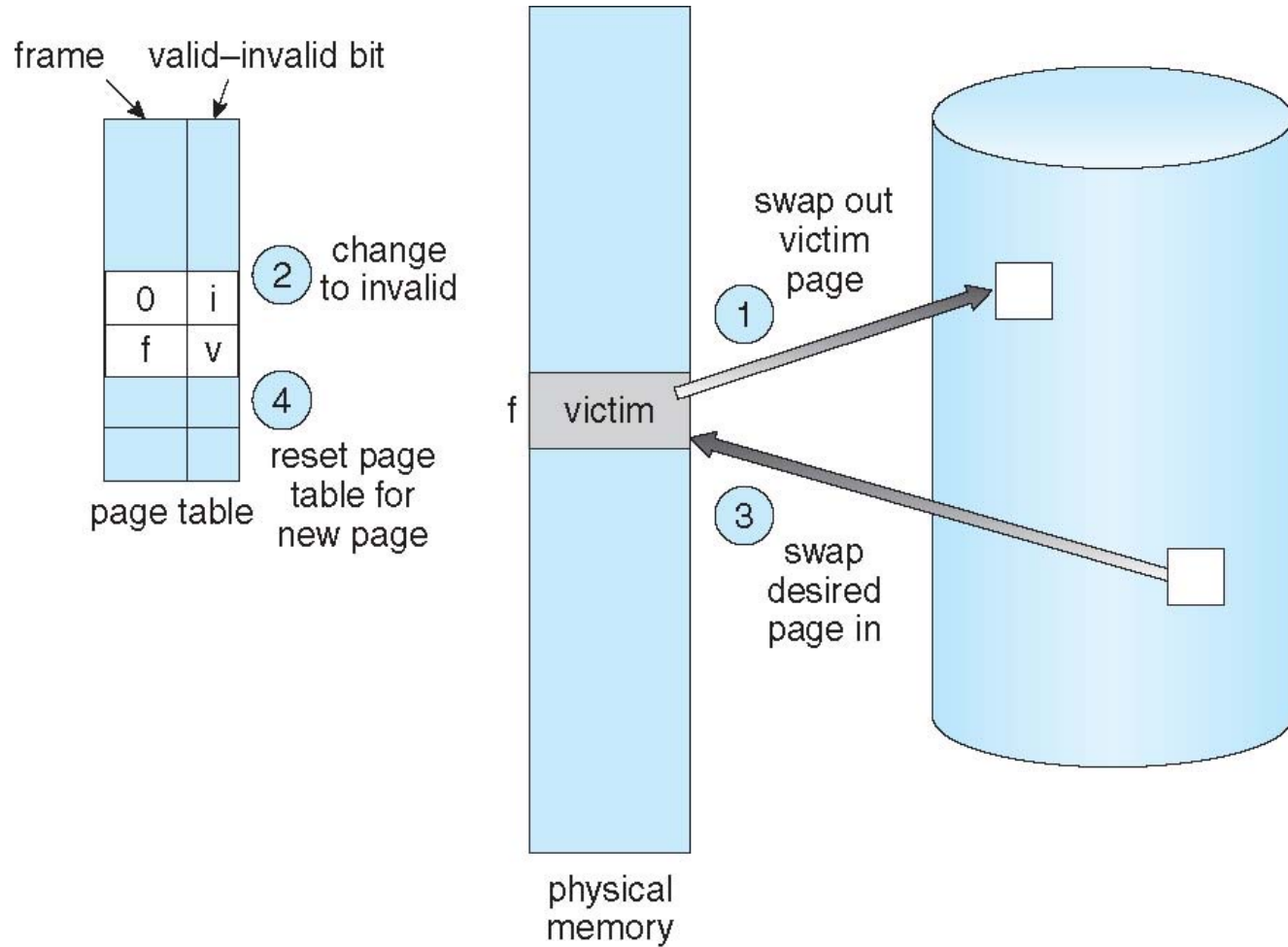
Page Replacement



4. (a) once the transfer is complete, change the page table for (n) to point to the frame and note that it is valid
- (b) the instruction that caused the page fault is restarted, and can now access the data it needs (unless other addresses in the instruction are not currently valid!)



Page Replacement

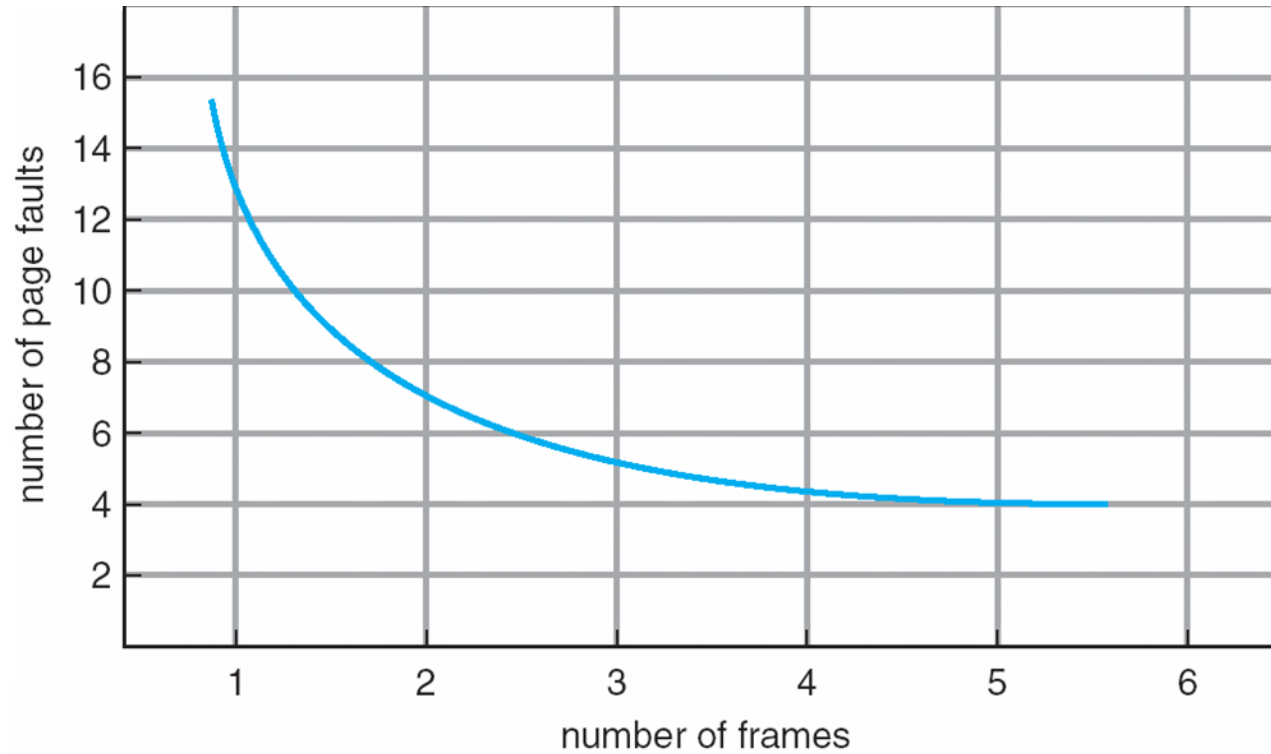


Page and Frame Replacement Algorithms

- **Frame-allocation algorithm** determines
 - How many frames to give each process
 - Which frames to replace
- **Page-replacement algorithm**
 - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
 - String is just page numbers, not full addresses
 - Repeated access to the same page does not cause a page fault
 - Results depend on number of frames available
- In all our examples, the **reference string** of referenced page numbers is

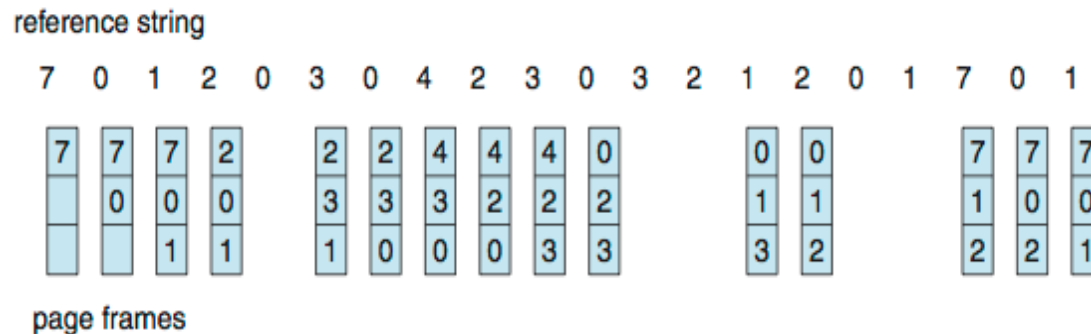
7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

Graph of Page Faults Versus the Number of Frames



First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)



15 page faults

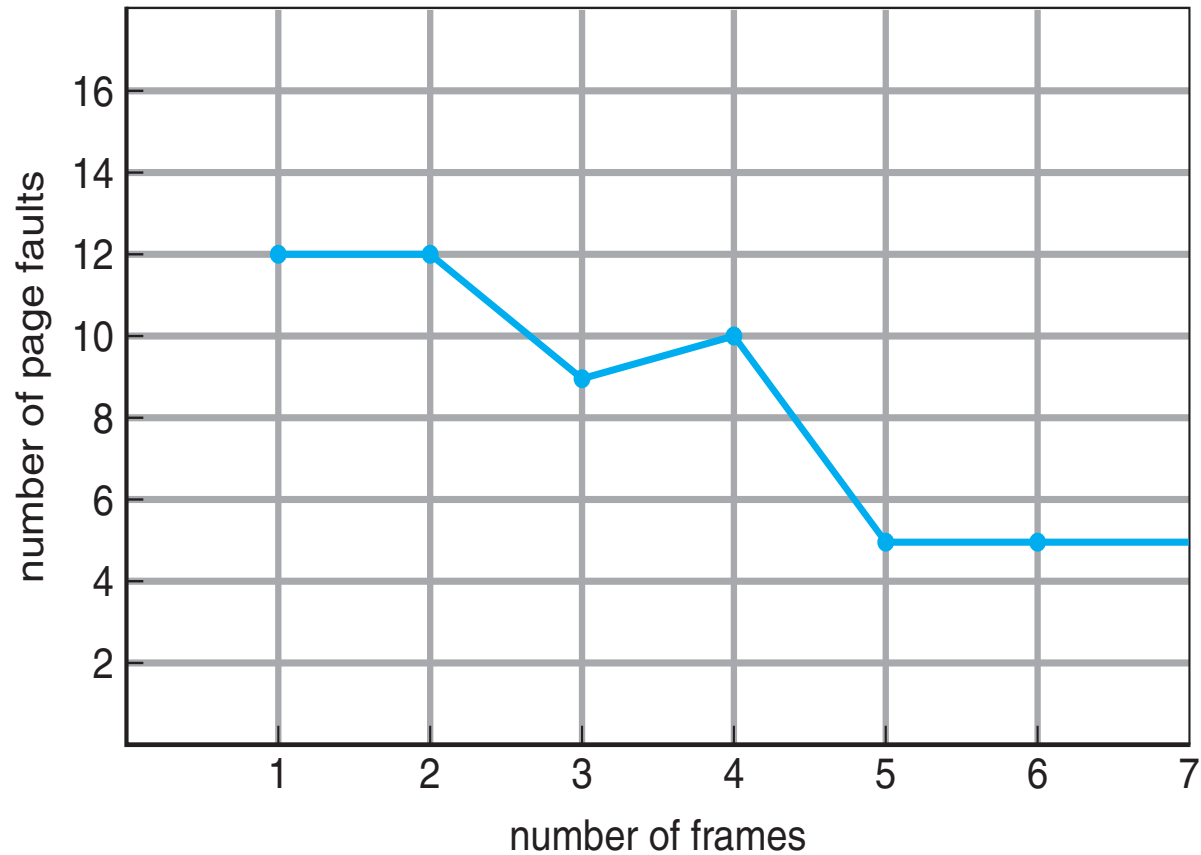
- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
 - Adding more frames can cause more page faults!
 - **Belady's Anomaly**
- How to track ages of pages?
 - Just use a FIFO queue

1,2,3,4,1,2,5,1,2,3,4,5

1	2	3	4	1	2	5	1	2	3	4	5	
<u>1</u>	1	1	4	4	4	5			5	5		
	2	2	2	1	1	1			3	3		
		3	3	3	2	2			2	4		

1	2	3	4	1	2	5	1	2	3	4	5	
1	1	1	1			5	5	5	5	4	4	
	2	2	2			2	1	1	1	1	5	
		3	3			3	3	2	2	2	2	
			4			4	4	4	3	3	3	

FIFO Illustrating Belady's Anomaly



Optimal Algorithm

- Replace page that will not be used for **longest period of time**
 - 9 is optimal for the example
- How do you know this?
 - **Can't read the future**
- Used for measuring how well your algorithm performs

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2		2		2						7		
	0	0	0		0		4		0		0						0		
		1	1		3		3		3		1						1		

page frames

Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2				2		4	4	4	0				1		1		1
	0	0	0				0		0	0	3	3				3		0		0
		1	1				3		3	2	2	2				2		2		7

page frames

- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

LRU Algorithm (Cont.)

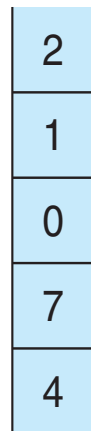
- Counter implementation
 - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
 - When a page needs to be changed, look at the counters to find smallest value
 - ▶ Search through table needed
- Stack implementation
 - Keep a stack of page numbers in a double link form:
 - Page referenced:
 - ▶ move it to the top
 - ▶ requires 6 pointers to be changed
 - But each update more expensive
 - No search for replacement

LRU Algorithm (Cont.)

- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly
- Use Of A Stack To Record Most Recent Page References

reference string

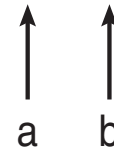
4 7 0 7 1 0 1 2 1 2 7 1 2



stack
before
a



stack
after
b



LRU Algorithm (Cont.)

- Use Of A Stack to Record Most Recent Page References

4	7	0	7	1	0	1	2	1	2	7	1	2
4	7	0	7	1	0	1	2	1	2	7	1	2
	4	7	0	7	1	0	1	2	1	2	7	1
		4	4	0	7	7	0	0	0	1	2	7
				4	4	4	7	7	7	0	0	0
							4	4	4	4	4	4



Sub QUIZ 2

1,2,3,4,5

1	2	3	4	5
<u>1</u>	1	1	4	x
	2	2	2	y
		3	3	z

FIFO

1,2,3,2,5

1	2	3	2	5
<u>1</u>	1	1	1	x
	2	2	2	y
		3	3	z

LRU



LRU Approximation Algorithms

- LRU needs special hardware and still slow
- ดังนั้นจึงต้องใช้การ Approximation
- **Reference bit**
 - With each page associate a bit, initially = 0
 - When page is referenced bit set to 1
 - Replace any with reference bit = 0 (if one exists)
 - ▶ We do not know the order, however

LRU Approximation Algorithms (cont.)

- เนื่องจาก **implementation** ของ **stack** ใช้เวลามาก จึงใช้วิธีประมาณ **LRU** แทน
- Many systems provide some help, however, in the form of a **reference bit**.
- The reference bit for a page is set by the hardware whenever that page is referenced (either a read or a write to any byte in the page).
- Reference bits are associated with each entry **in page table** (เมื่อมีการอ่านหรือเขียน **Page** ฮาร์ดแวร์ **MMU** จะ **set** ค่า **Ref bit** ให้เป็น 1)
 - เริ่มต้น = 0
 - เมื่อ **Page** ถูก **MMU set** เป็น 1 แล้ว **MMU** จะไม่ **set** กลับเป็น 0 ให้
 - **OS** ต้องเป็นผู้ **reset** ค่า **Reference bit** ให้เป็น 0 เอง

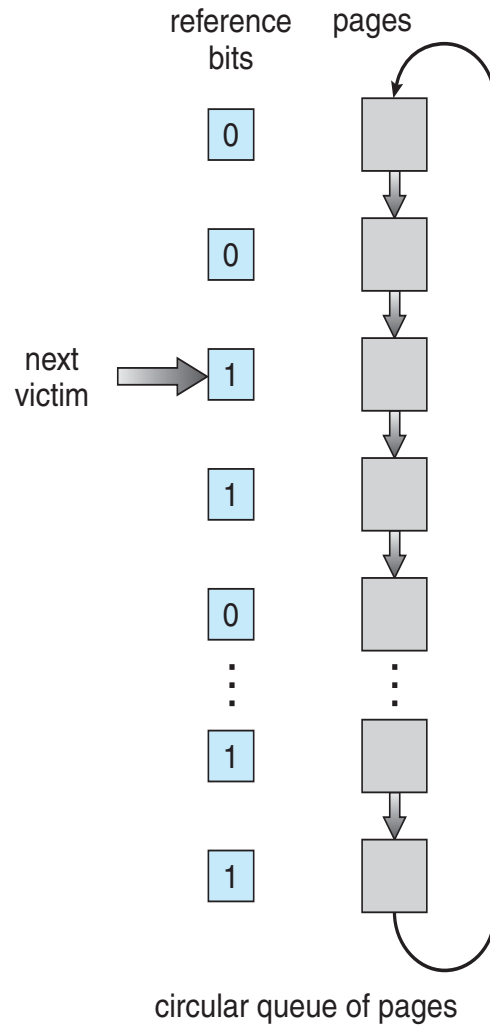


LRU Approximation Algorithms (cont.)

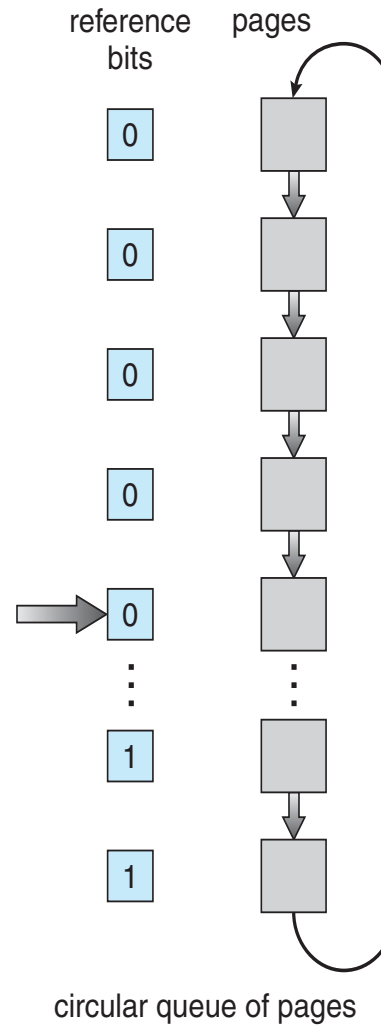
- **Second-chance algorithm**
 - Generally FIFO, plus **hardware**-provided reference bit
 - **Clock** replacement
 - If page to be replaced has
 - ▶ reference bit = 0 -> replace it
 - ▶ reference bit = 1 then:
 - set reference bit 0, leave page in memory
 - replace next page, subject to same rules



Second-chance Algorithm



(a)

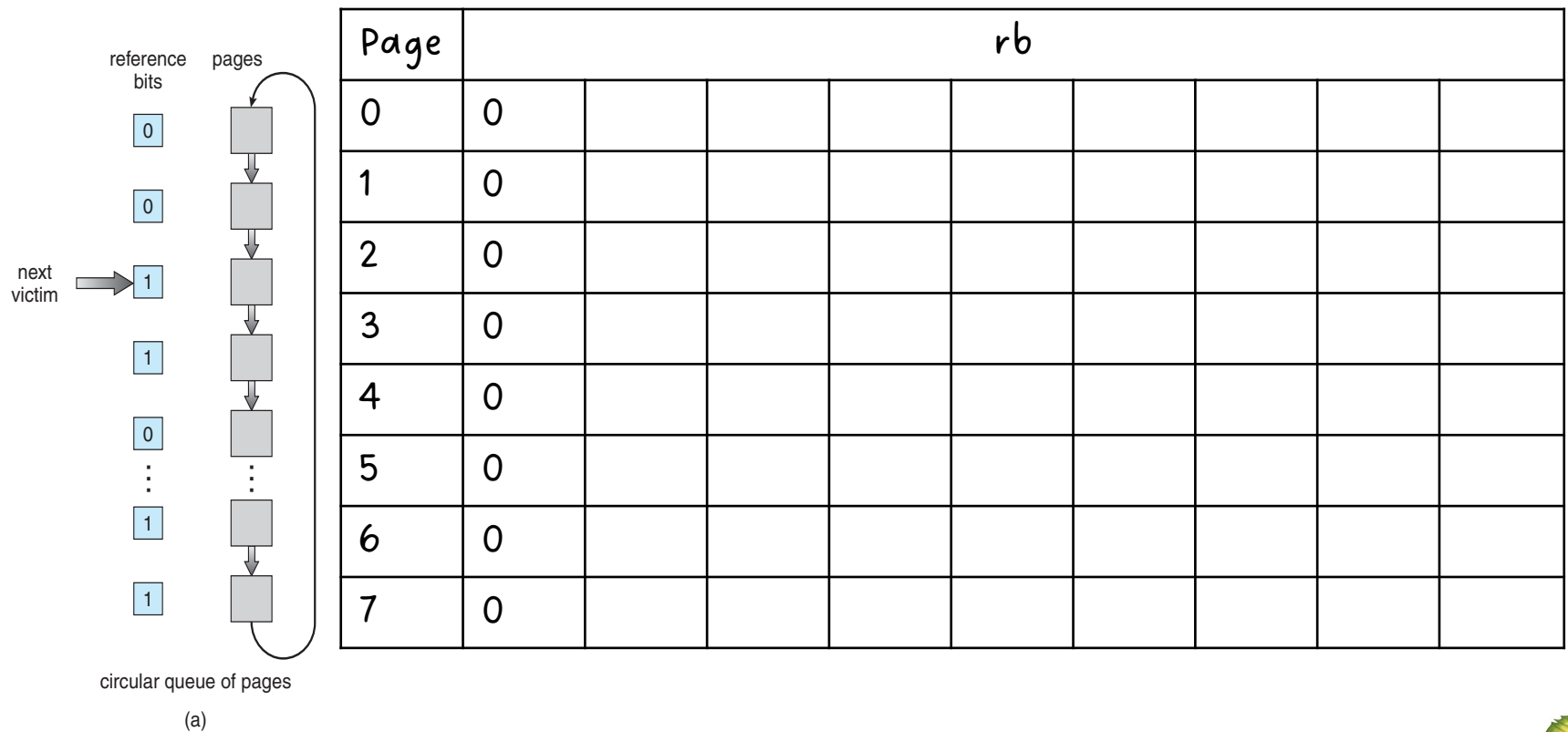


(b)



Second-chance Algorithm

1. Initial มีเพจเต็มทั้ง 8 frames เพื่อความง่าย สมมติว่าเป็น Page 0 ถึง 7 ของ Process หนึ่ง อยู่ใน second chance link list
ให้ rb = reference bit





Second-chance Algorithm

2. CPU refer to page 0,1,2,3,4

0	0	1							
1	0	1							
2	0	1							
3	0	1							
4	0	1							
5	0	0							
6	0	0							
7	0	0							





Second-chance Algorithm

3. Page fault occur

4. Find a victim page from ตำแหน่ง 0 ใน the link list

0	0	1	1						
1	0	1	1						
2	0	1	1						
3	0	1	1						
4	0	1	1						
5	0	0	0						
6	0	0	0						
7	0	0	0						





Second-chance Algorithm

3. Page fault occur
4. Find a victim page from 0 ... 4
OS เปลี่ยน rb ให้เป็น 0
5. พบ victim page ที่ 5

0	0	1	0						
1	0	1	0						
2	0	1	0						
3	0	1	0						
4	0	1	0						
5	0	0	0						
6	0	0	0						
7	0	0	0						





Second-chance Algorithm

6. Page fault handler ดำเนินการอ่าน page เข้าสู่ frame
7. page fault ทำงานเสร็จ ref bit 5 set ให้เป็น 1
8. next victim คือ page ที่ 6 ใน link list

0	0	1	0						
1	0	1	0						
2	0	1	0						
3	0	1	0						
4	0	1	0						
5	0	0	1						
6	0	0	0						
7	0	0	0						





Second-chance Algorithm

7. สมมุติว่ามีการอ้างอิง page 6,7,0,1

8. เกิด page fault อีกครั้ง

0	0	1	1						
1	0	1	1						
2	0	1	0						
3	0	1	0						
4	0	1	0						
5	0	0	1						
6	0	0	1						
7	0	0	1						





Second-chance Algorithm

9. จะได้ victim page = page 2 ใน link list

0	0	1	0						
1	0	1	0						
2	0	1	0						
3	0	1	0						
4	0	1	0						
5	0	0	1						
6	0	0	0						
7	0	0	0						





Second-chance Algorithm

10. page fault ทำงานเสร็จ ref bit 2 set ให้เป็น 1
11. จะได้ next victim page = 3

รอนกว่าจะเกิด **page fault** อีกครั้ง

แต่ถ้า page 3 ถูก refer ก่อนก็อาจถูก set reference bit เป็น 1 และอาจไม่เป็น victim page ในอนาคตก็ได้

0	0	1	0						
1	0	1	0						
2	0	1	1						
3	0	1	0						
4	0	1	0						
5	0	0	1						
6	0	0	0						
7	0	0	0						



Enhanced Second-Chance Algorithm

- Improve algorithm by using **reference bit** and **modify bit** (if available) in concert
 - **reference bit** จะถูก set โดย HW เมื่อมีการอ่านค่าใน **frame**
 - **modify bit** จะถูก set โดย HW เมื่อมีการเปลี่ยนแปลงค่าใน **frame**
- Take ordered pair (reference, modify):
 - (0, 0) neither recently used not modified – best page to replace
 - (0, 1) not recently used but modified – not quite as good, must write out before replacement
 - (1, 0) recently used but clean – probably will be used again soon
 - (1, 1) recently used and modified – probably will be used again soon and need to write out before replacement
- When page replacement called for, use the clock scheme but use the four classes replace page in lowest non-empty class
 - Might need to search circular queue several times



Enhanced Second-Chance Algorithm

- Improve algorithm by using **reference bit** and **modify bit** (if available) in concert
 - **reference bit** จะถูก set โดย HW เมื่อมีการอ่านค่าใน frame
 - **modify bit** จะถูก set โดย HW เมื่อมีการเปลี่ยนแปลงค่าใน frame
- Take ordered pair (reference, modify) (หรือ (r,m)):
 1. จนวน circular list หา (0, 0) ก่อน ถ้าไม่มีก็ไปข้อถัดไป ถ้ามีก็ใช้เป็น victim frame
 2. จนวน circular list หา (0, 1) ถ้าไม่มีก็ไปข้อถัดไป ถ้ามีก็ใช้เป็น victim frame
(ต้อง swap ค่าเก่าใน frame ไปเก็บใน swap space)
 2. จนวน circular list หา (1, 0) ถ้าไม่มีก็ไปข้อถัดไป
 3. จนวน circular list หา (1, 1) ถ้ามีก็ใช้เป็น victim frame
(ต้อง swap ค่าเก่าใน frame ไปเก็บใน swap space)





Enhanced Second-Chance Algorithm

- หลังจากอ่านค่าเข้ามาใน victim frame แล้ว
 1. ถ้า page fault เพราะอ่านค่า ให้ set ค่า (r,m) ของ victim frame เป็น $(1,0)$
 2. ถ้า page fault เพราะเขียนค่า ให้ set เป็น $(0,1)$



Counting Algorithms

- Keep a counter of the number of references that have been made to each page
- เป็นการนับจำนวนการ reference ของแต่ละ page ตั้งแต่หน้า page เข้ามา
 - Not common
- **Least Frequently Used (LFU) Algorithm:**
 - Replaces page with smallest count
 - แทนที่ page ที่ถูกอ้างอิงน้อยที่สุด
- **Most Frequently Used (MFU) Algorithm:**
 - Based on the argument that the page with the smallest count was probably just brought in and has yet to be used
 - แทนที่ page ที่ถูกอ้างอิงมากที่สุด เพราะ page ที่มีค่าอ้างอิงน้อย น่าจะเพิ่งถูก swap in เข้ามา

Page-Buffering Algorithms

- Keep a pool of free frames, always
 - Then frame available when needed, not found at fault time
 - Read page into free frame and select victim to evict and add to free pool
 - When convenient, evict victim
- Possibly, keep list of modified pages
 - When backing store otherwise idle, write pages there and set to non-dirty
- Possibly, keep free frame contents intact and note what is in them
 - If referenced again before reused, no need to load contents again from disk
 - Generally useful to reduce penalty if wrong victim frame selected



Page-Buffering Algorithm (1)

- Page Buffering Algorithm คือกระบวนการเลือกกลุ่ม (pool) ของ free frame ไว้ล่วงหน้าจำนวนหนึ่ง ก่อนที่จะเกิด page fault (pf) เพื่อให้ OS สามารถอ่านค่าเพจใหม่เข้ามาใน free frame ได้เลย
- เลือกเพจจำนวนหนึ่งที่จะนำมาใช้ free frame

frame	Page
0	P0 pg1
1	P0 pg0
2	P1 pg1
3	P0 pg5
4	P1 pg2
5	P1 pg3
6	P0 pg3
7	P0 pg8

1. ก่อนมี pf สร้าง free pool
2. เลือก victim ใส่ใน free pool สาม frames
Frames 7, 2, 4
3. เมื่อเกิด pf สามารถเลือก free frame แล้วอ่าน page ใหม่เข้า free frame ได้เลย ทำให้สามารถทำ pf ได้เร็ว
ในกรณีที่ page ที่อยู่ใน free frame ที่เลือก dirty ต้องเขียนลง swap space ก่อนแล้วจึงอ่าน page ใหม่เข้ามา



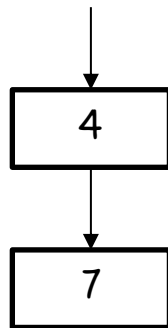


Page-Buffering Algorithm (2)

- Optimization 1: ในระหว่างที่สร้าง pool ของ free frame ก็ให้สร้าง list ของ frame ที่ถูก modified ไว้ด้วย (เรียกว่า modified page list ในภาพ)
- ให้ OS ทอยเขียน frame ใน modified page list ลงสู่ swap space และ set ให้ dirty bit ของ page เหล่านั้น (ใน page table) เป็น 0

frame	Page
0	P0 pg1
1	P0 pg0
2	P1 pg1
3	P0 pg5
4	P1 pg2
5	P1 pg3
6	P0 pg3
7	P0 pg8

modified
pages



1. ก่อนมี pf สร้าง list ของ modify pages สมมุติว่า p1 pg2 และ p0 pg8 ในตัวอย่าง dirty
2. พอ I/O ว่างงาน ก็เขียน swap space
3. update page table ของ p0 และ p1 ให้ dirty bit ของ p1 pg2 p0 pg8 เป็น 0 (จะได้ไม่ต้องเขียนลง swap space ตอน pf)
4. เมื่อเขียน List ของ modified page ลง swap Space แล้ว frame เหล่านี้ก็เป็น free frame ที่อ่านข้อมูลเข้าได้เลย





Page-Buffering Algorithms (3)

- ในกรณีที่มีการอ้างอิงข้อมูลใน frame ที่ถูกทำให้เป็น free frame แล้ว (แต่ยังไม่ได้ถูกเอาไปใช้) page buffering algorithm ก็สามารถคืน frame นั้นให้กับ page table ของ process ที่ใช้งาน frame นั้นก่อนที่จะถูกเลือกเป็น free frame ได้

frame	Page
0	P0 pg1
1	P0 pg0
2	P1 pg1
3	P0 pg5
4	P1 pg2
5	P1 pg3
6	P0 pg3
7	P0 pg8



frame	Page
0	P0 pg1
1	P0 pg0
2	P1 pg1
3	P0 pg5
4	P1 pg2
5	P1 pg3
6	P0 pg3
7	P0 pg8

ตัวอย่าง:

สมมติว่า pg1 ของ P1 ถูกเลือกเป็น free frame แต่ ก่อนที่จะเกิด pf ในระบบ P1 กลับมาใช้งานข้อมูล P1 pg1 อีกครั้งหนึ่ง OS ก็สามารถลบ frame 2 จาก free frame pool ได้เลย แล้ว update page table ของ P1 ว่า page pg1 อยู่ใน frame 2



Allocation of Frames

- Each process needs *minimum* number of frames
- Example: IBM 370 – 6 pages to handle SS MOVE instruction:
 - instruction is 6 bytes, might span 2 pages
 - 2 pages to handle *from*
 - 2 pages to handle *to*
- **Maximum** is total frames in the system
- Two major allocation schemes
 - fixed allocation
 - priority allocation
- Many variations

Fixed Allocation

- **Equal allocation** – For example, if there are 100 frames (after allocating frames for the OS) and 5 processes, give each process 20 frames
 - Keep some as free frame buffer pool
- **Proportional allocation** – Allocate according to the size of process
 - Dynamic as degree of multiprogramming, process sizes change

– s_i = size of process p_i

– $S = \sum s_i$

– m = total number of frames

– a_i = allocation for $p_i = \frac{s_i}{S} \times m$

$$m = 64$$

$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \times 62 \approx 4$$

$$a_2 = \frac{127}{137} \times 62 \approx 57$$

Global vs. Local Allocation

- **Global replacement** – process selects a replacement frame from the set of all frames; one process can take a frame from another
 - But then process execution time can vary greatly
เวลาที่ **process** หนึ่งประมวลผลแต่ละครั้งอาจต่างกัน ขึ้นอยู่กับจำนวน **frame** ที่มีให้ใช้
 - But greater throughput so more common
ใช้แบบนี้กันมาก เพราะทำให้อัตราการจบการประมวลผลของ **process** สูง
- **Local replacement** – each process selects from only its own set of allocated frames
 - More consistent per-process performance
เวลาในการรัน **process** แต่ละครั้งจะสม่ำเสมอและใกล้เคียงกัน
 - But possibly underutilized memory
การใช้งาน **memory** อาจไม่คุ้มค่าเพราะอาจมี **frame** ที่ไม่ถูกใช้งาน
 - อะไรคือข้อดีและข้อเสียของ **global** และ **local replacement**

Reclaiming Pages

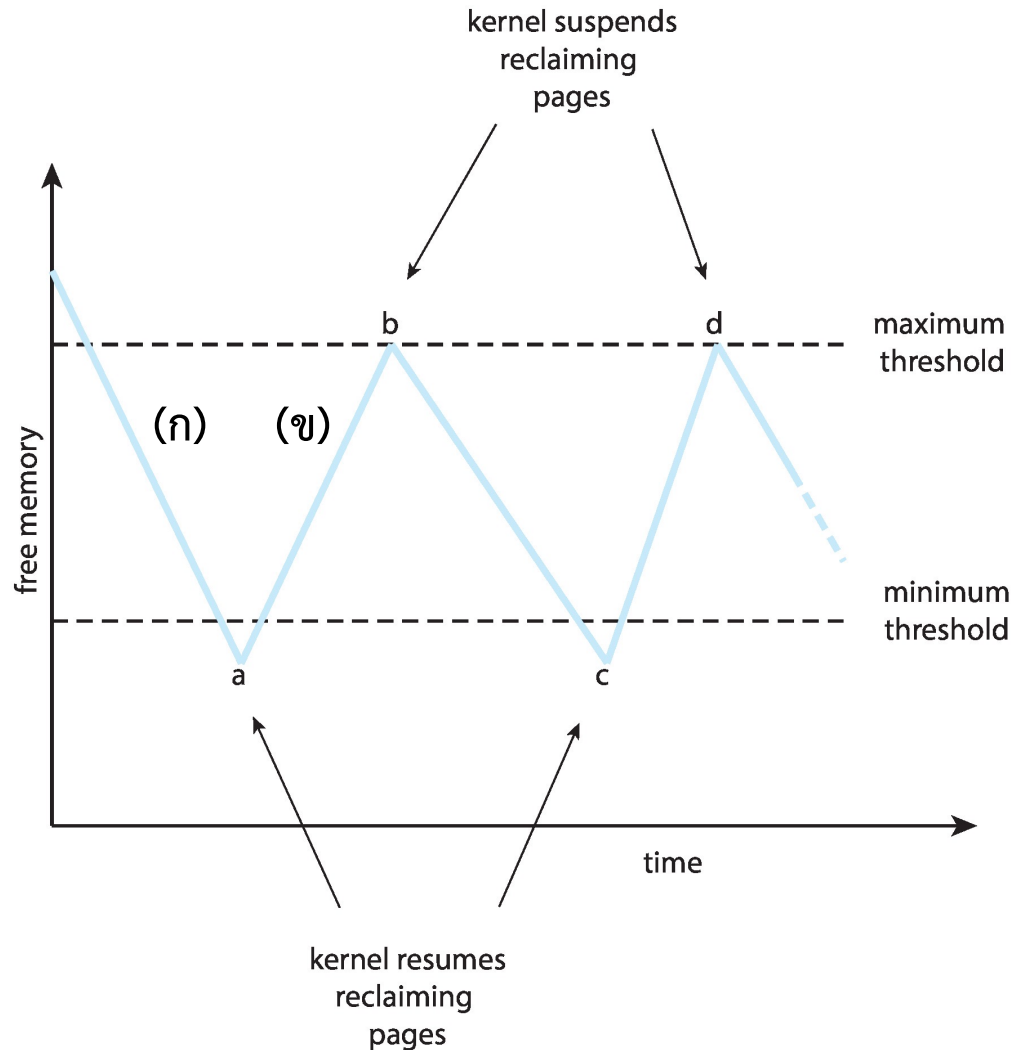
- A strategy to implement **global page-replacement** policy
Reclaiming page คือ นโยบายในการแทนที่ page แบบ global
- OS จะ reclaim pages โดยใช้ **page buffering algorithm** สร้าง free-frame list ไว้ก่อนที่จะมีการแทนที่จริงเมื่อเกิดเหตุการณ์ page fault
- All memory requests are satisfied from the free-frame list, rather than waiting for the list to drop to zero before we begin selecting pages for replacement,
- Page replacement **is triggered** when the list falls below a certain threshold.

Page buffering algor ใช้ นโยบายนี้จะดำเนินการ reclaim page และเพิ่ม frame เข้าไปใน free-frame list เมื่อจำนวน frame ใน list ต่ำกว่าค่า Threshold

- This strategy attempts to ensure there is always sufficient free memory to satisfy new requests.

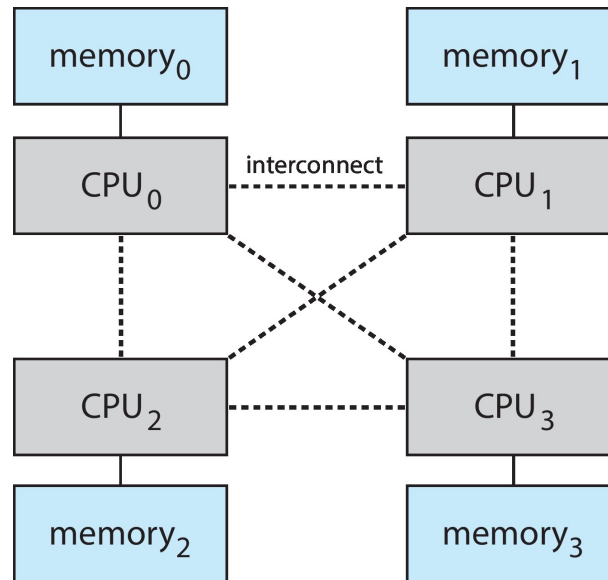


Reclaiming Pages Example



Non-Uniform Memory Access

- So far, we assumed that all memory accessed equally
- Many systems are **NUMA** – speed of access to memory varies
 - Consider system boards containing CPUs and memory, interconnected over a system bus
- NUMA multiprocessing architecture



Non-Uniform Memory Access (Cont.)

- Optimal performance comes from allocating memory “close to” the CPU on which the thread is scheduled
 - And modifying the scheduler to schedule the thread on the same system board when possible
 - Linux มี free frame list แยกสำหรับแต่ละ NUMA node

Thrashing

- If a process does not have “enough” pages, the page-fault rate is
very high
 - Page fault to get page
 - Replace existing frame
 - **But quickly need replaced frame back**
- This leads to:
 - ▶ Low CPU utilization
 - ▶ Operating system thinking that it needs to increase the degree of multiprogramming
 - เมื่อการใช้งาน **CPU** ต่ำเนื่องจากมีการทำ **page fault** บ่อย ทำให้ **OS** เพิ่มจำนวน **process** เพื่อหวังจะเพิ่ม **CPU utilization**
 - เข้าใจผิดอย่างแรง
 - ▶ Another process added to the system
 - เมื่อ เพิ่ม **process** ที่ให้เกิด **page fault** เพิ่มขึ้นอีก



ตัวอย่าง Trashing

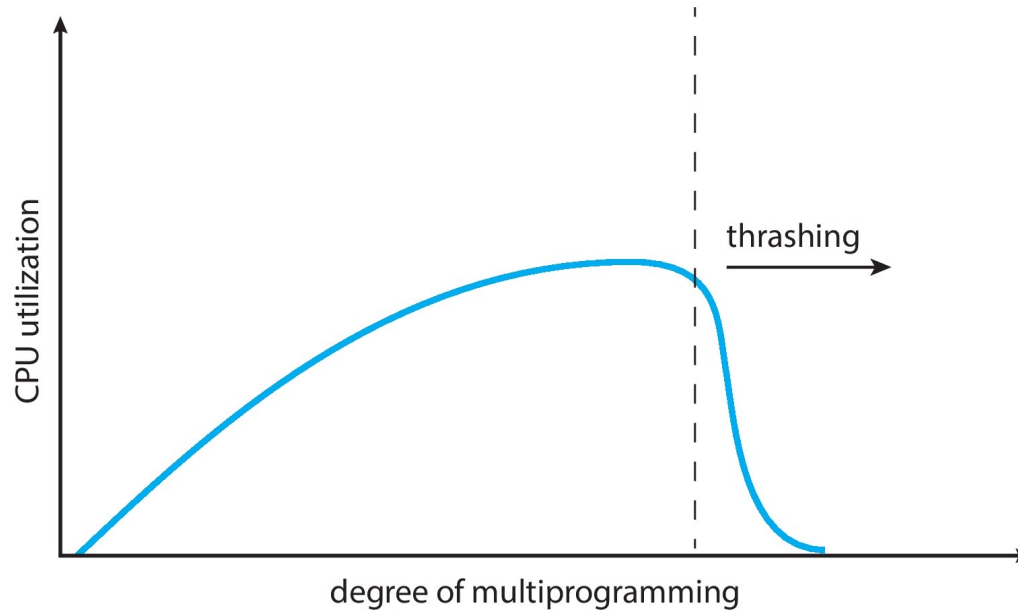
frame	Page
0	P0 pg1
1	P0 pg0
2	P1 pg1
3	P0 pg5
4	P1 pg2
5	P1 pg3
6	P0 pg3
7	P0 pg8

- สมมุติ P0 ต้องการ Pg 9 และ OS โหลดมันเข้ามาใน **frame 2** ส่งผลให้ Pg1 ของ P1 ถูกแทนที่
- แต่หลังจากนั้น ปรากฏว่า P1 ต้องการ Pg 4 ทำให้ OS ต้องโหลดมันขึ้นมาใน **frame 4** ซึ่งทำให้ Pg2 ของ P1 ถูกแทนที่
- แต่หลังจากนั้น P1 กลับต้องการค่าจาก Pg2 ขึ้นมา OS เลยต้องโหลด Pg2 ของ P1 ไปไว้ใน **frame 7**
- การอ้างอิง **memory** ของ **Process** ที่อ้างอิง **page** ที่เพิ่งถูกนำออกไปจาก **physical memory** ซ้ำๆแบบนี้ทำให้เกิด **trashing**



Thrashing (Cont.)

- **Thrashing.** A process is busy swapping pages in and out



Demand Paging and Thrashing

- Why does demand paging work?

Locality model

- Process **migrates** from one locality to another
- Localities may **overlap**

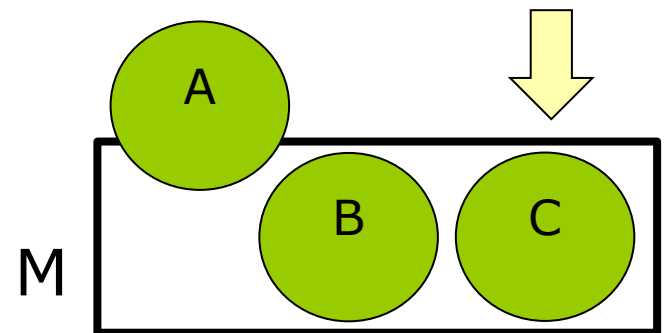
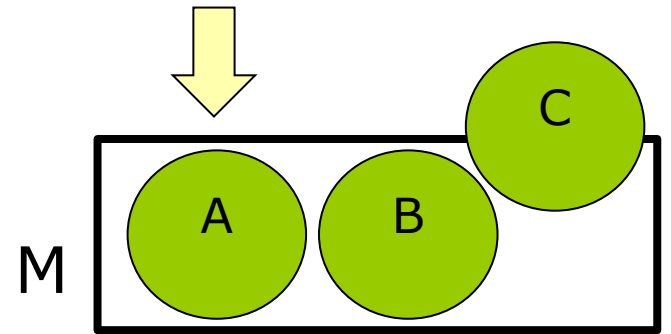
- Why does thrashing occur?

Σ size of locality > total memory size

$D = \Sigma$ size of locality

M = total memory size

Thus, thrashing occurs if $D > M$



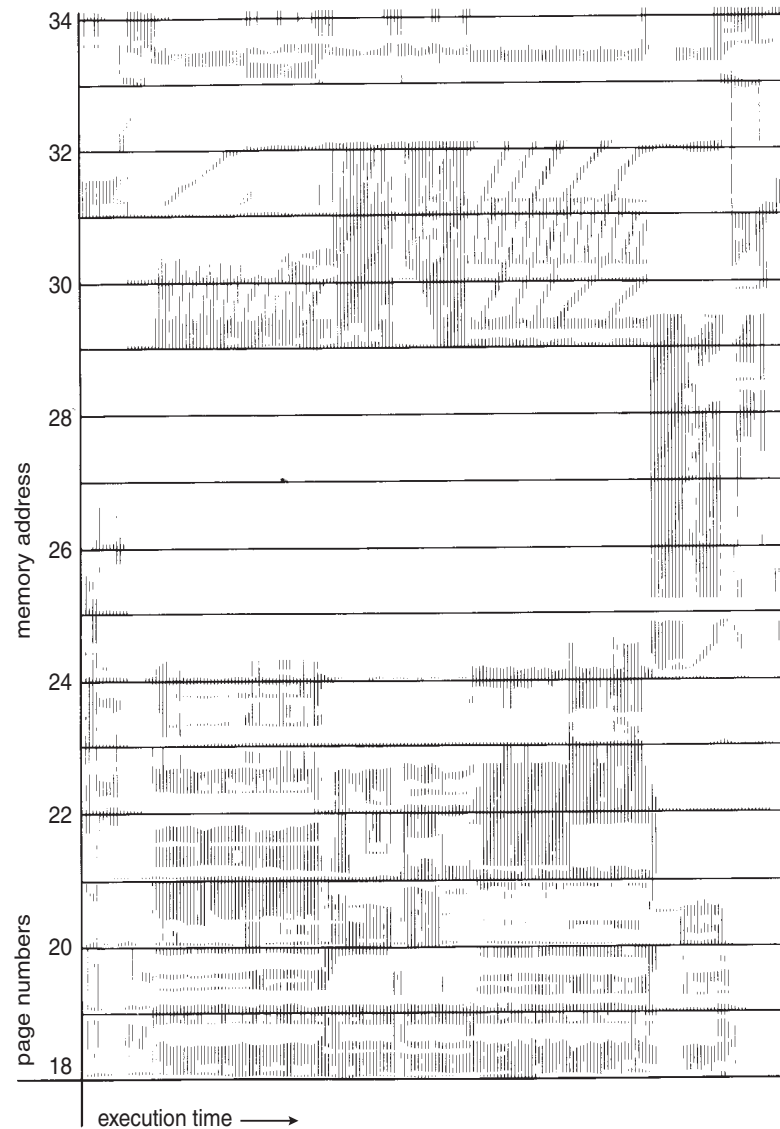


Trashing Solution

- if $D > m \Rightarrow$ **Thrashing**
 - ถ้าขนาดรวมของ **working set** ของทุก **process** มากกว่าขนาดของ **memory** ก็จะเกิด **trashing**
- Solutions: Policy if $D > m$, then suspend or swap out one of the processes
 - สามารถแก้ปัญหานี้ได้โดย หยุดบาง **process** ชั่วคราว หรือ **swap** เนื้อหาของบาง **process** ออกไป
- Mitigation: Limit effects by using **local** or **priority** page replacement
 - **Local** คือ **replace** เฉพาะ **frame** ที่ **Process** (ตนเอง) ใช้อยู่
 - **Priority** คือเลือก **Process** ที่มี **Priority** ต่ำไม่สามารถ **replace** **frame** ของ **Process** ที่มี **Priority** สูงกว่า

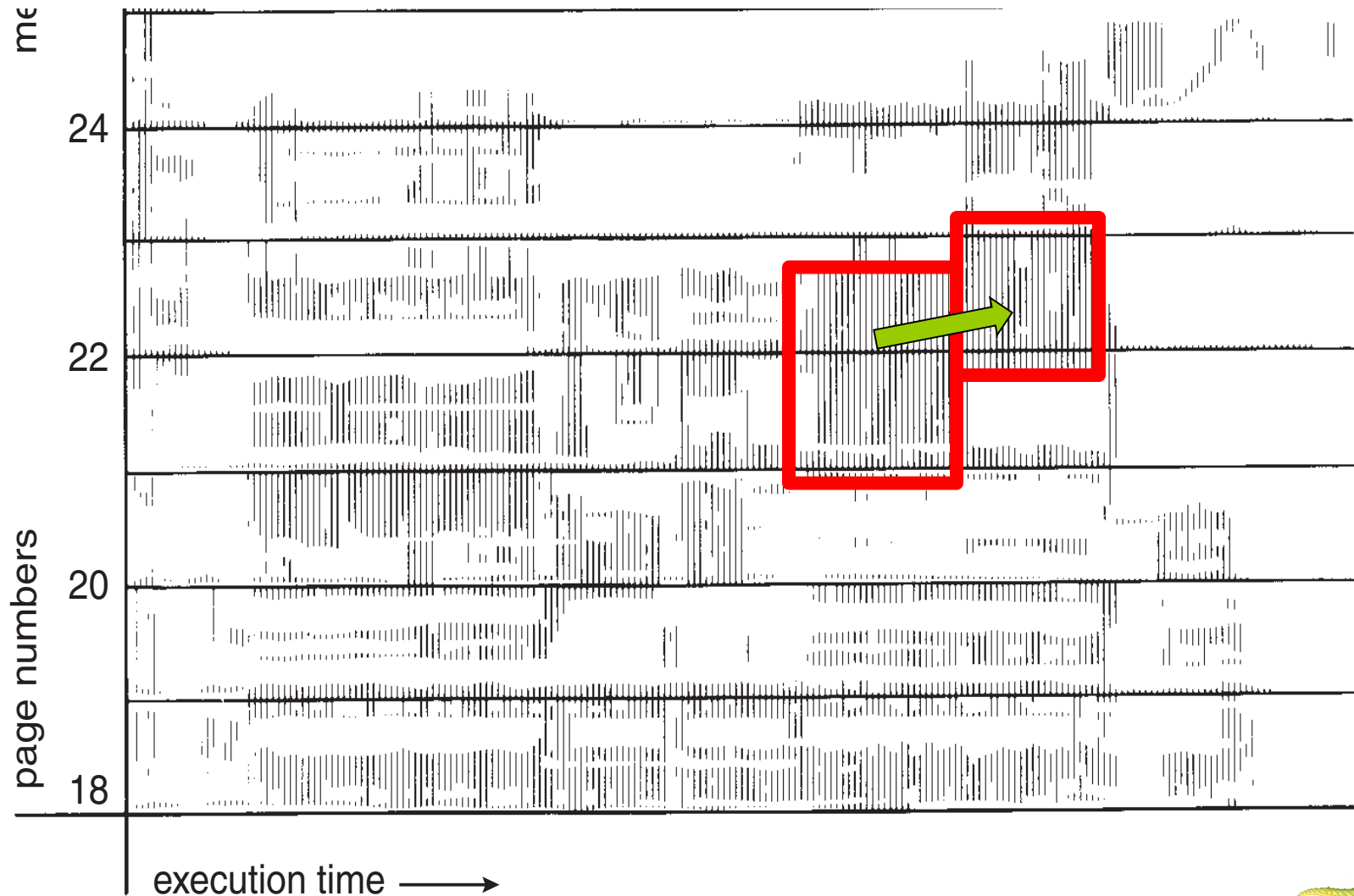


Locality In A Memory-Reference Pattern



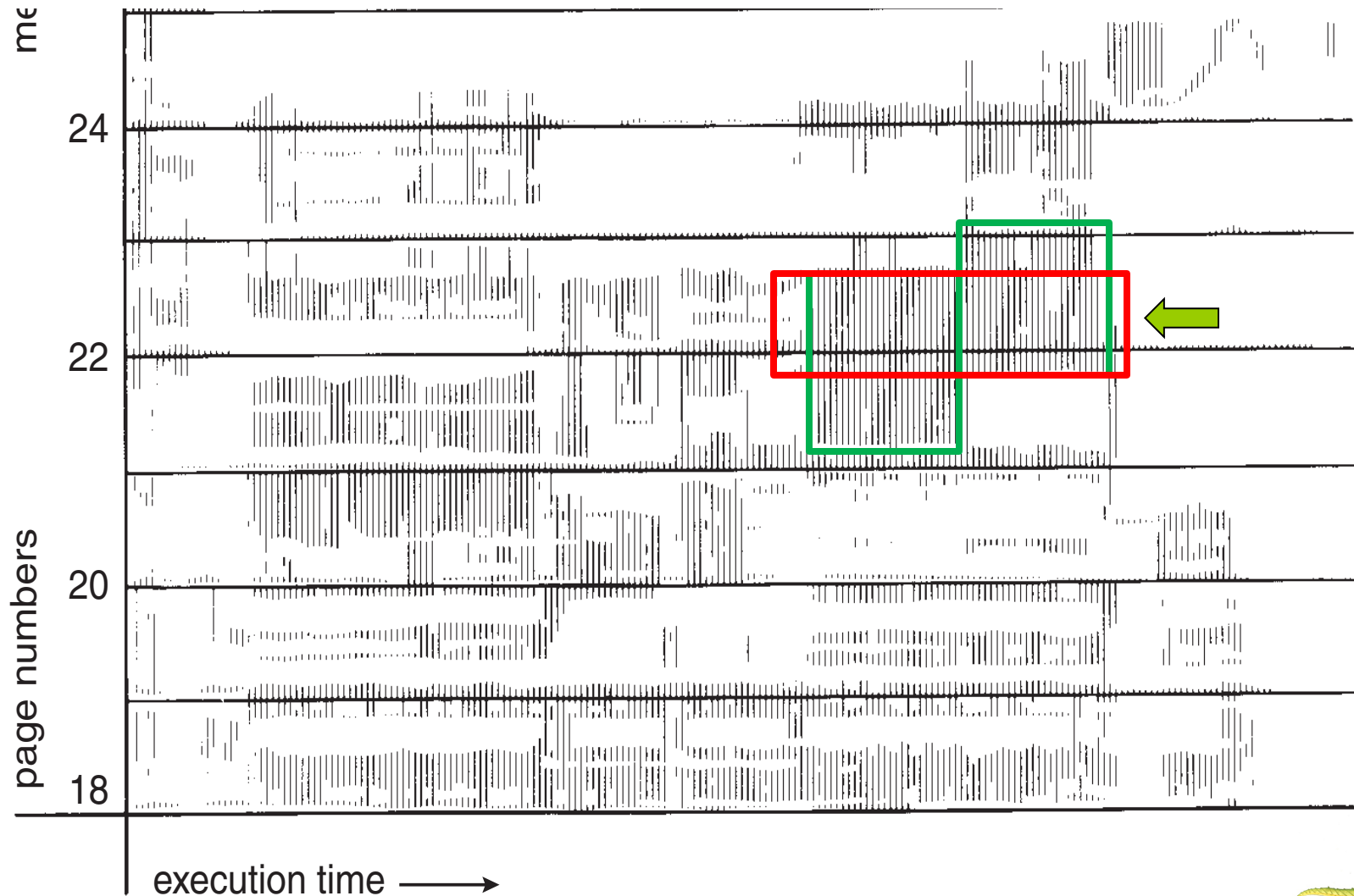


Locality Migration





Locality Overlap



Working-Set Model

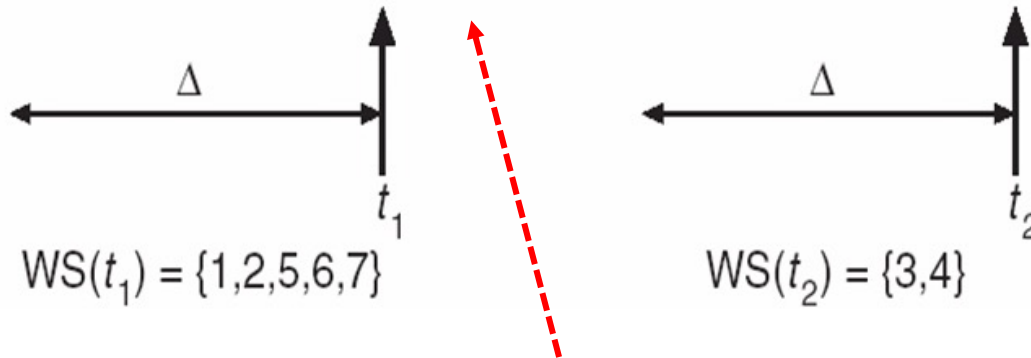
- The working set is an approximation of the program's locality.
- $\Delta \equiv$ working-set window $\equiv$ a fixed number of page references
Example: 10,000 instructions
- WSS_i (working set of Process P_i) = total number of pages referenced in the most recent Δ (varies in time)
 - if Δ too small will not encompass entire locality
 - if Δ too large will encompass several localities
 - if $\Delta = \infty \Rightarrow$ will encompass entire program
- $D = \sum WSS_i \equiv$ total demand frames
 - Approximation of locality

Keeping Track of the Working Set

- การ Keeping Track การเข้าถึง memory ของ process ทุกครั้งที่ CPU เข้าถึง memory เพื่อหา Working set ของ Process นั้นมี Overheads (เวลาที่ hardware ต้องเก็บค่า page number ที่เข้าถึง) มาก ทำให้ CPU ประมวลผลช้า

page reference table

... 2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...



ภาพนี้แสดง Sequence ของ Memory Page access (มีอันดับ) ไม่ใช่ Set ของ Memory pages (ไม่แคร์อันดับ)

Keeping Track of the Working Set

- OS เช่น Linux จึงใช้วิธีประมาณ (Approximate) การเข้าถึง memory ของ Process หนึ่งๆ โดยใช้กระบวนการ interval timer + a reference bit
 - Reference bit คือกลไกของ Hardware ที่ MMU จะ update ค่า Reference bit ของ Page Table เมื่อมีการอ่านหรือเขียนข้อมูลใน Memory page
 - MMU จะ Set ค่า bit อย่างเดียวเท่านั้น มันไม่ลบหรือ Reset ให้
 - ▶ ดังนั้นเมื่อคอมพิวเตอร์ประมวลผลไปเรื่อยๆ ในที่สุด Reference bit ของ Process ก็จะถูก set ในแทบจะทุก page (ซึ่งไม่มีประโยชน์)
 - ▶ OS ต้องออกคำสั่ง Reset



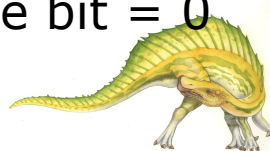
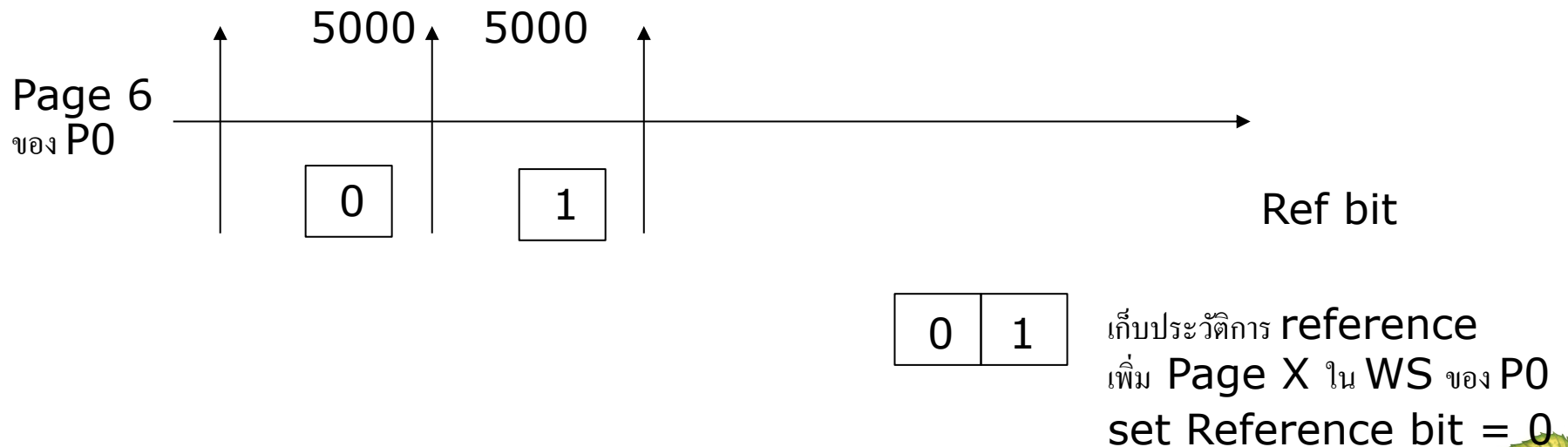
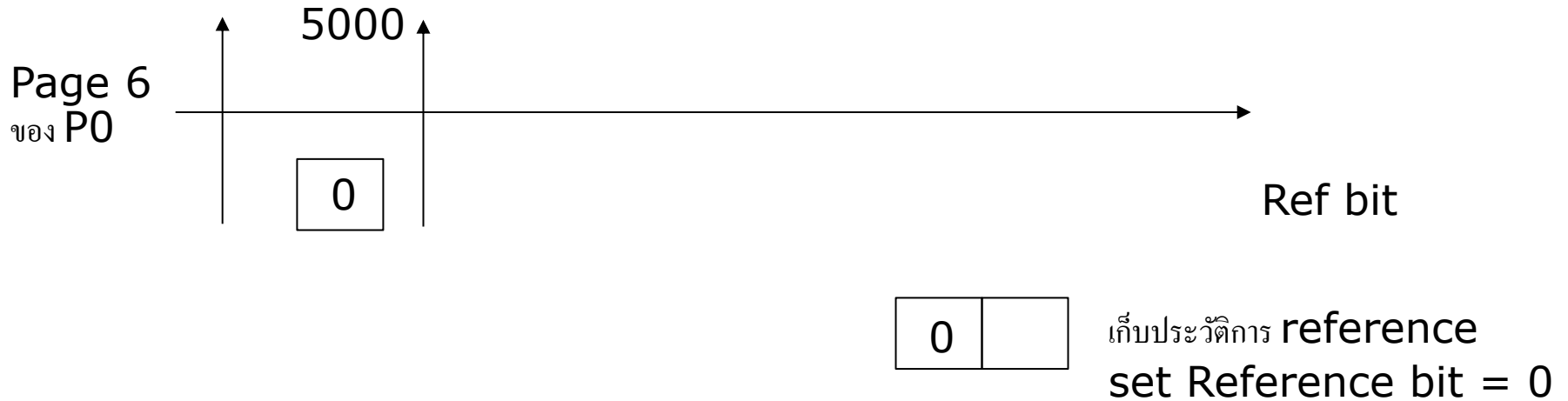
Keeping Track of the Working Set

- ด้วยเหตุนี้ OS จึงใช้ interval timer เพื่อตั้งเวลา ให้ Hardware สร้างสัญญาณ Timer Interrupts เป็นระยะๆ (Interval)
- Example: $\Delta = 10,000$
 - Timer interrupts after every 5000 time units
 - Keep in memory 2 bits for each page (ประวัติการอ้างอิง)
 - Whenever a timer interrupts copy and **sets the values of all reference bits to 0**
 - If one of the bits in memory = 1 $\Rightarrow$ page in working set



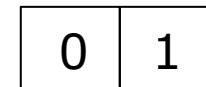
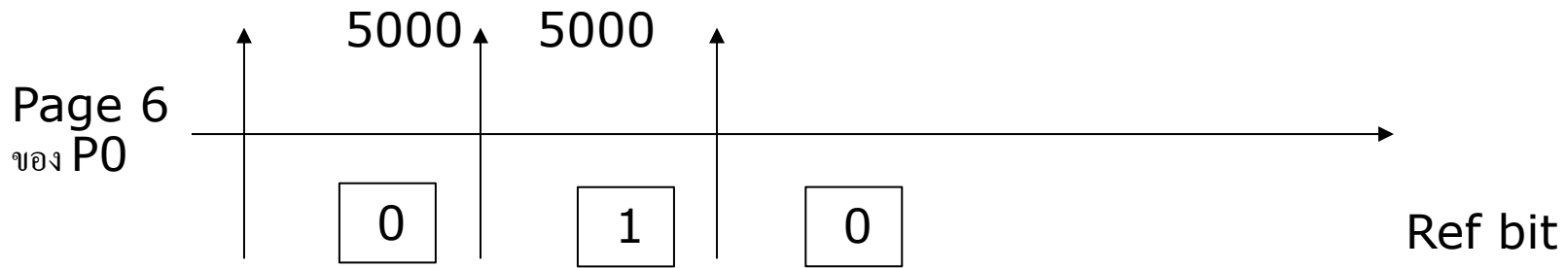


Keeping Track of the Working Set





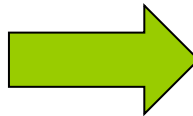
Keeping Track of the Working Set



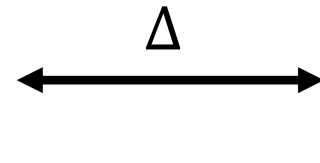
เก็บประวัติการ reference
เพิ่ม Page X ใน WS ของ P0
set Reference bit = 0

สมมติว่า P0 มี 4 pages

Page 6	0	1
Page 7	0	0
Page 8	1	0
Page 9	1	1



..... {8,9} {6,9}



$$\Delta = 10000$$

$$WS(t) = \{6,8,9\}$$





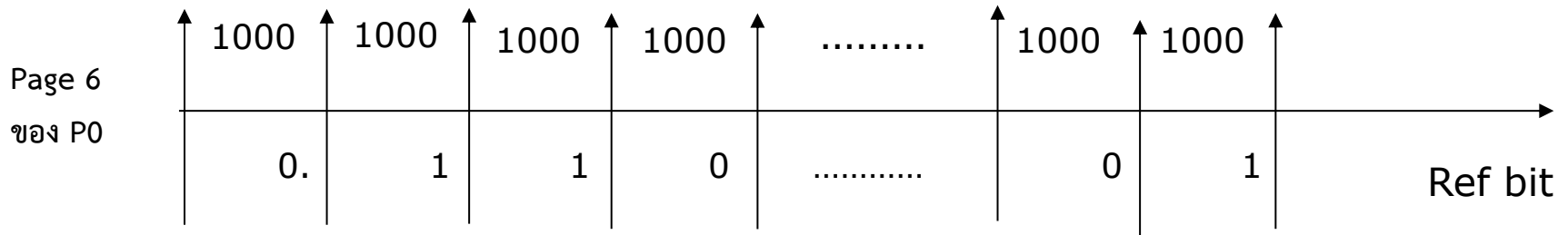
Keeping Track of the Working Set

- วิธีนี้ทำให้ OS ตรวจได้ว่า Page X อยู่ใน Working Set โดยไม่ต้อง track ทุก memory access แต่เก็บข้อมูลทุกช่วงเวลา 5000 time unit
- Why is this not completely accurate?
- Improvement = 10 bits and interrupt every 1000 time units
 - But has higher interrupt overheads
- การเพิ่มจำนวน In Memory bits เป็น 10 และ Interrupt ทุก 1000 time units ทำให้ record การอ้างอิงได้ละเอียดขึ้น ทำให้เราสามารถตรงใจได้ว่าในช่วงเวลา ย่อยใดที่มีการอ้างอิง แต่ OS เสียเวลาดำเนินการตรวจจับมากขึ้น





Keeping Track of the Working Set



Reference bit

Records Page 6

0	1	1	0	1	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---

ได้ Sequence ของ Set ของ Memory Page access

Page 6

0	1	1	0	1	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---

Page 7

1	1	0	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---

Page 8

0	0	1	1	1	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---

Page 9

0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---



7 {6,7} {6,8} 8 {6,8} 7 7 * 8 {6,7}

Δ

$WS(t) = \{6,7,8\}$

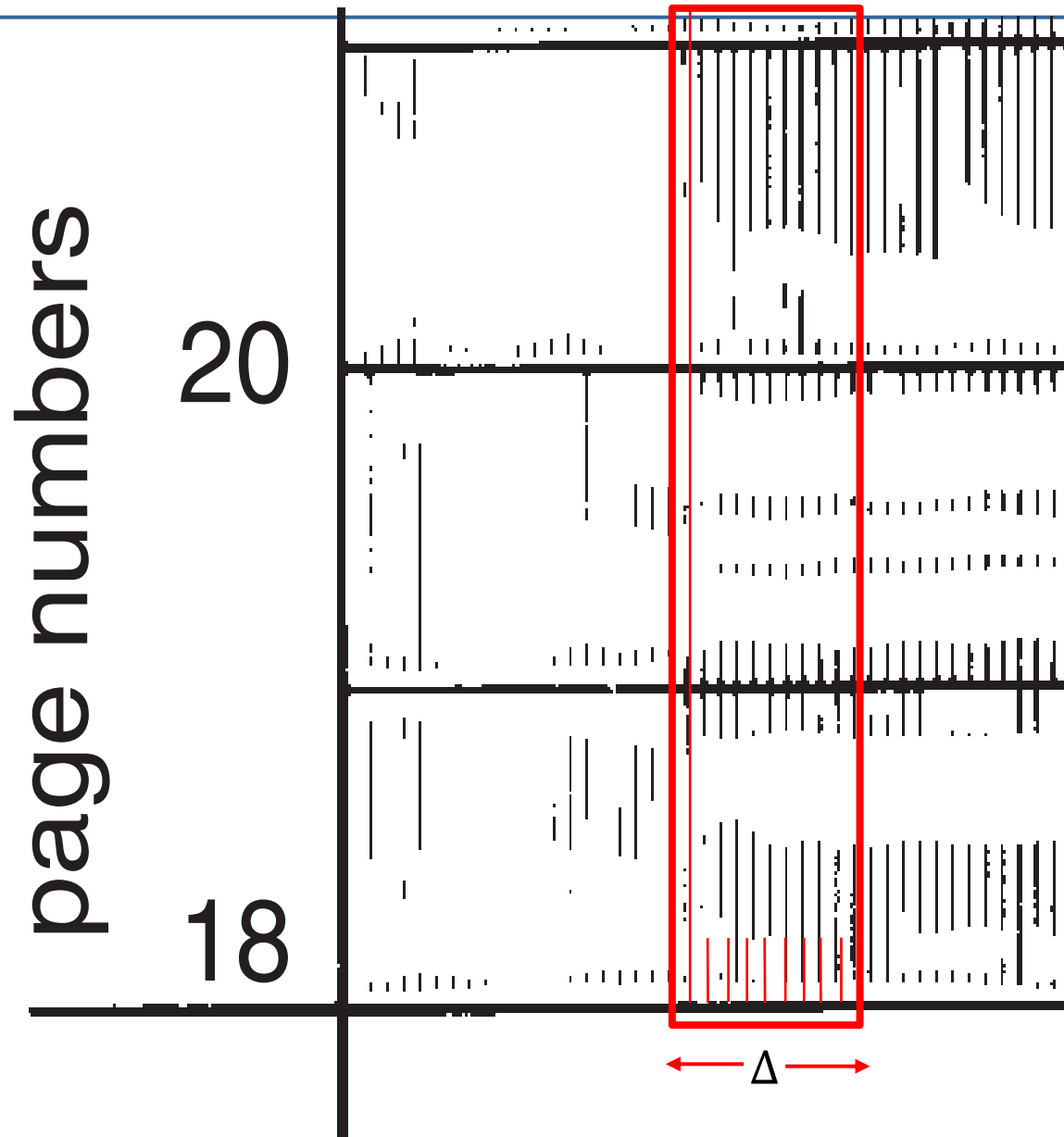
$\Delta = 10000$

* = no ref



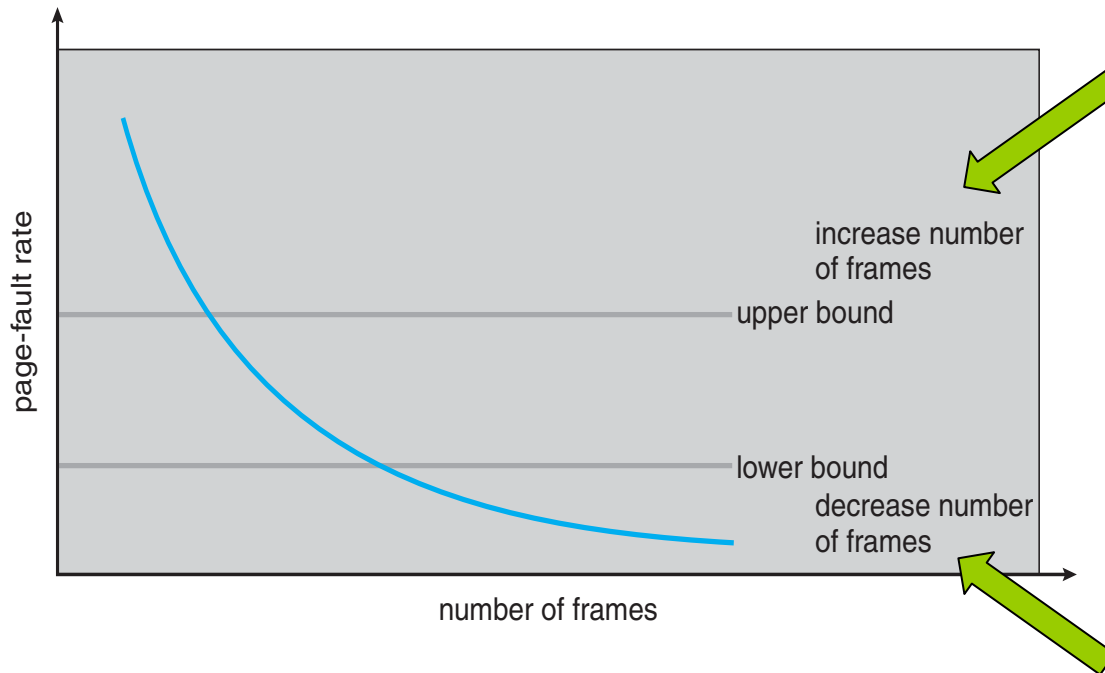


Put it into a Mem Access Chart



Page-Fault Frequency

- More direct approach than WSS
- Establish “acceptable” **page-fault frequency (PFF)** rate and use **local replacement policy**
 - If actual rate too low, process loses frame
 - If actual rate too high, process gains frame



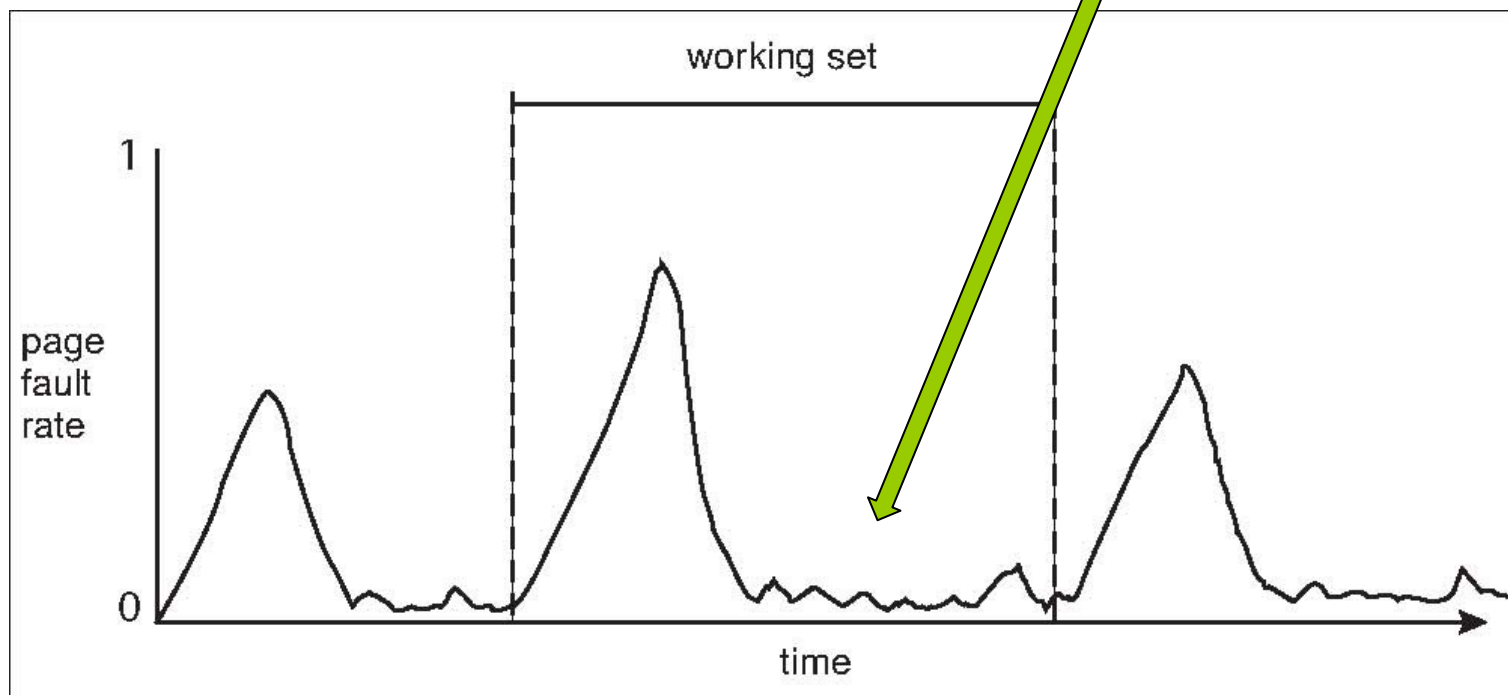
1. ถ้า page fault rate สูง
แสดงว่า frame สำหรับ
process นี้ไม่พอ OS ต้องเพิ่ม
จำนวน frame ให้ process

2. ถ้า page fault rate ต่ำ
แสดงว่า frame สำหรับ
process นี้มากเกินไป OS ต้อง
ลดจำนวน frame ของ process
เอาไปให้ process อื่น

Working Sets and Page Fault Rates

- Direct relationship between working set of a process and its page-fault rate
- Working set changes over time
- Peaks and valleys over time

แสดงความเพียงพอของ
จำนวน frame



Other Considerations

- Prepaging
- Page size
- TLB reach
- Program structure
- I/O interlock and page locking

Prepaging

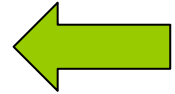
- To reduce the large number of page faults that occurs at process startup
- Prepage all or some of the pages a process will need, before they are referenced
- But if prepaged pages are unused, I/O and memory was wasted
- Assume s pages are prepaged and α of the pages is used
 - Is cost of $s * \alpha$ save pages faults $>$ or $<$ than the cost of prepaging $s * (1 - \alpha)$ unnecessary pages?
 - α near zero $\Rightarrow$ prepaging loses

Page Size

- Sometimes OS designers have a choice
 - Especially if running on custom-built CPU
- Page size selection must take into consideration:
 - Fragmentation
 - Page table size
 - **Resolution**
 - I/O overhead
 - Number of page faults
 - Locality
 - TLB size and effectiveness
- Always power of 2, usually in the range 2^{12} (4,096 bytes) to 2^{22} (4,194,304 bytes)
- On average, growing over time

TLB Reach

- TLB Reach - The amount of memory accessible from the TLB
- $\text{TLB Reach} = (\text{TLB Size}) \times (\text{Page Size})$
- Ideally, the working set of each process is stored in the TLB
 - Otherwise there is a high degree of page faults
- Increase the Page Size
 - This may lead to an increase in fragmentation as not all applications require a large page size
- Provide Multiple Page Sizes
 - This allows applications that require larger page sizes the opportunity to use them without an increase in fragmentation
 - E.g. Huge page in Linux



Program Structure

- Program structure

- `int[128,128] data;`
- Each row is stored in one page
- Program 1

```
for (j = 0; j < 128; j++)  
    for (i = 0; i < 128; i++)  
        data[i,j] = 0;
```

128 x 128 = 16,384 page faults

- Program 2

```
for (i = 0; i < 128; i++)  
    for (j = 0; j < 128; j++)  
        data[i,j] = 0;
```

128 page faults



Program Structure

- Program 1

```
for (j = 0; j < 128; j++)  
    for (i = 0; i < 128; i++)  
        data[i,j] = 0;
```

1 Page

128 x 128 = 16,384 page faults

0,0	0,1	0,2	...	...	...	0,126	0,127
1,0	1,1	1,2	...	...	...	1,126	1,127
2,0	2,1	2,2	...	...	...	2,126	2,127
...	...	...	...	...	...	...	...
126,0	126,1	126,2	...	...	...	126,126	126,127
127,0	127,1	127,2	...	...	...	127,126	127,127

A red line with arrows traces the path of the inner loop (i) for j=0, j=1, and j=2, showing it wraps from the end of the row back to the beginning. A blue arrow points from the text '1 Page' to the row containing indices 2,0 through 2,127, indicating that this entire row fits within a single memory page.





Program Structure

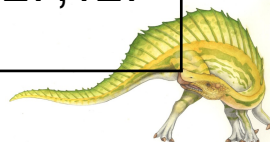
- Program structure
 - Program 2

```
for (i = 0; i < 128; i++)  
    for (j = 0; j < 128; j++)  
        data[i,j] = 0;
```

128 page faults

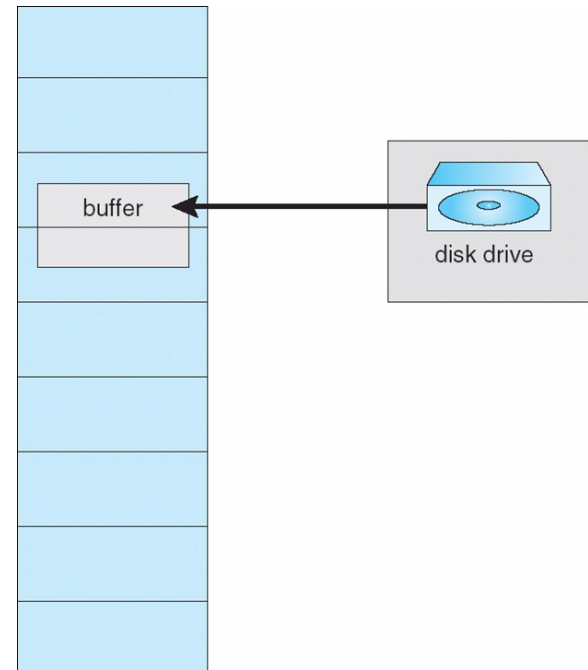
1 Page

0,0	0,1	0,2	...	...	...	0,126	0,127
1,0	1,1	1,2	...	...	...	1,126	1,127
2,0	2,1	2,2	...	...	...	2,126	2,127
126,0	126,1	126,2	...	...	...	126,126	126,127
127,0	127,1	127,2	...	...	...	127,126	127,127



I/O interlock

- **I/O Interlock** – Pages must sometimes be locked into memory
- Consider I/O - Pages that are used for copying a file from a device must be locked from being selected for eviction by a page replacement algorithm
- **Pinning** of pages to lock into memory



End of Chapter 11

